

AI-BASED ROAD ACCIDENT DETECTION AND EMERGENCY RESPONSE SYSTEM USING YOLOv12



**Adeel Danish
(FA22-BSE-001)**

**Syed Ali Ibtissam Bukhari
(FA22-BSE-023)**

**Syed Qosain Hyder Bukhari
(FA22-BSE-025)**

Session 2022-2026

**Department of Software Engineering
Faculty of Engineering & Technology**

**Mirpur University of Science and Technology (MUST), Mirpur
AJK Pakistan**

AI-BASED ROAD ACCIDENT DETECTION AND EMERGENCY RESPONSE SYSTEM USING YOLOv12

By

**Adeel Danish
(FA22-BSE-001)**

**Syed Ali Ibtissam Bukhari
(FA22-BSE-023)**

**Syed Qosain Hyder Bukhari
(FA22-BSE-025)**

A Project Report is submitted in partial fulfillment of
the requirement for the degree of

Bachelor of Science

In

SOFTWARE ENGINEERING

Session 2022-2026

**Department of Software Engineering
Faculty of Engineering & Technology**

**Mirpur University of Science and Technology (MUST), Mirpur
(AJK) Pakistan**

CERTIFICATION

I hereby undertake that this final year project is an original one and no part of this final year project falls under plagiarism. If found otherwise, at any stage, I will be responsible for the consequences.

Student's Name: Adeel Danish	Signature: _____
Registration No.: FA22-BSE-001	Date: _____
Student's Name: Syed Ali Ibtissam Bukhari	Signature: _____
Registration No.: FA22-BSE-023	Date: _____
Student's Name: Syed Qosain Hyder Bukhari	Signature: _____
Registration No.: FA22-BSE-025	Date: _____

Certified that the contents and form of final year project entitled
**“AI-BASED ROAD ACCIDENT DETECTION AND EMERGENCY
RESPONSE SYSTEM USING YOLOv12”**
submitted by
“Adeel Danish, Syed Ali Ibtissam Bukhari, Syed Qosain Hyder Bukhari”
have been found satisfactory for the requirement of the degree.

Supervisor: Engr. Iqra Gillani	Signature: _____
Co-Supervisor (if any):	Signature: _____
External Examiner:	Signature: _____
Chairperson:	Signature: _____

DEDICATION

*To our parents, whose patience, prayers and sacrifices made our education possible,
to our teachers, who taught us to ask better questions,
and to every first responder who runs toward an emergency so that others may live.*

TABLE OF CONTENTS

	Page No.
List of Figures.....	vii
List of Tables.....	ix
List of Abbreviations.....	x
Acknowledgments.....	xi
Abstract.....	xii
CHAPTER 1: INTRODUCTION.....	1
1.1 BACKGROUND.....	1
1.2 MOTIVATION.....	2
1.3 PROBLEM STATEMENT.....	2
1.4 AIMS AND OBJECTIVES.....	3
1.5 SCOPE OF THE PROJECT.....	3
1.6 PROPOSED SOLUTION.....	4
1.7 CONTRIBUTIONS.....	5
1.8 SIGNIFICANCE OF THE STUDY.....	5
1.9 ORGANISATION OF THE REPORT.....	5
CHAPTER 2: LITERATURE REVIEW.....	7
2.1 ROAD TRAFFIC ACCIDENTS AND EMERGENCY RESPONSE.....	7
2.2 SENSOR- AND SMARTPHONE-BASED ACCIDENT DETECTION.....	7
2.3 VISION-BASED ACCIDENT DETECTION.....	8
2.4 DEEP-LEARNING OBJECT DETECTION.....	9
2.4.1 Two-stage detectors.....	9
2.4.2 Single-stage detectors and the YOLO family.....	9
2.4.3 Transformer- and attention-based detectors.....	10
2.4.4 YOLOv12: attention-centric real-time detection.....	10
2.5 VISION-BASED FIRE DETECTION.....	10
2.6 DATASETS AND EVALUATION METRICS FOR DETECTION.....	11
2.7 LANGUAGE MODELS FOR EMERGENCY DECISION SUPPORT.....	11
2.8 SECURE WEB PLATFORMS FOR INCIDENT MANAGEMENT.....	11
2.9 COMPARATIVE ANALYSIS OF RELATED WORK.....	12
2.10 RESEARCH GAP.....	12
CHAPTER 3: REQUIREMENTS ANALYSIS AND METHODOLOGY.....	13
3.1 DEVELOPMENT METHODOLOGY.....	13
3.2 FEASIBILITY STUDY.....	14
3.3 STAKEHOLDERS AND USER CLASSES.....	14
3.4 FUNCTIONAL REQUIREMENTS.....	15
3.5 NON-FUNCTIONAL REQUIREMENTS.....	16

3.6	USE-CASE MODEL.....	17
3.7	DATASET PREPARATION.....	18
3.8	MODEL SELECTION.....	20
3.9	TRAINING CONFIGURATION AND AUGMENTATION.....	20
3.10	EVALUATION METRICS.....	22
3.11	TEMPORAL CONFIRMATION ALGORITHM FOR VIDEO.....	23
3.12	EMERGENCY ASSESSMENT METHODOLOGY.....	24
3.13	TOOLS AND TECHNOLOGIES.....	25
CHAPTER 4: SYSTEM ARCHITECTURE.....		26
4.1	ARCHITECTURAL DESIGN APPROACH.....	26
4.2	HIGH-LEVEL ARCHITECTURE.....	27
4.3	SUBSYSTEM DECOMPOSITION.....	27
4.3.1	S1 – Web client.....	28
4.3.2	S2 – Identity and access.....	28
4.3.3	S3 – Detection.....	28
4.3.4	S4 – Video processing.....	29
4.3.5	S5 – Emergency response.....	29
4.3.6	S6 – Incident management and analytics.....	29
4.3.7	S7 – Persistence and storage.....	29
4.4	DATA-FLOW ARCHITECTURE.....	29
4.5	DEPLOYMENT ARCHITECTURE.....	31
4.6	SECURITY ARCHITECTURE.....	31
4.7	STORAGE ARCHITECTURE.....	32
CHAPTER 5: DETAILED SYSTEM DESIGN.....		33
5.1	COMPONENT DESIGN APPROACH.....	33
5.2	COMPONENT SPECIFICATIONS.....	33
5.3	CLASS DIAGRAM.....	38
5.4	DATABASE DESIGN.....	39
5.5	DYNAMIC BEHAVIOUR.....	41
5.5.1	Authentication sequence.....	41
5.5.2	Image detection sequence.....	42
5.5.3	Video detection sequence.....	43
5.5.4	Emergency agent activity.....	44
5.5.5	State machines.....	45
5.6	API DESIGN.....	46
5.7	USER INTERFACE DESIGN.....	46
CHAPTER 6: IMPLEMENTATION AND TESTING.....		47
6.1	DEVELOPMENT ENVIRONMENT.....	47
6.2	IMPLEMENTATION OF MODULES.....	47
6.2.1	Model training.....	47
6.2.2	Back-end application.....	48
6.2.3	Inference engine.....	48

6.2.4	Video pipeline.....	48
6.2.5	Alerts and emergency agent.....	48
6.2.6	Incident reports.....	49
6.2.7	User interface.....	49
6.3	SECURITY IMPLEMENTATION.....	54
6.4	TESTING STRATEGY.....	55
6.5	AUTOMATED TEST RESULTS.....	55
6.6	SYSTEM TEST CASES.....	56
6.7	NON-FUNCTIONAL TESTING.....	58
6.8	DEPLOYMENT PLAN.....	59
CHAPTER 7: RESULTS AND DISCUSSION.....		60
7.1	TRAINING CONVERGENCE.....	60
7.2	QUANTITATIVE DETECTION RESULTS.....	61
7.3	ERROR ANALYSIS.....	62
7.4	QUALITATIVE RESULTS.....	63
7.5	RUNTIME PERFORMANCE.....	64
7.6	END-TO-END SYSTEM EVALUATION.....	65
7.7	DISCUSSION.....	67
7.8	THREATS TO VALIDITY.....	68
CHAPTER 8: CONCLUSION AND FUTURE WORK.....		69
8.1	SUMMARY OF THE WORK.....	69
8.2	SUMMARY OF RESULTS.....	69
8.3	CONTRIBUTIONS.....	70
8.4	STRENGTHS OF THE SYSTEM.....	70
8.5	LIMITATIONS.....	70
8.6	COST INFORMATION.....	71
8.7	REFLECTIONS ON THE DESIGN PROCESS.....	72
8.8	FUTURE WORK.....	72
8.9	CONCLUDING REMARKS.....	73
REFERENCES.....		74
APPENDIX A: API REFERENCE.....		76
APPENDIX B: CONFIGURATION PARAMETERS.....		78
APPENDIX C: INSTALLATION AND USER GUIDE.....		79
APPENDIX D: PROJECT TIMELINE AND WORK DIVISION.....		80

LIST OF FIGURES

Figure No.	Page No.
1.1 Conceptual overview of the proposed accident detection and emergency response pipeline.....	4
2.1 Evolution of deep-learning object detectors relevant to this project.....	9
3.1 Iterative and incremental development process and the five increments of the project.....	13
3.2 Use-case diagram of the AI accident detection and emergency response system.....	17
3.3 Dataset composition: (a) annotated object instances per class and split; (b) images per split.....	19
3.4 Label statistics produced by Ultralytics for the training set: class counts, box positions and box sizes.....	20
3.5 Model-training pipeline from the merged dataset to the exported model bundle.....	21
3.6 An augmented training batch showing mosaic composition, colour jitter and ground-truth boxes (0 = fire, 1 = moderate, 2 = severe).....	22
3.7 Flowchart of frame sampling and three-frame temporal confirmation in video processing.....	24
4.1 Layered architecture of the AI accident detection and emergency response system.....	27
4.2 Context-level data-flow diagram (DFD level 0).....	30
4.3 Level-1 data-flow diagram showing the main processes and data stores.....	30
4.4 Deployment diagram of the Docker Compose stack.....	31
5.1 UML class diagram of the back end.....	39
5.2 Entity–relationship diagram of the database.....	40
5.3 Sequence diagram: user authentication.....	42
5.4 Sequence diagram: image detection and alerting.....	43
5.5 Sequence diagram: video detection with live streaming and confirmation.....	44
5.6 Activity diagram of the emergency agent.....	45
5.7 State machines: (a) incident status; (b) video-processing job.....	45
6.1 Landing page of the web application.....	49
6.2 Login and registration page.....	50
6.3 Operator dashboard with KPIs, activity trend, class distribution and recent incidents.....	50
6.4 Result of analysing a night-time rollover: severe collision detected with 95% confidence, alert dispatched and AI assessment shown.....	51
6.5 Video-analysis result for a 26-frame dash-cam clip: incident confirmed as severe after temporal confirmation.....	52
6.6 Incidents page with evidence cards, filters and triage actions.....	52
6.7 Analytics page with trend, confidence, class and alert-delivery charts.....	53
6.8 User-management page (administrators only).....	53
6.9 Settings page showing profile and live system configuration.....	54

6.10	Responsive dashboard on a mobile-sized screen (390 px).....	54
6.11	Automated back-end tests per module (36 of 36 passed).....	56
7.1	Training and validation losses per epoch: (a) box regression, (b) classification, (c) distribution focal loss.....	60
7.2	Validation precision, recall, mAP@0.5 and mAP@0.5:0.95 per epoch.....	61
7.3	Per-class precision, recall, mAP@0.5 and mAP@0.5:0.95 on the validation and test sets.....	62
7.4	Precision–recall curves (left) and F1 score versus confidence threshold (right) per class on the validation set.....	62
7.5	Normalised confusion matrix on the validation set (columns: true class; rows: predicted class).....	63
7.6	Model predictions with confidence scores on a validation batch.....	64
7.7	Sample predictions on held-out test images: fire, moderate and severe incidents.....	64
7.8	(a) CPU inference latency over 300 test images; (b) image-level classification accuracy on the same images.....	65
7.9	First page of an A4 incident report generated automatically by the system.....	66

LIST OF TABLES

Table No.	Page No.
1.1 Scope of the project.....	4
2.1 Qualitative comparison of related work with the proposed system.....	12
3.1 User roles and their permissions.....	15
3.2 Functional requirements.....	15
3.3 Non-functional requirements.....	16
3.4 Use-case description: UC-03 Analyse image.....	17
3.5 Use-case description: UC-04 Analyse video.....	18
3.6 Training hyper-parameters of YOLOv12n.....	21
3.7 Tools and technologies.....	25
4.1 Subsystems and their responsibilities.....	27
4.2 Security layers and controls.....	31
5.1 Component C1 – Authentication and access control.....	33
5.2 Component C2 – Detection engine.....	34
5.3 Component C3 – Image detection controller.....	34
5.4 Component C4 – Video job manager and video processor.....	35
5.5 Component C5 – Alert service.....	35
5.6 Component C6 – Emergency agent and WhatsApp service.....	35
5.7 Component C7 – Report generator.....	36
5.8 Component C8 – Persistence layer.....	36
5.9 Component C9 – Dashboard and analytics service.....	37
5.10 Component C10 – Web client (single-page application).....	37
5.11 Database tables.....	40
5.12 Data dictionary of the detection_events table.....	41
6.1 Development and evaluation environment.....	47
6.2 Testing levels and techniques.....	55
6.3 Automated test modules and coverage.....	56
6.4 System test cases and results.....	57
7.1 Overall detection performance of YOLOv12n.....	61
7.2 Per-class results on the validation and test sets.....	61
7.3 Model size and inference speed.....	65
7.4 Achievement of project objectives.....	66
8.1 Development and operating costs.....	71

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
AP	Average Precision
API	Application Programming Interface
CCTV	Closed-Circuit Television
CNN	Convolutional Neural Network
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
DETR	DEtection TRansformer
DFD	Data Flow Diagram
DFL	Distribution Focal Loss
FN	False Negative
FP	False Positive
FR	Functional Requirement
GFLOPs	Giga Floating-Point Operations
GPU	Graphics Processing Unit
HTTP	HyperText Transfer Protocol
IoU	Intersection over Union
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
mAP	mean Average Precision
MJPEG	Motion JPEG
MIME	Multipurpose Internet Mail Extensions
NFR	Non-Functional Requirement
NMS	Non-Maximum Suppression
ORM	Object–Relational Mapping
PDF	Portable Document Format
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SPA	Single-Page Application
SQL	Structured Query Language
TP	True Positive
UI	User Interface
UML	Unified Modeling Language
YOLO	You Only Look Once

ACKNOWLEDGMENTS

All praise is due to Almighty Allah, who gave us the strength, health and perseverance to complete this project.

We express our sincere gratitude to our supervisor, Engr. Iqra Gillani, for continuous guidance, constructive criticism and encouragement throughout the design, implementation and evaluation of this system. Her insistence on measurable results and honest evaluation shaped the quality of this work.

We are thankful to the faculty of the Department of Software Engineering, Mirpur University of Science and Technology (MUST), for the knowledge and skills imparted over the last four years, and to the members of the evaluation committee for their valuable feedback during the proposal and progress presentations.

We also acknowledge the open-source communities behind Ultralytics YOLO, PyTorch, OpenCV, FastAPI, React and MySQL, and the free GPU resources provided by Kaggle, without which training a modern detector would not have been possible within a student budget.

Finally, we thank our families and friends for their unconditional support, patience and motivation during the long hours spent on this project.

Adeel Danish
Syed Ali Ibtissam Bukhari
Syed Qosain Hyder Bukhari

ABSTRACT

Road traffic crashes kill approximately 1.19 million people every year and injure tens of millions more, and the time between a crash and the arrival of help strongly influences whether victims survive. In Pakistan, including Azad Jammu and Kashmir, crashes are still reported manually by passers-by, causing delays and leaving responders without information about severity. This project presents an AI-based road accident detection and emergency response system that analyses road-scene images and dash-cam or CCTV video, classifies each incident by severity, and immediately notifies responders with an evidence-backed, auditable report.

The detection core is a YOLOv12n object detector fine-tuned on a class-balanced dataset of 18,000 labelled images (14,400 training, 1,800 validation and 1,800 test) covering three classes: fire, moderate collision and severe collision. The model was trained for 40 epochs on a free Kaggle Tesla T4 GPU in 2.45 hours. On the held-out test set it achieves a precision of 92.5%, a recall of 86.8%, a mAP@0.5 of 89.9% and a mAP@0.5:0.95 of 79.0%, with per-class mAP@0.5 of 99.4% for severe, 97.3% for moderate and 72.9% for fire. With only 2.56 million parameters, the model runs in 5.8 ms per image on the GPU and 109 ms on an ordinary laptop CPU, and correctly classified 93.0% of 300 random test images at image level.

Around the model, a complete web platform was engineered with a layered client–server architecture: a FastAPI back end with JWT authentication, role-based access control and per-user data isolation; a normalised MySQL database; a video pipeline that streams annotated frames live and raises an alert only after three consecutive positive frames; a fail-safe emergency agent combining deterministic rules with an optional Google Gemini language model; e-mail and WhatsApp notifications; A4 PDF incident reports; and a responsive React dashboard. The system was verified with 36 automated tests and 40 documented system test cases, all of which passed, and is packaged for reproducible deployment with Docker Compose. The results show that a lightweight, attention-centric detector combined with careful system engineering delivers fast, explainable and low-cost accident alerting suitable for a pilot deployment, while the lower fire-class recall identifies the main direction for future work.

INTRODUCTION

Road traffic crashes are one of the most persistent public-health problems of the modern world. Every crash that is detected late or reported inaccurately wastes the minutes in which medical intervention is most effective. This chapter introduces the problem of delayed and unstructured accident reporting, explains the motivation for an automated, vision-based detection and response system, states the problem formally, and sets out the aims, objectives, scope and contributions of the project. The chapter closes with an outline of the organisation of the remainder of this report.

1.1 BACKGROUND

According to the World Health Organization, approximately 1.19 million people die every year as a result of road traffic crashes, and between 20 and 50 million more suffer non-fatal injuries, many of which lead to long-term disability [1]. Road traffic injuries are the leading cause of death for children and young adults aged 5 to 29 years, and more than nine in ten road deaths occur in low- and middle-income countries [1]. Pakistan, and the mountainous road network of Azad Jammu and Kashmir in particular, combines high traffic growth, mixed vehicle types, narrow roads and long distances between population centres and hospitals. In such conditions the time between a crash and the arrival of emergency help is frequently long, and much of that time is spent before anyone has even raised an alarm.

Emergency medicine has long emphasised that the earliest period after a serious injury is the most valuable for treatment. The chain of survival for a crash victim therefore begins not at the hospital but at the moment of detection: an incident that is recognised and reported immediately, with an accurate location and an indication of its severity, allows dispatchers to send the right resources to the right place. Today most crashes are reported through a voice call made by a witness. The caller may be in shock, may not know the exact location, and cannot reliably judge whether a vehicle is on fire or whether casualties are likely. No structured, visual record of the scene exists at the moment of dispatch.

At the same time, cameras have become ubiquitous on roads. Traffic-surveillance CCTV, dash-cams and smartphones continuously capture road scenes that could reveal a crash within seconds. Deep learning has made it possible to detect objects and events in such images in real time: single-stage detectors of the YOLO (You Only Look Once) family process an image in a single forward pass of a convolutional or attention-based neural network and can run at video frame rates on modest hardware [2], [3]. The most

recent member of the family, YOLOv12, introduces an attention-centric design that improves accuracy while retaining real-time speed [4]. These advances make it feasible to build an automated system that watches visual evidence, recognises an accident, estimates its severity and alerts responders without waiting for a human to make a phone call.

1.2 MOTIVATION

The motivation for this project comes from three observations. First, the information that reaches emergency services today is slow and unstructured; a system that attaches an annotated image, a severity class, a confidence score and a location to every alert gives dispatchers far more to act on than a verbal description. Second, existing research prototypes usually stop at the detection model: they report accuracy on a benchmark but do not deliver an operational pipeline that records incidents, notifies people, produces documentation and supports human review. Third, modern lightweight detectors and free cloud GPUs have made it realistic for a student team to train a competitive model and to deploy it on ordinary hardware, so the remaining challenge is primarily one of careful software engineering.

The project was also motivated by the need for decision support rather than full automation. An automated detector will occasionally be wrong, and emergency resources are expensive. A useful system must therefore combine speed with transparency: it should explain what it saw, show how confident it is, suppress obviously spurious alerts, record every notification that it sends and let a human operator acknowledge an incident or mark it as a false alarm.

1.3 PROBLEM STATEMENT

Road accidents and vehicle fires are currently detected and reported manually, which causes delays in emergency response, provides responders with little or no information about the severity of the incident, and leaves no reliable digital record for later review. Research on vision-based accident detection has shown that deep neural networks can recognise crashes in images and video [5]–[7], but most published approaches focus on a single binary decision (accident or no accident), are evaluated offline, and are not integrated with notification, reporting and incident-management workflows. Sensor- and smartphone-based systems [8], [9] depend on equipment inside the vehicle and provide no visual evidence of the scene.

There is therefore a need for an integrated, low-cost system that (i) automatically detects accidents and vehicle fires in both still images and video, (ii) classifies each incident by severity so that the appropriate services can be dispatched, (iii) minimises false alarms in video by requiring temporal confirmation, (iv) notifies responders

immediately through widely used channels with structured, evidence-backed information, and (v) keeps a secure, auditable record of every detection and notification that authorised operators can review and act upon.

1.4 AIMS AND OBJECTIVES

The aim of this project is to design, implement and evaluate an AI-based road accident detection and emergency response system that detects and classifies accidents from visual input in near real time and delivers actionable alerts to responders through a secure web platform. The specific objectives are:

1. To curate a class-balanced, labelled image dataset covering vehicle fire, moderate collision and severe collision, and to fine-tune a YOLOv12 detector on it.
2. To achieve a test-set mAP@0.5 of at least 85% and an inference time suitable for interactive use on a standard laptop CPU.
3. To build a RESTful back end that performs image and video inference, stores incidents and evidence, and enforces authentication, role-based authorisation and per-user data isolation.
4. To implement a video pipeline with live annotated streaming and a temporal confirmation rule that raises an alert only after several consecutive positive frames.
5. To generate a structured emergency assessment for every alert, using a large language model when available and a deterministic rule-based fallback otherwise.
6. To deliver notifications through e-mail and WhatsApp and to produce a formal PDF incident report for each assessed incident.
7. To provide a responsive web dashboard for analysis, incident triage, analytics and user management.
8. To verify the system through automated and manual testing and to package it for reproducible deployment.

1.5 SCOPE OF THE PROJECT

The scope of the project is summarised in Table 1.1. The system is designed as a decision-support prototype for a control room or a road-safety authority. It analyses media that is uploaded by an operator; it does not yet connect directly to live RTSP camera streams, and it does not replace verification of the scene by trained emergency personnel.

Table 1.1: Scope of the project

In scope	Out of scope
Detection of fire, moderate collision and severe collision in images and videos	Detection of other road events (traffic jams, violations, pedestrian incidents)
Severity-aware alerting with temporal confirmation for video	Direct integration with national emergency numbers or dispatch systems
E-mail and WhatsApp notifications; PDF incident reports	Voice calls to responders (only a local audit entry is recorded)
Web dashboard with authentication, roles and analytics	Native mobile applications
Operator-entered incident location	Automatic GPS geolocation of camera feeds
Single-host deployment with Docker Compose	Horizontally scaled, multi-region production deployment

1.6 PROPOSED SOLUTION

The proposed solution is a web-based system that follows the processing chain shown in Figure 1.1. An operator uploads a road-scene image or a dash-cam or CCTV video through the web dashboard. The back end runs the fine-tuned YOLOv12n model to localise and classify accident evidence. For videos, frames are sampled and analysed in a background job while annotated frames are streamed live to the browser; an incident is confirmed only when an accident class is detected in three consecutive sampled frames. When an incident is confirmed, the system records it, sends an e-mail alert with the annotated evidence, generates an emergency assessment that recommends services and immediate actions, dispatches a WhatsApp message, and produces a formal PDF report. All incidents are then available on the dashboard, where operators can review evidence, acknowledge incidents or mark them as false alarms, and analyse trends.

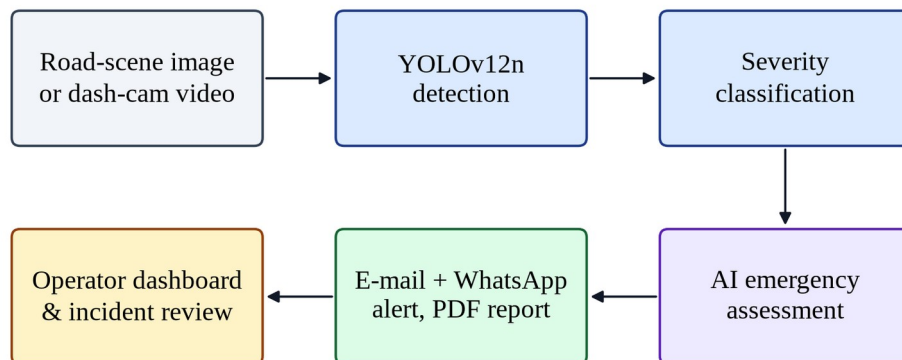


Figure 1.1: Conceptual overview of the proposed accident detection and emergency response pipeline

1.7 CONTRIBUTIONS

The main contributions of this project are as follows:

- **A three-class severity-aware detector.** A YOLOv12n model fine-tuned to distinguish vehicle fire, moderate collision and severe collision, reaching 89.9% mAP@0.5 on a held-out test set of 1,800 images with only 2.56 million parameters.
- **Temporal confirmation for video.** A simple, explainable rule that requires three consecutive positive sampled frames before an alert is raised, reducing the risk of alerts caused by a single misclassified frame.
- **An end-to-end response pipeline.** Detection is connected to persistence, e-mail and WhatsApp notification, a structured LLM-assisted assessment with a guaranteed rule-based fallback, and an A4 PDF incident report.
- **Security by design.** JWT authentication, role-based access control, per-user data isolation, private storage of original uploads, upload validation and hardened HTTP headers, all covered by regression tests.
- **A reproducible, documented platform.** A Dockerised deployment, a migration-managed database schema and an evaluation of accuracy and CPU performance on real hardware.

1.8 SIGNIFICANCE OF THE STUDY

For emergency services, the system shortens the time between an accident being captured on camera and responders being notified, and it provides visual evidence and a severity estimate that can inform the choice between, for example, sending an ambulance alone or an ambulance together with the fire brigade. For road-safety authorities, the persisted history of incidents, locations and severities provides data for identifying dangerous road segments. For the research community, the project demonstrates how a state-of-the-art detector can be engineered into a dependable, auditable service, and it reports honest per-class results that highlight where current models still struggle.

1.9 ORGANISATION OF THE REPORT

The rest of this report is organised as follows. **Chapter 2** reviews the literature on accident detection, object detection, fire detection and language-model-assisted decision support, and identifies the research gap. **Chapter 3** presents the development methodology, the requirements, the use-case model, the dataset and the model-training and alerting methodology. **Chapter 4** describes the system architecture, subsystems, data flows and deployment. **Chapter 5** gives the detailed design, including component specifications, the class diagram, the database design and the dynamic behaviour of the

system. **Chapter 6** explains the implementation and the testing strategy, test cases and deployment plan. **Chapter 7** presents and discusses the results. **Chapter 8** concludes the report, discusses strengths, limitations and costs, and outlines future work.

LITERATURE REVIEW

This chapter reviews the body of work on which the project builds. It begins with the scale of the road-safety problem, then surveys the three main families of automatic accident detection: in-vehicle sensor systems, smartphone systems and vision-based systems. Because the detector is the heart of the proposed system, the chapter then traces the evolution of deep-learning object detectors from two-stage region-based methods to single-stage YOLO models and attention-based detectors, including YOLOv12. It also covers vision-based fire detection, the evaluation metrics used for detectors, and the use of large language models and secure web technologies in decision-support systems. The chapter ends with a comparative analysis and a statement of the research gap that motivates this project.

2.1 ROAD TRAFFIC ACCIDENTS AND EMERGENCY RESPONSE

The World Health Organization's *Global Status Report on Road Safety 2023* estimates that road traffic crashes cause approximately 1.19 million deaths each year, that road traffic injury is the leading killer of people aged 5–29 years, and that about 92% of the world's road fatalities occur in low- and middle-income countries even though these countries have only about 60% of the world's vehicles [1]. The report stresses post-crash response as one of the pillars of the Safe System approach: rapid detection, notification and pre-hospital care reduce both deaths and the severity of long-term disability.

The weakest link in post-crash response is often the very first step, namely knowing that a crash has happened. Manual reporting depends on the presence and willingness of witnesses, on their ability to describe the location, and on their judgement of severity. Automatic detection aims to remove this dependency. The literature approaches the problem in two broad ways: by instrumenting the vehicle or the driver's phone, or by analysing visual data from cameras that observe the road.

2.2 SENSOR- AND SMARTPHONE-BASED ACCIDENT DETECTION

Early automatic crash notification systems were built into vehicles and relied on airbag deployment or crash sensors. Because such systems are expensive and limited to new vehicles, researchers turned to devices that people already carry. White et al. presented *WreckWatch*, a smartphone application that uses the phone's accelerometer and acoustic data to detect crashes and to notify emergency responders, and that can also provide situational information such as images collected from bystanders [8]. Zaldivar et al. combined an OBD-II interface with an Android smartphone so that the vehicle's own

crash indicators, such as airbag deployment, trigger an automatic notification that includes the vehicle's position [9].

These systems demonstrate the value of automatic notification, but they share several limitations. They only protect vehicles or drivers equipped with the particular device or application; they can be triggered by non-crash events such as a dropped phone; and, most importantly, they provide little direct evidence of what the scene looks like, whether a vehicle is on fire or how many vehicles are involved. They cannot detect accidents involving third parties who carry no such device. Vision-based systems address these limitations by observing the road itself.

2.3 VISION-BASED ACCIDENT DETECTION

Vision-based methods analyse images or video from traffic-surveillance cameras, dash-cams or other sources. A recent survey by Fang et al. organises the field into accident detection, which recognises a crash that has occurred, and accident anticipation, which predicts that a crash is about to occur, and identifies the scarcity of large labelled datasets and the difficulty of generalising across viewpoints, weather and lighting as the central challenges [7].

For surveillance cameras, Ijjina et al. proposed a framework that detects vehicles with a deep instance-segmentation network, tracks them across frames and flags an accident when anomalies in speed, trajectory and the overlap of vehicle regions indicate a collision [5]. Ghahremannezhad et al. similarly combined a YOLO-based detector with multi-object tracking and trajectory analysis to detect accidents in real time from traffic-surveillance video [6]. Publicly available CCTV accident datasets nevertheless remain small, which reflects the need for domain-specific training data [7].

For ego-centric dash-cam video, Chan et al. formulated accident anticipation, introducing a recurrent network with dynamic spatial attention that predicts an imminent accident from dash-cam footage [10]. Bao et al. modelled spatio-temporal relations between objects with graph-based learning and estimated the uncertainty of their accident-anticipation predictions [11].

Two observations from this literature shape the present project. First, tracking- and trajectory-based pipelines are powerful for fixed cameras but are complex to tune and depend heavily on camera geometry, whereas detecting the visual *evidence* of an accident, such as deformed vehicles, overturned vehicles or flames, generalises more easily across still images, dash-cams and CCTV. Second, most published systems output a single accident/no-accident decision and are evaluated offline; very few report how the detection is turned into a notification, a record or a decision by a human operator.

2.4 DEEP-LEARNING OBJECT DETECTION

Object detection is the task of localising every object of interest in an image with a bounding box and assigning it a class label. It is the natural formulation for this project because an accident scene may contain several vehicles and a fire at the same time, and because the bounding boxes explain *where* the model found evidence. Figure 2.1 summarises the evolution of the detectors discussed in this section.

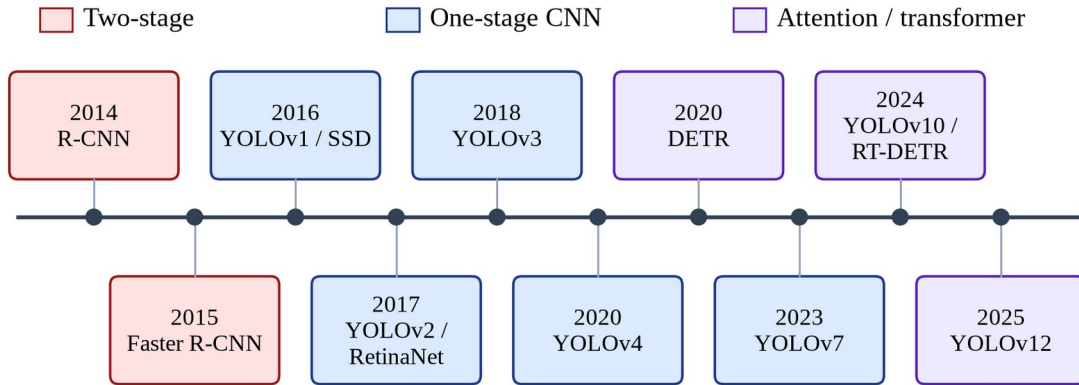


Figure 2.1: Evolution of deep-learning object detectors relevant to this project

2.4.1 Two-stage detectors

Girshick et al. introduced R-CNN, which generates region proposals, extracts features for each region with a convolutional neural network and classifies them, greatly improving detection accuracy over hand-crafted features [12]. Ren et al. then proposed Faster R-CNN, in which a Region Proposal Network shares convolutional features with the detection head so that proposals are almost free, making two-stage detection far faster and end-to-end trainable [13]. Two-stage methods remain highly accurate, but their per-region processing makes them heavier than single-stage models, which matters when every frame of a video must be analysed on a CPU.

2.4.2 Single-stage detectors and the YOLO family

Single-stage detectors predict boxes and classes directly from feature maps in one pass. Liu et al.'s SSD predicts from default boxes on multiple feature maps to handle objects of different sizes [14]. Lin et al. identified the extreme imbalance between foreground and background examples as the main reason single-stage detectors lagged behind two-stage ones, and introduced the focal loss in RetinaNet to down-weight easy background examples [15].

The YOLO family has become the most widely deployed line of single-stage detectors. The original YOLO framed detection as a single regression problem over a grid, reaching real-time speed [2]. YOLOv2 (YOLO9000) added anchor boxes, batch normalisation and multi-scale training, and YOLOv3 introduced a deeper backbone

with predictions at three scales [3]. YOLOv4 systematically combined training techniques (a 'bag of freebies', including mosaic augmentation) and architectural modules to improve the accuracy–speed trade-off [16]. YOLOv7 contributed trainable bag-of-freebies and efficient layer aggregation [3], and YOLOv10 removed the need for non-maximum suppression through consistent dual label assignment [17]. Terven et al. provide a comprehensive review of the architectural changes across the family [3]. The open-source Ultralytics framework [18], built on PyTorch, provides training, validation and deployment tooling for these models and is used in this project.

2.4.3 Transformer- and attention-based detectors

The transformer architecture, built entirely on attention mechanisms [19], has also transformed detection. Carion et al.'s DETR treats detection as direct set prediction with a transformer encoder–decoder and bipartite matching, removing hand-designed components such as anchors and non-maximum suppression, but it converges slowly and is computationally heavy [20]. Zhao et al.'s RT-DETR showed that a carefully designed transformer detector can compete with YOLO models in real time [21].

2.4.4 YOLOv12: attention-centric real-time detection

Tian, Ye and Doermann proposed YOLOv12, the first YOLO model that is centred on attention while retaining the speed of CNN-based predecessors [4]. Its key ideas are an *area attention* module, which divides feature maps into a small number of areas so that attention remains efficient at high resolution; *residual efficient layer aggregation networks* (R-ELAN), which stabilise the optimisation of attention-based blocks; and architectural simplifications that make the attention compatible with fast inference. The authors report higher accuracy than earlier YOLO versions at comparable latency, with the smallest variant, YOLOv12-N, designed for resource-constrained deployment. Because the proposed system must run on ordinary CPUs as well as GPUs and must process many video frames, the nano variant was selected, as discussed in Chapter 3.

2.5 VISION-BASED FIRE DETECTION

Vehicle fires are among the most dangerous post-crash events, and the appropriate response (fire brigade rather than only an ambulance) differs from that for a collision. Traditional fire detectors in video relied on hand-crafted colour, motion and flicker features, which produce many false alarms from sunsets, headlights, reflections and red objects. Muhammad et al. showed that convolutional neural networks fine-tuned for fire detection in surveillance video substantially outperform such hand-crafted approaches while remaining efficient enough for real-time use [22]. Fire nevertheless remains visually ambiguous: flames have no fixed shape, vary in scale from a small engine fire to a burning vehicle, and are easily confused with bright lights. The results in Chapter 7 confirm that fire is the hardest of the three classes for the proposed model.

2.6 DATASETS AND EVALUATION METRICS FOR DETECTION

Detectors are evaluated by comparing predicted boxes with ground-truth boxes using the Intersection over Union (IoU). The PASCAL VOC challenge popularised average precision (AP) computed at an IoU threshold of 0.5, while the Microsoft COCO benchmark averages AP over IoU thresholds from 0.5 to 0.95 in steps of 0.05, rewarding precise localisation [23]. Both metrics are reported in this work as mAP@0.5 and mAP@0.5:0.95. COCO is also the source of the pretrained weights from which YOLO models are commonly fine-tuned, a form of transfer learning that makes it possible to reach high accuracy with a domain-specific dataset of moderate size. The definitions used in this project are given in Section 3.10.

2.7 LANGUAGE MODELS FOR EMERGENCY DECISION SUPPORT

Large language models (LLMs) based on the transformer architecture [19] can turn structured facts into fluent, well-organised text and can follow instructions to produce machine-readable output such as JSON. Google's Gemini family of multimodal models [24] is available through an API with a free tier suitable for prototypes. In emergency response, an LLM can summarise a detection into a situation report and recommend services and immediate actions. However, LLMs can produce incorrect or inconsistent output and depend on network availability, so safety-critical systems must never make notification contingent on the LLM. The design in this project therefore sends a deterministic, rule-based alert first and uses the LLM only to enrich the persisted assessment, falling back to rules whenever the model is unavailable or returns invalid output.

2.8 SECURE WEB PLATFORMS FOR INCIDENT MANAGEMENT

An incident-management platform handles sensitive images and personal data and must therefore authenticate users and restrict what each user can see. JSON Web Tokens (JWT), standardised in RFC 7519, allow a server to issue a signed, self-contained token that carries the user's identity and expiry time, enabling stateless authentication for REST APIs. Passwords must be stored with a deliberately slow, salted hash; the bcrypt scheme uses an adaptive cost factor so that hashing can be made slower as hardware improves. Modern Python frameworks such as FastAPI provide dependency injection that makes it straightforward to apply authentication and authorisation checks consistently to every endpoint, and component-based front-end libraries such as React support responsive single-page dashboards. Container platforms such as Docker make such multi-service systems reproducible to deploy.

2.9 COMPARATIVE ANALYSIS OF RELATED WORK

Table 2.1 compares representative related work with the proposed system along the dimensions that matter for an operational emergency-response tool. The comparison is qualitative, because the studies use different datasets, inputs and definitions of an accident, so their reported accuracies are not directly comparable.

Table 2.1: Qualitative comparison of related work with the proposed system

Work	Input	Technique	Severity levels	Notification / integration
WreckWatch [8]	Smartphone sensors	Accelerometer and acoustic thresholds	No	Automatic notification
Zaldivar et al. [9]	OBD-II + smartphone	Vehicle crash indicators	No	Automatic notification with location
Ijjina et al. [5]	CCTV video	Deep detection + tracking + anomaly rules	No	Not addressed
Ghahremannezhad et al. [6]	CCTV video	YOLO detection + tracking + trajectory analysis	No	Not addressed
Chan et al.; Bao et al. [10], [11]	Dash-cam video	Recurrent / graph-based anticipation	No	Not addressed
Muhammad et al. [22]	Surveillance video	CNN fire classification	Fire only	Not addressed
Proposed system	Images and video (CCTV, dash-cam)	YOLOv12n detection + temporal confirmation + LLM/rule assessment	Fire, moderate, severe	E-mail, WhatsApp, PDF report, dashboard, audit log

2.10 RESEARCH GAP

The review reveals a clear gap between research models and usable emergency-response tools. Sensor-based systems notify quickly but provide no visual evidence and cover only instrumented vehicles. Vision-based research achieves good detection accuracy but typically (i) reduces the problem to a binary decision without severity, (ii) treats fire detection and collision detection as separate problems, (iii) evaluates offline without addressing false-alarm control in continuous video, and (iv) stops before notification, documentation, access control and human review. Recent attention-centric detectors such as YOLOv12 have not yet been widely applied to this domain. This project addresses the gap by combining a lightweight YOLOv12n detector for three severity-aware classes with temporal confirmation, a fail-safe LLM-assisted assessment, multi-channel notification and a secure, auditable web platform, and by evaluating both model accuracy and practical CPU performance.

REQUIREMENTS ANALYSIS AND METHODOLOGY

This chapter explains how the project was carried out and what the system is required to do. It describes the software development methodology and the feasibility study, identifies the users of the system, and specifies the functional and non-functional requirements together with the use-case model. It then presents the machine-learning methodology in detail: the dataset, the choice of model, the training configuration, the augmentation strategy and the evaluation metrics. Finally, it describes the algorithm that confirms incidents in video, the method used to produce emergency assessments, and the tools and technologies used.

3.1 DEVELOPMENT METHODOLOGY

The project combined two kinds of work with different levels of uncertainty: machine-learning experimentation, in which the outcome of a training run is not known in advance, and software engineering, in which requirements are clearer but interact with each other. An **iterative and incremental** process model was therefore adopted, as illustrated in Figure 3.1. Each increment passed through requirements refinement, design, implementation, testing and review with the supervisor, and delivered a working, demonstrable part of the system. Feedback from each review, such as the need to reduce false alarms in video or to show evidence in alerts, was fed into the next iteration.

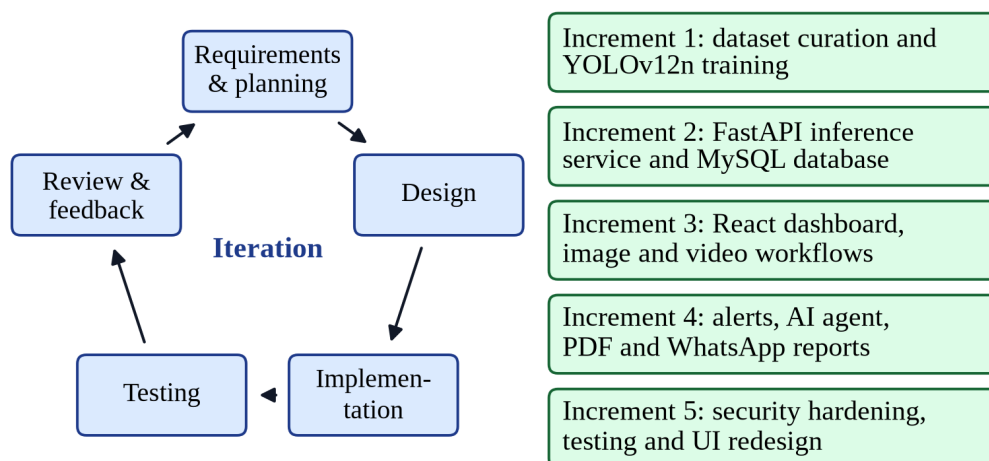


Figure 3.1: Iterative and incremental development process and the five increments of the project

The five increments were: (1) dataset curation and YOLOv12n training; (2) a FastAPI inference service with MySQL persistence; (3) the React dashboard with image and

video workflows; (4) alerts, the emergency agent, PDF reports and WhatsApp messages; and (5) security hardening, systematic testing and a redesign of the user interface. Version control with Git, automated tests and code linting were used throughout so that each increment could be integrated without breaking earlier functionality.

3.2 FEASIBILITY STUDY

Before development, the feasibility of the project was assessed from technical, economic, operational and legal–ethical perspectives.

- **Technical feasibility.** Pretrained YOLO weights, the Ultralytics training framework and free GPU sessions on Kaggle make it possible to train a modern detector without dedicated hardware. FastAPI, React and MySQL are mature, well-documented technologies within the team's skills.
- **Economic feasibility.** All software used is open source or offered with a free tier. Training used a free Kaggle Tesla T4 GPU, and the system runs on an ordinary laptop CPU, so no hardware purchase was required (see Section 8.6).
- **Operational feasibility.** The system fits the existing workflow of a control room: operators already receive images and videos; the system adds automatic analysis, alerting and record keeping without requiring special training beyond a web browser.
- **Legal and ethical feasibility.** Images of accidents may contain people and number plates. The design stores original uploads outside the public web root, restricts access by role and ownership, and positions the model as decision support that requires human verification rather than as an autonomous dispatcher.

3.3 STAKEHOLDERS AND USER CLASSES

The system distinguishes three user roles, summarised in Table 3.1. Roles are assigned by an administrator; self-registration always creates a viewer account, so a member of the public cannot grant themselves administrative rights. External systems (the SMTP e-mail server, the Kapso WhatsApp gateway and the Google Gemini API) act as secondary actors.

Table 3.1: User roles and their permissions

Role	Typical user	Permissions
Viewer	Newly registered user, trainee operator	Analyse images and videos; view own incidents, alerts and analytics; acknowledge own incidents; download own PDF reports
Operator	Control-room staff	Same as viewer; intended for staff who triage incidents
Administrator	Supervisor, system owner	All permissions over all users' data; manage users and roles; delete incidents; test notification channels

3.4 FUNCTIONAL REQUIREMENTS

The functional requirements were derived from the problem statement, the review of related systems and discussions with the supervisor. They are listed in Table 3.2 with a priority according to the MoSCoW scheme (M = must have, S = should have, C = could have).

Table 3.2: Functional requirements

ID	Requirement	Priority
FR-01	The system shall allow a user to register with a unique username, a valid e-mail address and a password of 6 to 72 bytes.	M
FR-02	The system shall authenticate users and issue a signed access token that expires after 60 minutes.	M
FR-03	The system shall accept JPG, JPEG, PNG and WEBP images up to 100 MB and reject other types, corrupt files and images above 25 megapixels.	M
FR-04	The system shall detect and localise fire, moderate and severe accident evidence in an uploaded image and return the class, confidence and bounding boxes.	M
FR-05	The system shall return an annotated copy of the analysed image.	M
FR-06	The system shall accept MP4, AVI, MOV and MKV videos up to 100 MB and process them as background jobs.	M
FR-07	The system shall report video-processing progress and stream annotated frames live to the browser.	S
FR-08	The system shall raise a video alert only after an accident class is detected in three consecutive sampled frames.	M
FR-09	The system shall record every detection with its media type, class, confidence, boxes, evidence path, location, timestamp and owner.	M
FR-10	The system shall send an e-mail alert containing the annotated evidence for every confirmed incident when SMTP is configured.	M
FR-11	The system shall generate a structured emergency assessment (level, summary, services, actions, casualty risk) for each confirmed incident.	M
FR-12	The system shall send a WhatsApp incident message and record whether the provider accepted it.	S
FR-13	The system shall produce a downloadable A4 PDF incident report.	S

ID	Requirement	Priority
FR-14	The system shall list incidents with evidence and allow them to be acknowledged, resolved or marked as false alarms.	M
FR-15	The system shall show dashboard statistics, detection trends, class distribution and confidence distribution.	S
FR-16	The system shall show delivery logs for e-mail, WhatsApp and call-audit channels.	S
FR-17	The system shall allow administrators to change user roles and activate or deactivate accounts.	M
FR-18	The system shall allow administrators to delete incidents and to send test notifications.	C
FR-19	The system shall let the user enter the incident location with each upload.	S
FR-20	The system shall expose a health endpoint reporting whether the model is loaded.	C

3.5 NON-FUNCTIONAL REQUIREMENTS

The quality attributes that the system must satisfy are listed in Table 3.3. Each is stated in a verifiable form so that it can be checked during testing (Chapter 6) and evaluation (Chapter 7).

Table 3.3: Non-functional requirements

ID	Attribute	Requirement
NFR-01	Accuracy	Test-set mAP@0.5 $\geq$ 85% over the three classes.
NFR-02	Performance	Single-image inference $\leq$ 200 ms on a mid-range laptop CPU; the API must stay responsive while inference runs.
NFR-03	Reliability	A failure of the LLM, WhatsApp or e-mail provider must never prevent the incident from being recorded; a fallback assessment is always stored.
NFR-04	Security	Passwords hashed with bcrypt; JWT authentication; role and ownership checks on every data endpoint; original uploads not publicly served; secrets never committed to version control.
NFR-05	Privacy	Users can only see their own incidents unless they are administrators.
NFR-06	Usability	A new operator can analyse an image in three steps (upload, optionally enter location, run) with clear severity colours and messages.
NFR-07	Portability	The system must be deployable on any host with Docker Compose, with configuration supplied through environment variables.
NFR-08	Maintainability	Layered code structure, database migrations, linting and automated tests that run in under one minute.
NFR-09	Responsiveness	The dashboard must be usable on screens from 390 px (mobile) to desktop width.
NFR-10	Auditability	Every notification attempt and every change of incident status is recorded with actor and outcome.

3.6 USE-CASE MODEL

Figure 3.2 shows the use-case diagram. The primary actor, the viewer/operator, registers, logs in, analyses images and videos, reviews incidents, updates incident status, views analytics and downloads reports. The administrator inherits all of these use cases and additionally manages users, deletes incidents and tests the alert channels. The analysis use cases include the *Dispatch emergency alerts* use case whenever an incident is confirmed, which in turn includes *Generate AI assessment*. The e-mail/WhatsApp provider and the Gemini LLM service are secondary actors.

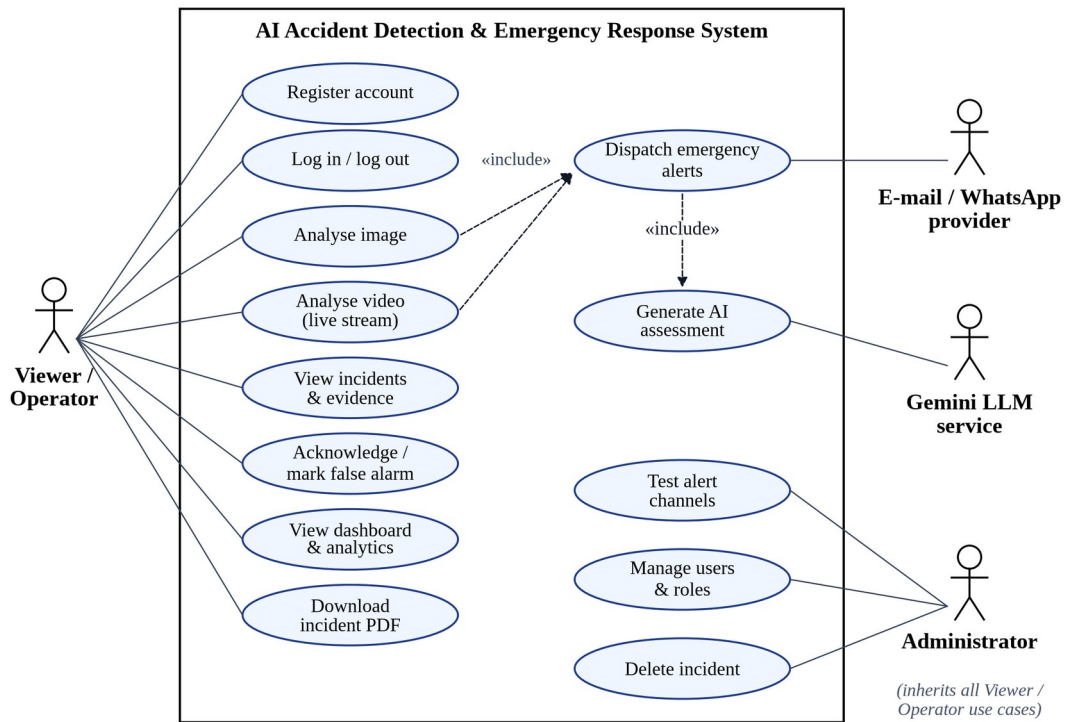


Figure 3.2: Use-case diagram of the AI accident detection and emergency response system

The most important use cases are described in Table 3.4 and Table 3.5. The complete set of use cases is covered by the system test cases in Section 6.6.

Table 3.4: Use-case description: UC-03 Analyse image

Field	Description
Actor	Viewer / Operator (primary); e-mail, WhatsApp and Gemini services (secondary)
Pre-conditions	The user is logged in and holds a valid token; the model is loaded.

Field	Description
Main flow	1. The user opens Image Analysis and drops or selects an image. 2. The user optionally enters the incident location and presses Run detection. 3. The system validates the file type, MIME type, size and resolution. 4. The system runs YOLOv12n and draws the detected boxes. 5. The system stores the incident and returns the class, confidence, boxes and annotated image. 6. If the class is fire, moderate or severe, the system triggers UC-10 Dispatch emergency alerts. 7. The user sees the verdict, metrics, detected objects and, shortly after, the AI assessment.
Alternative flows	3a. Invalid or corrupt file: the system rejects it with a clear message. 4a. No accident detected: the verdict shows 'No incident' and no alert is sent.
Post-conditions	A detection event exists; for an incident, alert, assessment, WhatsApp and PDF records exist.

Table 3.5: Use-case description: UC-04 Analyse video

Field	Description
Actor	Viewer / Operator
Pre-conditions	The user is logged in; the video is at most 100 MB and 9,000 frames.
Main flow	1. The user drops a video, optionally enters a location and presses Process video. 2. The system stores the upload privately, verifies that it can be decoded, and creates a background job. 3. The browser displays upload and frame-analysis progress and a live annotated stream. 4. The system analyses every second frame and applies the three-frame confirmation rule. 5. At the end, the system stores the incident, the key frame and per-class frame counts, and deletes the original upload. 6. If an incident was confirmed, UC-10 is triggered. 7. The user sees the dominant class, frame breakdown and AI assessment.
Alternative flows	2a. Corrupt video: rejected immediately with HTTP 400. 4a. Frame limit exceeded or processing error: the job ends in the failed state with an error message.
Post-conditions	A detection event and a video-processing log exist; the job result is available for one hour.

3.7 DATASET PREPARATION

A supervised detector requires images annotated with bounding boxes. The project team merged several collections of road-accident and vehicle-fire images into a single cleaned dataset and exported it in the YOLO format, in which each image has a text file listing one object per line as a class index and a normalised box centre, width and height. Three classes were defined, in a fixed order that is shared by the training configuration, the model file and the back-end code: **0 = fire** (a vehicle or its surroundings on fire), **1 =**

moderate (a collision with visible damage but a vehicle largely intact) and **2 = severe** (a major crash such as a rollover, head-on collision or a heavily deformed vehicle).

The dataset was balanced by class at the image level and split into training, validation and test subsets with a ratio of 80:10:10, as shown in Figure 3.3. Before training, an automated audit verified that every image had exactly one matching label file and that every label referred to a valid class; the audit reported no missing or orphaned files. Figure 3.3 shows the resulting distribution of annotated objects and images. Because an image of a burning vehicle often contains several separate flame regions, the fire class contains roughly twice as many boxes as the other classes (11,974 fire boxes, 6,085 moderate and 6,143 severe, for a total of 24,202 boxes) even though the number of images per class is identical.

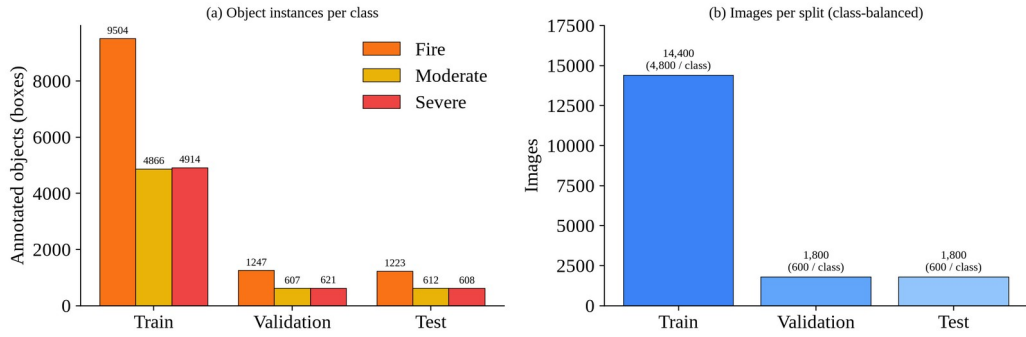


Figure 3.3: Dataset composition: (a) annotated object instances per class and split; (b) images per split

Figure 3.4 shows the label statistics of the training set. Bounding boxes are spread across the whole image area and cover a wide range of sizes, from small distant vehicles to boxes that fill most of the frame, which is typical of mixed CCTV, dash-cam and hand-held photographs.

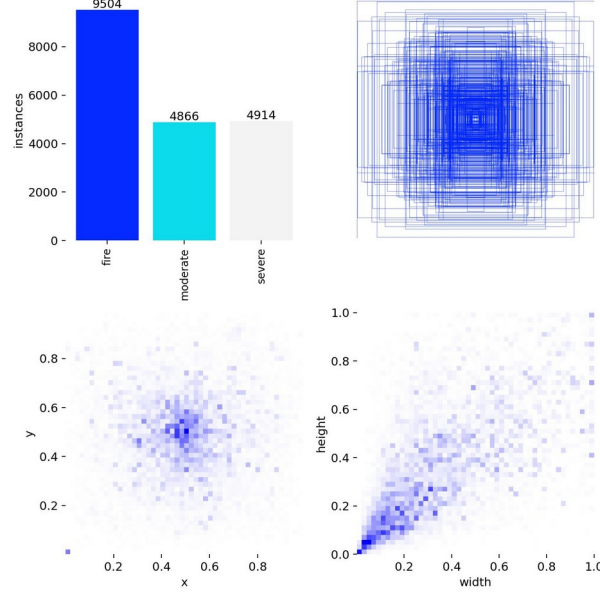


Figure 3.4: Label statistics produced by Ultralytics for the training set: class counts, box positions and box sizes

3.8 MODEL SELECTION

Several criteria guided the choice of detector: real-time inference on a CPU, strong accuracy with a modest dataset, active maintenance and simple deployment. Based on the review in Section 2.4, YOLOv12 was selected because its attention-centric design delivers better accuracy than earlier YOLO versions at similar latency [4]. Among its variants (N, S, M, L and X), the **nano (YOLOv12n)** variant was chosen. After layer fusion it contains 159 layers, 2,557,313 parameters and requires 7.3 GFLOPs per 640×640 image, and its trained weight file is only 5.5 MB. This makes it practical to analyse every second frame of a video on a laptop CPU, which a larger variant would not allow. The model was initialised from COCO-pretrained weights [23] and fine-tuned on the accident dataset, a transfer-learning strategy that reuses general visual features and converges much faster than training from scratch.

3.9 TRAINING CONFIGURATION AND AUGMENTATION

Training was performed in a Kaggle notebook on an NVIDIA Tesla T4 GPU (15 GB) using PyTorch 2.10 and Ultralytics 8.4 [18]. The pipeline is summarised in Figure 3.5 and the hyper-parameters are listed in Table 3.6. A fixed random seed (42) was used for reproducibility, and automatic mixed precision (AMP) was enabled to speed up training.

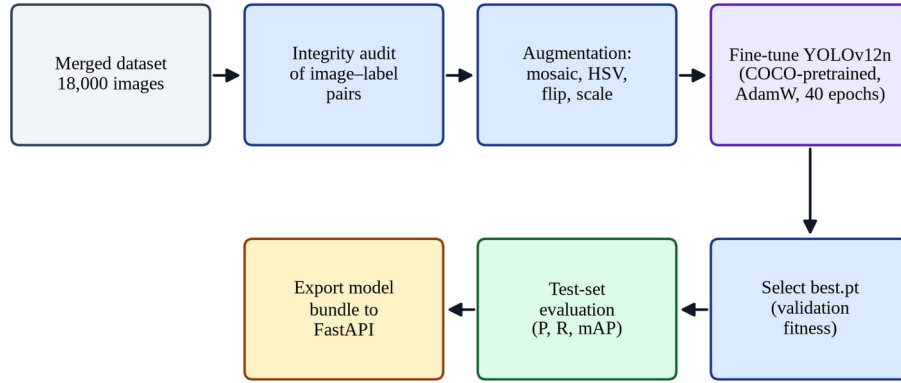


Figure 3.5: Model-training pipeline from the merged dataset to the exported model bundle

Table 3.6: Training hyper-parameters of YOLOv12n

Parameter	Value	Purpose
Base weights	yolo12n.pt (COCO)	Transfer learning from general object detection
Epochs / patience	40 / 15	Maximum epochs; early stopping if no improvement for 15 epochs
Batch size / image size	32 / 640×640	Fits T4 memory (6.3 GB used); standard YOLO resolution
Optimiser	AdamW	Adaptive optimiser with decoupled weight decay
Initial LR / final LR factor	0.01 / 0.01	Linear decay to 1% of the initial learning rate
Momentum / weight decay	0.937 / 0.0005	Optimiser momentum (β_1) and L2 regularisation
Warm-up	3 epochs	Gradual ramp-up of the learning rate and momentum
Loss weights (box / cls / DFL)	7.5 / 0.5 / 1.5	Relative weight of localisation, classification and distribution focal loss
Mosaic / close mosaic	1.0 / last 10 epochs	Combine four images per sample; disable at the end to fine-tune on natural images
HSV (hue / sat / value)	0.015 / 0.7 / 0.4	Colour jitter for robustness to lighting and weather
Flip LR / translate / scale	0.5 / 0.1 / 0.5	Geometric robustness
Random erasing / auto-augment	0.4 / RandAugment	Occlusion robustness and policy-based augmentation
Seed / AMP	42 / enabled	Reproducibility and faster mixed-precision training

Augmentation is particularly important for this domain. Accident images vary widely in illumination (day, night, headlights, flames), viewpoint (overhead CCTV, dash-cam, hand-held) and scale. Mosaic augmentation [16] places four images in one training sample, exposing the model to objects at different scales and to partial objects at image borders. Colour jitter in HSV space makes the detector less sensitive to lighting, and horizontal flipping, translation and scaling improve geometric robustness. Mosaic was switched off for the final ten epochs so that the model could adapt to the statistics of

natural, unmodified images. Figure 3.6 shows an example of an augmented training batch.



Figure 3.6: An augmented training batch showing mosaic composition, colour jitter and ground-truth boxes (0 = fire, 1 = moderate, 2 = severe)

YOLOv12 is trained with a composite loss. The bounding-box loss uses the complete IoU between predicted and ground-truth boxes, the distribution focal loss (DFL) refines box edges by treating each coordinate as a discrete probability distribution, and the classification loss is a binary cross-entropy over the class scores. The weighted sum of these terms, with the weights given in Table 3.6, is minimised during training. The best checkpoint is selected by a fitness score equal to $0.1 \times \text{mAP}@0.5 + 0.9 \times \text{mAP}@0.5:0.95$ on the validation set, which favours precise localisation.

3.10 EVALUATION METRICS

The detector is evaluated with the standard metrics introduced in Section 2.6. A predicted box is counted as a true positive (TP) if it has the correct class and its IoU with an unmatched ground-truth box exceeds a threshold; otherwise it is a false positive (FP). Ground-truth boxes without a matching prediction are false negatives (FN). The IoU of a predicted box B_p and a ground-truth box B_g is the area of their intersection divided by the area of their union:

$$IoU = |B_p \cap B_g| / |B_p \cup B_g| \quad (3.1)$$

Precision measures how many of the predicted incidents are real, and recall measures how many real incidents are found:

$$Precision = TP / (TP + FP) \quad (3.2)$$

$$Recall = TP / (TP + FN) \quad (3.3)$$

$$F1 = 2 \times Precision \times Recall / (Precision + Recall) \quad (3.4)$$

The average precision (AP) of a class is the area under its precision–recall curve, obtained by sweeping the confidence threshold. The mean average precision (mAP) is the mean of AP over the $C = 3$ classes. mAP@0.5 uses an IoU threshold of 0.5, while mAP@0.5:0.95 averages the AP over ten IoU thresholds from 0.50 to 0.95 [23]:

$$AP = \int_0^1 p(r) dr, \quad mAP = (1 / C) \sum_c AP_c \quad (3.5)$$

In addition to these object-level metrics, two system-level measures are reported in Chapter 7: the inference latency per image on a laptop CPU, and the image-level top-1 accuracy, defined as the proportion of test images for which the highest-confidence detection has the same class as the image's ground-truth label.

3.11 TEMPORAL CONFIRMATION ALGORITHM FOR VIDEO

A detector applied independently to every video frame will occasionally misclassify a single frame, for example because of motion blur or a reflection. Sending an emergency alert on such a frame would waste responders' time and erode trust in the system. The project therefore applies a simple, explainable **temporal confirmation** rule, shown as a flowchart in Figure 3.7.

Frames are read sequentially and a frame limit of 9,000 protects the server from excessively long videos. Only every second frame (the *stride*, configurable) is analysed, which halves the computation while preserving temporal coverage; unanalysed frames are written unchanged to the output. For each analysed frame the annotated result is published to the live stream. A counter of consecutive positive frames is incremented when the top detection belongs to an accident class and reset to zero otherwise. When the counter reaches the confirmation threshold of three, the incident is confirmed. At the end of the video, the frame with the highest detection confidence is saved as the key frame, the dominant class is taken as the most frequent accident class, and, only if the incident was confirmed, alerts and the emergency assessment are triggered. With a stride of two, three consecutive analysed frames span about six video frames, or 0.2 seconds at 30 frames per second, so genuine incidents are confirmed almost immediately while isolated misclassifications are ignored.

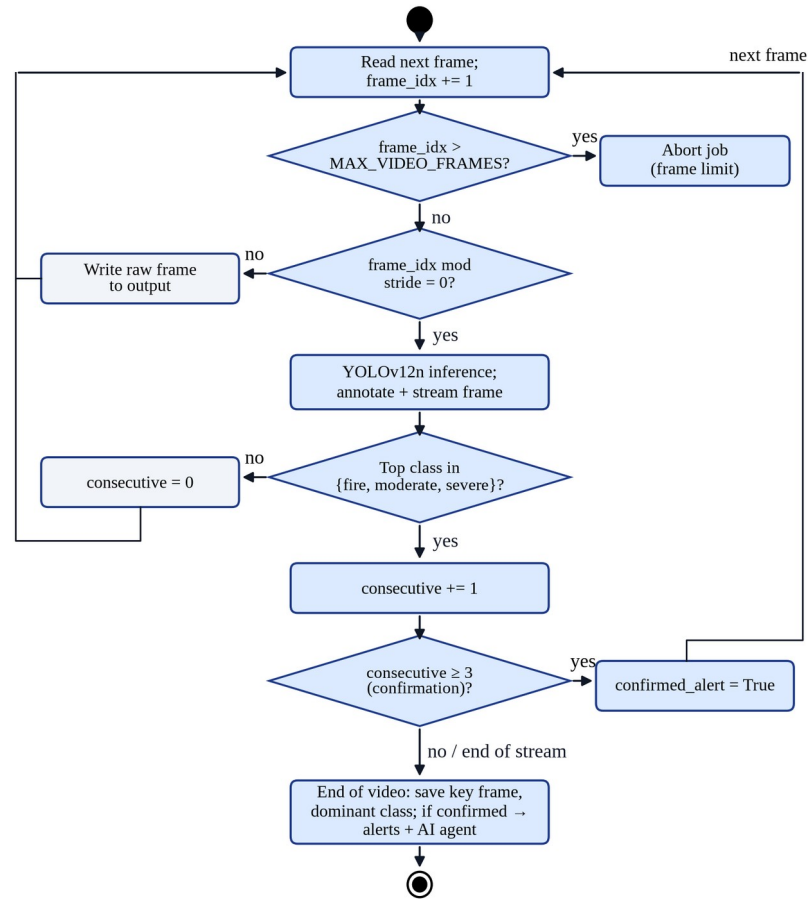


Figure 3.7: Flowchart of frame sampling and three-frame temporal confirmation in video processing

3.12 EMERGENCY ASSESSMENT METHODOLOGY

For every confirmed incident the system produces a structured assessment containing an incident level (CRITICAL, HIGH, MODERATE or LOW), a situation summary, visible hazards, recommended services, immediate actions and a casualty-risk estimate. The methodology deliberately separates *speed* from *richness*:

1. A deterministic rule maps the detected class to a level and to services: severe → CRITICAL (fire department and ambulance, casualties likely); fire → HIGH (fire department and ambulance); moderate → MODERATE (police and ambulance).
2. This rule-based alert is sent immediately over WhatsApp, with up to three attempts and exponential back-off of one and two seconds, so notification never waits for a language model.
3. If a Gemini API key is configured, the incident facts are then sent to the Gemini model with an instruction to return JSON only, with temperature 0 and a timeout. The response is parsed, validated and missing fields are filled from the rule-based report.

4. If the LLM is not configured, times out or returns invalid output, the rule-based assessment is stored and the error is recorded for transparency.
5. The final assessment and the notification outcome are written to the database and rendered into a formal A4 PDF report.

This design satisfies NFR-03: an operator always receives a timely and consistent assessment, and the LLM is used only where it adds value.

3.13 TOOLS AND TECHNOLOGIES

Table 3.7 lists the principal tools, libraries and services used in the project together with the reason for each choice.

Table 3.7: Tools and technologies

Layer	Technology (version)	Reason for selection
Model training	Python 3.12, PyTorch 2.10, Ultralytics 8.4, Kaggle Tesla T4	State-of-the-art YOLO tooling; free GPU
Inference	Ultralytics 8.3, OpenCV 4.10, NumPy 2.1	Model loading, image/video decoding and annotation
Back end	FastAPI 0.111, Uvicorn 0.29, Pydantic 2.9	Fast asynchronous REST API with typed validation
Persistence	MySQL 8, SQLAlchemy 2.0, Alembic 1.13	Reliable relational storage with ORM and migrations
Security	python-jose (JWT), bcrypt 4.1	Standard token authentication; adaptive password hashing
AI agent	LangChain Google GenAI, Google Gemini API	Structured LLM assessment with free tier
Notifications	SMTP (e-mail), Kapso WhatsApp Cloud API, httpx	Widely used channels for responders
Reporting	ReportLab 4.2, Pillow 10.4	A4 PDF reports and WhatsApp report images
Front end	React 18, Vite 7, Tailwind CSS 3.4, Recharts 2, Axios	Responsive single-page dashboard and charts
Testing and quality	pytest 8.2, ESLint 8	Automated regression tests and code linting
Deployment	Docker, Docker Compose, Nginx	Reproducible three-container deployment
Development	Git, Visual Studio Code, Linux	Version control and development environment

SYSTEM ARCHITECTURE

This chapter describes the high-level structure of the system. It explains the architectural design approach and the principles behind it, presents the overall architecture and its decomposition into subsystems with a functional description of each, and shows how data flows through the system using data-flow diagrams. It then describes the deployment architecture, the security architecture and the storage architecture. Low-level design of individual components follows in Chapter 5.

4.1 ARCHITECTURAL DESIGN APPROACH

The system follows a **layered client-server architecture** with a **service-oriented back end**. The client is a single-page web application that runs in the operator's browser; the server exposes a stateless REST API; and the server is internally organised into layers, each of which may depend only on the layer beneath it. This approach was selected over alternatives such as a monolithic server-rendered web application or a set of independently deployed microservices for the following reasons:

- **Separation of concerns.** User-interface code, HTTP handling, domain logic (inference, alerting, assessment) and data access can be developed and tested independently.
- **Statelessness and scalability.** Every request carries its own JWT, so the API layer keeps no session state and additional API instances could be added behind a load balancer.
- **Appropriate complexity.** A layered monolithic back end deployed as one container is far simpler to build, test and operate for a student team than microservices, while still allowing the inference service to be split out later.
- **Replaceability.** The detection engine is isolated behind a single interface, so a new model can be dropped in without changes to the API, the database or the user interface.

Several design principles were applied consistently: **least privilege** (each role sees only what it needs); **fail-safe defaults** (notification and record keeping continue even if optional services fail); **defence in depth** (validation at the client, the API and the database); and **configuration over code** (all deployment-specific values are supplied as environment variables).

4.2 HIGH-LEVEL ARCHITECTURE

Figure 4.1 shows the layered architecture. The **presentation layer** is a React 18 single-page application served by Nginx. It communicates exclusively with the **API layer**, a FastAPI application whose routers group endpoints by resource (/auth, /detection, /agent, /alerts, /dashboard and /users). The API layer delegates work to the **service layer**, which contains the domain logic: the detection engine, the video job manager, the alert service, the emergency agent and the report generator. The **data-access layer** uses the SQLAlchemy ORM and a small set of CRUD helper functions to read and write the seven database tables, with the schema managed by Alembic migrations. Metadata is stored in MySQL, while media files are stored on the file system, and the service layer calls external services for e-mail, WhatsApp and LLM assessment.

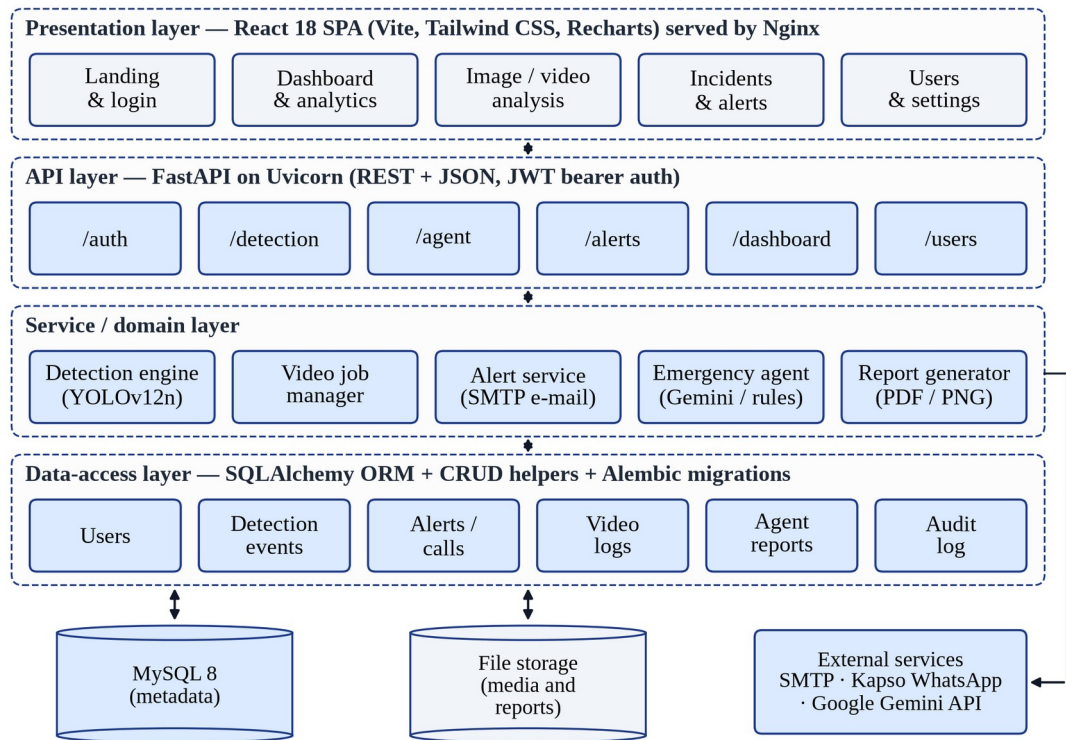


Figure 4.1: Layered architecture of the AI accident detection and emergency response system

4.3 SUBSYSTEM DECOMPOSITION

The system is decomposed into seven subsystems, listed in Table 4.1. The following subsections give a functional description of each.

Table 4.1: Subsystems and their responsibilities

ID	Subsystem	Responsibilities
S1	Web client (presentation)	Authentication screens, analysis workflows, incident triage, analytics, user management, live status
S2	Identity and access	Registration, login, token issue and verification, roles, ownership filtering

ID	Subsystem	Responsibilities
S3	Detection	Model loading, image validation and inference, annotation of evidence
S4	Video processing	Background jobs, frame sampling, live MJPEG stream, temporal confirmation, job lifecycle
S5	Emergency response	E-mail alerts, rule/LLM assessment, WhatsApp dispatch, PDF and image reports, audit
S6	Incident management and analytics	Incident history, status transitions, statistics, timelines, confidence distribution, delivery logs
S7	Persistence and storage	Database schema, CRUD operations, migrations, private and public file storage

4.3.1 S1 – Web client

The web client provides the landing page, the login and registration screen and an authenticated application shell with a sidebar and a top bar. The top bar polls the public health endpoint every 30 seconds and displays whether the model is online. The shell contains eight routed pages: Dashboard, Analytics, Image Analysis, Video Analysis, Incidents, Alerts Center, Users (administrators only) and Settings. The access token is kept only in memory rather than in browser storage, which limits the impact of a cross-site scripting vulnerability; an expired token causes an automatic redirect to the login page.

4.3.2 S2 – Identity and access

This subsystem registers users with bcrypt-hashed passwords, authenticates them and issues HS256-signed JWTs that expire after 60 minutes. A dependency attached to every protected endpoint decodes the token, loads the user and rejects inactive accounts. A second dependency restricts administrative endpoints. Data endpoints apply an ownership filter so that non-administrators receive only their own incidents, alerts, reports and statistics.

4.3.3 S3 – Detection

The detection subsystem loads the YOLOv12n weights once at start-up into a thread-safe singleton. For images it validates the extension, the declared MIME type, the file size and the pixel count, decodes the image, runs inference in a worker thread so that the server remains responsive, draws colour-coded boxes and labels, and stores the annotated image. The class with the highest confidence becomes the image's detected class. If the model file is missing, inference safely returns 'no detection' and the health endpoint reports that the model is not loaded.

4.3.4 S4 – Video processing

Videos are saved in chunks to private storage with a 100 MB limit, probed to ensure they can be decoded, and handed to a background job. The job manager tracks the status, stage, progress and frame counts of each job, keeps the latest annotated frame for the live MJPEG stream, applies the confirmation algorithm of Section 3.11, persists the result, deletes the original upload and removes finished jobs from memory after one hour.

4.3.5 S5 – Emergency response

When an incident is confirmed, this subsystem sends an HTML e-mail with the annotated image embedded, runs the emergency agent described in Section 3.12, dispatches a WhatsApp message through the Kapso gateway (or a mock gateway during development), renders a WhatsApp report image and an A4 PDF report, and records every outcome in the agent-report and audit tables. It is designed so that any failure is recorded rather than propagated.

4.3.6 S6 – Incident management and analytics

Operators can list incidents with their evidence, filter them by class or status, and move an incident from *open* to *acknowledged*, *resolved* or *false alarm*; each transition is audited. Aggregation endpoints compute key performance indicators (totals, today's detections, average confidence, alerts, reports and WhatsApp messages), per-class daily timelines over 7 to 90 days, the confidence histogram and delivery logs per channel.

4.3.7 S7 – Persistence and storage

The persistence subsystem owns the relational schema (Section 5.4) and the file-system layout. Relational data and file data are deliberately separated: the database stores only metadata and relative paths, never binary media, which keeps it small and fast to back up.

4.4 DATA-FLOW ARCHITECTURE

The context diagram in Figure 4.2 treats the whole system as a single process and shows its interactions with external entities. Operators provide credentials, media and locations and receive results, incidents, analytics and reports. Administrators maintain users and test channels. The system sends alert e-mails to an SMTP server, incident messages to the WhatsApp gateway and incident context to the Gemini API, and receives acknowledgements, message identifiers and JSON assessments in return.

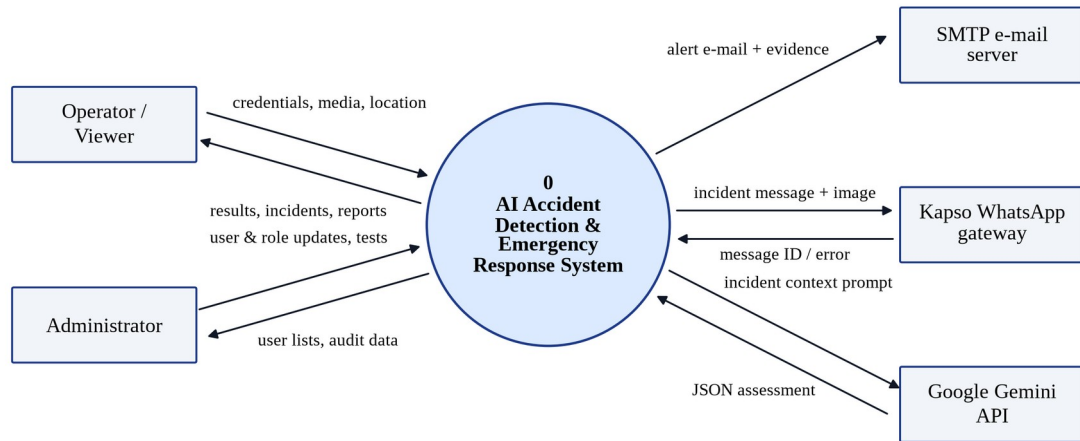


Figure 4.2: Context-level data-flow diagram (DFD level 0)

Figure 4.3 decomposes the system into six processes. Process 1.0 authenticates the user against the users store (D1). Process 2.0 validates and stores uploads; video originals are kept in private storage (D2). Process 3.0 runs inference and writes annotated evidence to public generated storage (D3). Process 4.0 records the incident in the detection tables (D4). Process 5.0 assesses the incident, dispatches alerts through external services and writes alert, assessment and audit records (D5). Process 6.0 produces reports and visualisations for the operator, storing PDF files in the reports store (D6).

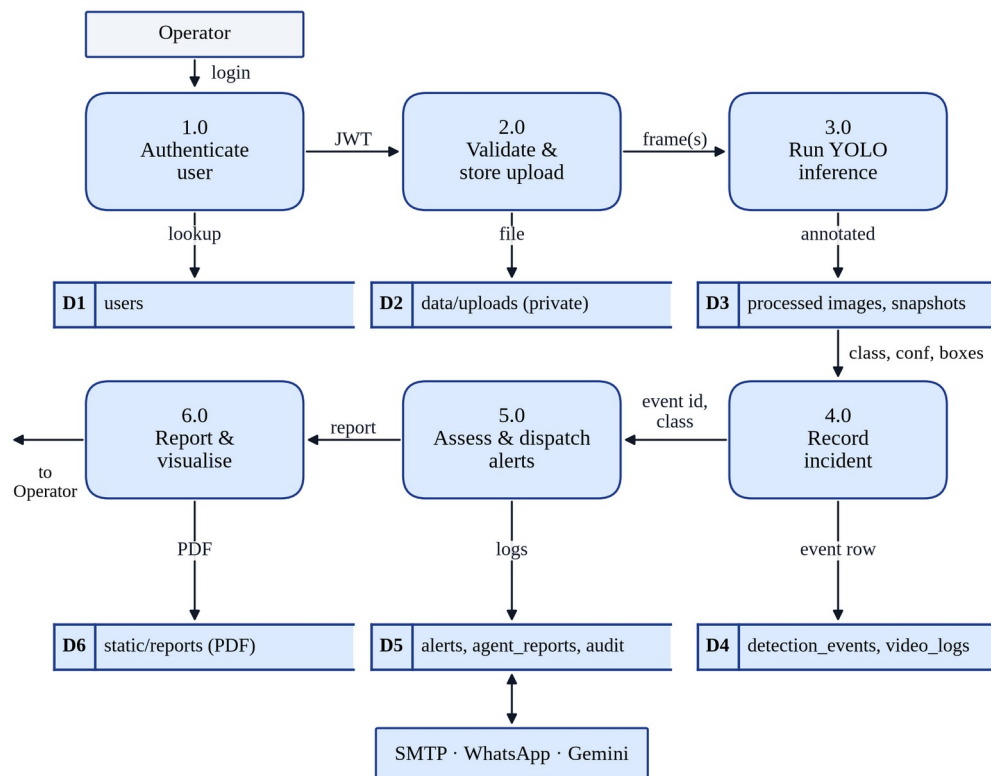


Figure 4.3: Level-1 data-flow diagram showing the main processes and data stores

4.5 DEPLOYMENT ARCHITECTURE

The system is packaged as three Docker containers orchestrated with Docker Compose on a private bridge network, as shown in Figure 4.4. The MySQL container is health-checked, and the back end starts only once the database is healthy. Persistent data survives container rebuilds in three named volumes. The trained model is mounted from the host so that it can be replaced without rebuilding the image. The back end runs a single Uvicorn worker because video jobs are held in process memory (see Section 8.5).

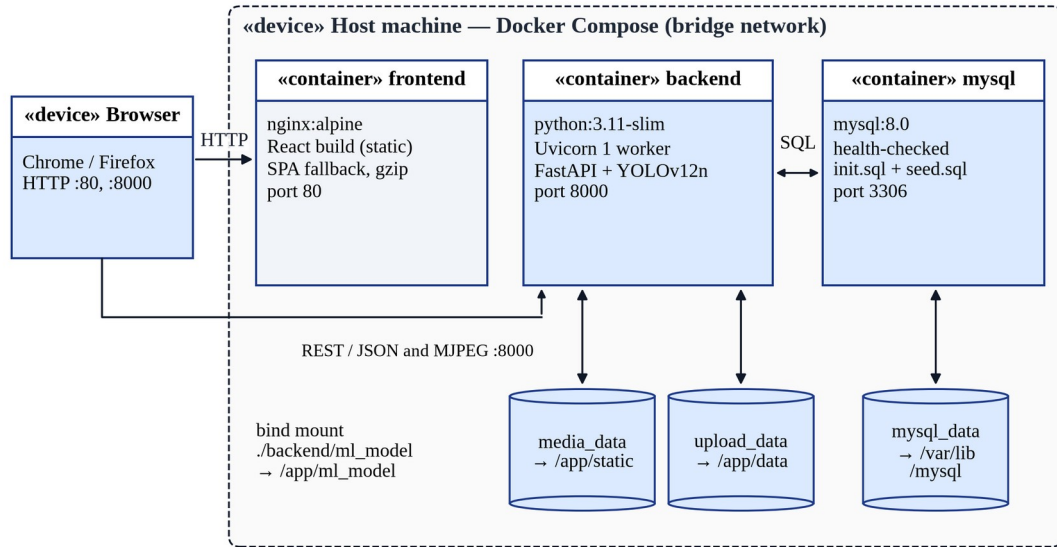


Figure 4.4: Deployment diagram of the Docker Compose stack

4.6 SECURITY ARCHITECTURE

Security controls are applied in several layers so that the failure of any one control does not expose the system. Table 4.2 summarises the layers.

Table 4.2: Security layers and controls

Layer	Controls
Network / HTTP	Trusted-host allow-list; CORS restricted to configured origins; security headers (nosniff, frame options, referrer and permissions policy, cross-origin opener policy, no-store caching for auth and agent responses; HSTS in production); API documentation disabled by default
Authentication	bcrypt password hashing (cost 12); passwords limited to 72 bytes; HS256 JWT with 60-minute expiry; inactive accounts rejected
Authorisation	Role checks for administrative endpoints; ownership filtering on all incident, alert, report, statistics and PDF endpoints; public registration cannot create administrators; administrators cannot deactivate or demote themselves
Input validation	Extension, MIME type, size (100 MB), pixel-count (25 MP) and frame-count (9,000) limits; corrupt media rejected before processing; Pydantic schema validation of all JSON input

Layer	Controls
Storage	Original uploads kept outside the web root; incident PDFs blocked from direct static access and served only through an authorised endpoint; generated file names randomised; secrets supplied by environment and excluded from version control
Output	HTML-escaping of user-supplied values in e-mails and PDFs to prevent injection

4.7 STORAGE ARCHITECTURE

The MySQL database holds only metadata and is reached through the API alone; original uploads are kept in a private directory that is never served and are deleted after processing; annotated images and key frames are served under random file names; incident PDFs are available only through an authorised endpoint; and the model weights are read by the back end only. Separating private uploads from generated, shareable evidence is an important privacy decision: the original video from a camera may contain unrelated people and vehicles and is therefore deleted after processing, whereas the annotated key frame is kept as evidence for the incident.

DETAILED SYSTEM DESIGN

Where Chapter 4 described the system at the level of layers and subsystems, this chapter specifies its internal design. It first describes each software component using the ten design-entity attributes of IEEE Std 1016 [25]. It then presents the static structure of the back end as a UML class diagram, the database design as an entity–relationship diagram with a data dictionary, and the dynamic behaviour of the system as sequence, activity and state diagrams. The chapter closes with the design of the REST API and of the user interface.

5.1 COMPONENT DESIGN APPROACH

IEEE Std 1016 describes each design entity by a fixed set of attributes so that every component is documented consistently [25]. The ten attributes used in this chapter are: **identification** (a unique name); **type** (the kind of component, such as a module, class or service); **purpose** (why the component exists, traced to requirements); **function** (what it does); **subordinates** (the entities it is composed of); **dependencies** (the entities it uses or requires); **interface** (how other components interact with it); **resources** (external resources it needs, such as files, memory or services); **processing** (how it performs its function, including algorithms and error handling); and **data** (the data it owns, reads or writes). Ten principal components, C1 to C10, are specified in the following section.

5.2 COMPONENT SPECIFICATIONS

Table 5.1: Component C1 – Authentication and access control

Attribute	Description
Identification	C1 — auth routes, security module and access dependencies (backend/app/api/routes/auth.py, core/security.py, api/deps.py)
Type (classification)	Back-end module group (API router + security utilities + dependency functions)
Purpose	Ensures that only registered, active users can use the system and that each user can only perform permitted actions (FR-01, FR-02, FR-17; NFR-04, NFR-05)
Function (responsibilities)	Registers users; verifies passwords; issues and decodes JWTs; resolves the current user for each request; enforces the administrator role
Subordinates	register, login and me endpoints; hash_password, verify_password, create_access_token, decode_access_token; get_current_user, require_admin
Dependencies	C8 Persistence (user look-up); Pydantic schemas; bcrypt; python-jose; application settings (SECRET_KEY, algorithm, expiry)
Interface	POST /auth/register (JSON), POST /auth/login (form-urlencoded, returns token and user), GET /auth/me; injected as FastAPI dependencies into all protected routes
Resources	SECRET_KEY from the environment; CPU time for bcrypt (cost factor 12)

Attribute	Description
Processing	Registration rejects duplicate usernames or e-mails and forces the viewer role. Login compares the bcrypt hash in constant time, rejects inactive accounts with 403 and returns a token with subject, role and 60-minute expiry. Each protected request decodes the token; invalid or expired tokens and inactive users yield 401.
Data	Reads and writes the users table; the token payload contains sub (username), role and exp

Table 5.2: Component C2 – Detection engine

Attribute	Description
Identification	C2 — DetectionEngine (backend/app/core/detection_engine.py)
Type (classification)	Class implementing the Singleton pattern; domain service
Purpose	Encapsulates the trained YOLOv12n model so that the rest of the system is independent of the ML framework (FR-04, FR-05, FR-20; NFR-01, NFR-02)
Function (responsibilities)	Loads the model once; runs inference on a BGR frame; converts raw detections to a class, confidence and box list; draws annotations; flags whether an alert is required
Subordinates	get_instance(), predict_frame(frame), _annotate(frame, boxes); class list [fire, moderate, severe]; colour map
Dependencies	Ultralytics YOLO, OpenCV, NumPy; settings (MODEL_PATH, confidence 0.45, IoU 0.45, image size 640)
Interface	predict_frame(ndarray) → {detected_class, confidence, bounding_boxes, annotated_frame, processing_ms, alert_required}
Resources	Model file best.pt (5.5 MB) loaded into memory; CPU or GPU; a threading lock serialising inference
Processing	Double-checked locking creates a single instance. Inference runs under a lock because the predictor is not thread-safe. The highest-confidence box defines the frame's class; 'no_detection' is returned with confidence 0 when nothing is found, when the model is missing or when inference raises an error, so callers never crash.
Data	No persistent data; returns per-frame results; class order must match the training data

Table 5.3: Component C3 – Image detection controller

Attribute	Description
Identification	C3 — detection router, image endpoint (backend/app/api/routes/detection.py)
Type (classification)	API controller (FastAPI route handler)
Purpose	Provides the image-analysis use case end to end (UC-03; FR-03 to FR-05, FR-09, FR-19)
Function (responsibilities)	Validates the upload, runs inference off the event loop, saves the annotated image, records the incident and schedules alerting in the background
Subordinates	_check_file (extension/MIME), image decoding and size/pixel checks, background-task scheduling, response serialisation
Dependencies	C1 (current user), C2 (inference), C5 and C6 (alerts and assessment), C8 (persistence); OpenCV
Interface	POST /detection/image?location=... (multipart file, Bearer token) → {event_id, detected_class, confidence, bounding_boxes, processed_url, processing_ms, alert_triggered, agent_pending}
Resources	Temporary memory for at most 100 MB + 1 byte; static/processed directory; thread pool
Processing	Rejects bad extensions or mismatched MIME types (400), oversized files (413), undecodable images (400) and images above 25 MP (413). Runs inference in a worker thread, writes the annotated JPEG with a random name, inserts a detection event and, if an alert is required, creates a pending assessment and schedules the e-mail alert and emergency agent as background tasks, then responds immediately.
Data	Writes detection_events and agent_reports (pending); writes an annotated image file

Table 5.4: Component C4 – Video job manager and video processor

Attribute	Description
Identification	C4 — video_jobs module and VideoProcessor class (backend/app/core/video_jobs.py, core/video_processor.py) with the video endpoints
Type (classification)	Service module with an in-memory job registry, plus a processing class
Purpose	Provides asynchronous video analysis with live feedback and false-alarm control (UC-04; FR-06 to FR-08)
Function (responsibilities)	Creates jobs; processes frames with stride and confirmation; publishes progress and the latest annotated frame; streams frames as MJPEG; persists results; cleans up uploads and old jobs
Subordinates	create_job, get_job, process_job, stream_job, _prune_finished_jobs; VideoProcessor.process
Dependencies	C2 (per-frame inference), C5, C6, C8; OpenCV VideoCapture/VideoWriter; FastAPI BackgroundTasks and StreamingResponse
Interface	POST /detection/video → {job_id, status}; GET /detection/video/jobs/{id} → {status, stage, progress, processed_frames, total_frames, result, error}; GET /detection/video/jobs/{id}/stream → multipart/x-mixed-replace JPEG stream
Resources	Private upload storage; process memory for job state and one JPEG per job; CPU for inference on every second frame
Processing	Implements the algorithm of Section 3.11 (stride 2, confirmation 3, 9,000-frame limit). Progress is mapped to 2–95% during frame analysis and 97–100% while saving. On success it stores the event and a video log; on failure it records the error. In both cases the upload is deleted and the job is marked with a finish time so that it is pruned after one hour. Job IDs are random 128-bit UUIDs.
Data	In-memory job dictionary; writes detection_events and video_processing_logs; writes processed video and key-frame snapshot files

Table 5.5: Component C5 – Alert service

Attribute	Description
Identification	C5 — alert_service module (backend/app/core/alert_service.py) and call_service module
Type (classification)	Service module
Purpose	Notifies responders by e-mail with visual evidence and keeps a delivery log (FR-10, FR-16; NFR-03, NFR-10)
Function (responsibilities)	Builds an HTML e-mail with class, confidence, event ID, file name, time, severity and an embedded annotated image; sends it via SMTP with STARTTLS; records the outcome; records simulated call-audit entries
Subordinates	maybe_send_alert, _send_email; _log_mock_call
Dependencies	C8 Persistence; Python smtplib and email packages; settings (SMTP host, port, user, password, recipient, FRONTEND_URL)
Interface	maybe_send_alert(event_id, class, confidence, filename, image_path) → None; _send_email(...) → 'sent' 'failed' 'skipped'
Resources	SMTP server and credentials (optional); annotated image file
Processing	If SMTP is not configured the alert is skipped and logged as such. User-supplied values are HTML-escaped. Any exception is caught, and the status and error are always written to the alerts table; on success the event's alert_sent flag is set.
Data	Writes the alerts and call_logs tables; updates detection_events.alert_sent and call_triggered

Table 5.6: Component C6 – Emergency agent and WhatsApp service

Attribute	Description
Identification	C6 — emergency_agent and whatsapp_service modules (backend/app/core/emergency_agent.py, core/whatsapp_service.py)
Type (classification)	Service modules (orchestrator + gateway adapter)

Attribute	Description
Purpose	Produces a structured emergency assessment and dispatches it to responders over WhatsApp without ever blocking on the LLM (FR-11, FR-12; NFR-03)
Function (responsibilities)	Builds rule-based alerts; sends WhatsApp messages with retries; requests and validates Gemini assessments; persists the final report and audit entry; handles idempotency and failures
Subordinates	run_emergency_agent, build_agent_report, _rule_based_report, _gemini_report, _normalise_report; format_whatsapp_message, send_whatsapp, kapso_configured
Dependencies	C7 (PDF and image rendering), C8 (persistence); LangChain Google GenAI; httpx; settings (AGENT_ENABLED, GEMINI_API_KEY, GEMINI_MODEL, timeout, WHATSAPP_MODE, Kapso credentials)
Interface	run_emergency_agent(event_id, class, confidence, media_path, media_type, filename) → None; send_whatsapp(message, media_url) → {sent, sid, error, status}
Resources	Kapso WhatsApp Cloud API (live mode) or mock gateway; Google Gemini API (optional); network access; the PDF output directory
Processing	Implements the activity in Figure 5.6: skip if a WhatsApp message was already sent for the event; send a rule-based alert with three attempts and back-off; then build the Gemini or rule-based assessment; generate the PDF; persist report and audit log; mark the event. Any unexpected error stores an 'error-fallback' HIGH-level report so that the operator always sees a result.
Data	Reads detection_events; writes agent_reports and audit_log; updates detection_events.whatsapp_sent

Table 5.7: Component C7 – Report generator

Attribute	Description
Identification	C7 — report_generator module (backend/app/core/report_generator.py)
Type (classification)	Utility service module
Purpose	Produces formal, printable documentation of each incident and a compact image suitable for messaging apps (FR-13)
Function (responsibilities)	Renders an A4 PDF with incident reference, time, location, classification, level, confidence and status; executive assessment; hazards; recommended services and actions; notification record; annotated evidence; operational notice. Renders a 1200 × 1500 PNG WhatsApp report.
Subordinates	generate_incident_pdf, generate_whatsapp_report_image, text-wrapping and escaping helpers, page footer
Dependencies	ReportLab, Pillow; settings (REPORTS_DIR)
Interface	generate_incident_pdf(event, report, image_path) → file path; generate_whatsapp_report_image(event, report, evidence_path) → file path
Resources	File system (reports directory); fonts
Processing	All text is HTML-escaped before being placed in ReportLab paragraphs. The notification record distinguishes a simulated (mock) message, a message accepted by the provider and a failed dispatch, so the report never overstates delivery. WhatsApp images receive a random suffix so they cannot be guessed.
Data	Reads the event and report; writes incident-<id>.pdf and whatsapp-incident-<id>-<random>.png

Table 5.8: Component C8 – Persistence layer

Attribute	Description
Identification	C8 — database module, ORM models and CRUD helpers (backend/app/db/database.py, models.py, crud.py) with Alembic migrations
Type (classification)	Data-access layer (Repository pattern over SQLAlchemy ORM)
Purpose	Provides reliable, consistent storage and retrieval of all system data with ownership filtering (FR-09, FR-14; NFR-05, NFR-08)

Attribute	Description
Function (responsibilities)	Defines the seven entities; opens and closes sessions per request; performs create, read, update and delete operations; applies owner filters; upserts assessments
Subordinates	Engine and SessionLocal; entities User, DetectionEvent, Alert, CallLog, VideoLog, AgentReport, AuditLog; about 30 CRUD functions
Dependencies	SQLAlchemy 2.0, PyMySQL, MySQL 8 (SQLite for tests); Alembic
Interface	get_db() dependency yielding a session; functions such as create_detection_event, get_detection_events(user_id), update_incident_status, create_agent_report, get_alerts(user_id)
Resources	MySQL server; connection pool (10 connections, 20 overflow, pre-ping, 1-hour recycle)
Processing	Each request receives its own session, closed after the response; background tasks open their own sessions. Duplicate usernames or e-mails are translated from integrity errors into user-friendly errors. Assessments are upserted so that a pending report is replaced by the final one.
Data	All seven tables described in Section 5.4

Table 5.9: Component C9 – Dashboard and analytics service

Attribute	Description
Identification	C9 — dashboard, alerts, agent-report and users routers (backend/app/api/routes/dashboard.py, alerts.py, agent.py, users.py)
Type (classification)	API controllers (read-mostly aggregation endpoints)
Purpose	Supplies the data for monitoring, analytics, delivery logs, report download and user administration (FR-14 to FR-18)
Function (responsibilities)	Computes KPIs and per-class counts; builds daily timelines and confidence histograms; lists e-mail, call and WhatsApp logs; returns and streams assessments and PDFs; lets administrators list and update users and test channels
Subordinates	get_stats, get_timeline, get_confidence_distribution; get_alerts, get_call_logs, test_email, test_call; get_report, get_report_pdf, get_reports, agent_status, test_whatsapp; list_users, update_user, delete_user
Dependencies	C1 (current user, admin check), C5, C6, C8
Interface	GET /dashboard/stats, /dashboard/timeline?days=1..90, /dashboard/confidence-distribution; GET /alerts, /alerts/calls; GET /agent/report/{id}, /agent/report/{id}/pdf, /agent/reports; GET/PUT/DELETE /users
Resources	Database read capacity; PDF files
Processing	Every query is filtered by owner for non-administrators. A missing assessment for an existing event returns a ‘PENDING’ placeholder so the client can keep polling. PDF download checks ownership and file existence before streaming. Administrators cannot deactivate or demote their own account.
Data	Reads all tables; updates users; no data owned exclusively

Table 5.10: Component C10 – Web client (single-page application)

Attribute	Description
Identification	C10 — React application (frontend/src): pages, components, API modules and authentication context
Type (classification)	Client-side application (component-based UI)
Purpose	Gives operators and administrators an efficient, accessible interface to every use case (all UCs; NFR-06, NFR-09)
Function (responsibilities)	Routing and access guards; login and registration; image and video analysis with live feedback; incident triage; analytics charts; delivery logs; user management; settings; system-health indicator; emergency toasts
Subordinates	App shell (Sidebar, Navbar); 10 pages; shared components (SeverityBadge, StatsCard, DropZone, ScanOverlay, AgentReportCard, DetectionCard, ProgressBar, PageHeader); API modules (auth, detection, alert, agent)

Attribute	Description
Dependencies	React 18, React Router, Axios, Recharts, react-dropzone, react-hot-toast, Tailwind CSS; the REST API
Interface	Browser routes /, /login, /dashboard, /analytics, /detect/image, /detect/video, /history, /alerts, /users, /settings; HTTP calls to the API with a Bearer token
Resources	Browser; network access to the API; in-memory token store
Processing	An Axios interceptor attaches the token and redirects to login on HTTP 401. Video analysis polls the job every 0.7 s and shows the MJPEG stream; the AI assessment is polled once per second for up to 90 s. Charts zero-fill missing days so that trends are always drawn over the whole period.
Data	Holds the token and user profile in memory only; no persistent browser storage

5.3 CLASS DIAGRAM

Figure 5.1 shows the static structure of the back end as a UML class diagram. Entity classes (green) are SQLAlchemy ORM models that map to database tables; service classes (blue) contain the domain logic; and the repository (amber) encapsulates data access. A User creates zero or more DetectionEvents. A DetectionEvent owns at most one AgentReport, Alert, CallLog and VideoLog, which are deleted with it, and may have several AuditLog entries. The VideoJobManager runs the VideoProcessor, which uses the singleton DetectionEngine; the job manager and the image controller call the AlertService and the EmergencyAgent, and the EmergencyAgent uses the ReportGenerator and the WhatsApp service. All services persist their results through the repository.

The design uses several well-known patterns. The **Singleton** pattern ensures that the 5.5 MB model is loaded only once; the **Repository** pattern keeps SQL out of the controllers; **Dependency Injection** (FastAPI dependencies) supplies the database session and the current user to every endpoint; the **Adapter** pattern hides the Kapso WhatsApp API and the Gemini SDK behind simple functions; and the **Strategy**-like fallback in the emergency agent selects either the LLM or the rule-based assessment at run time.

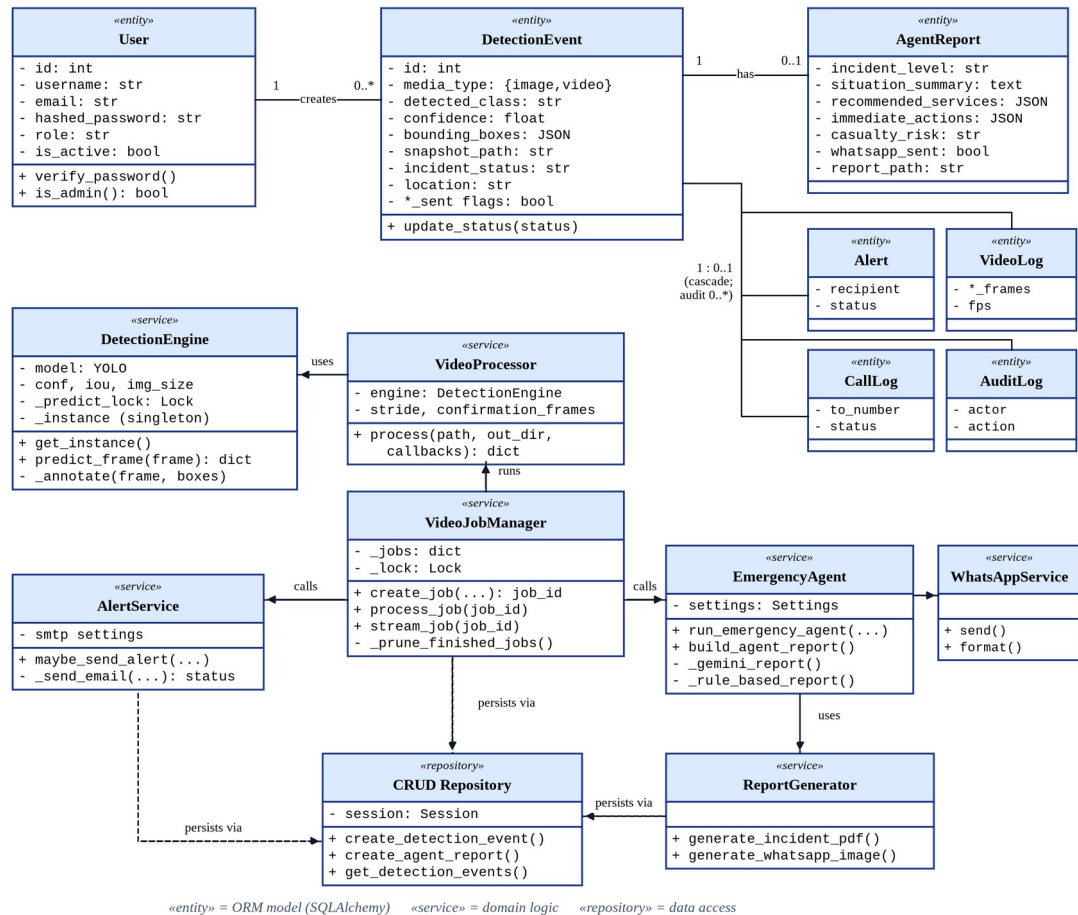


Figure 5.1: UML class diagram of the back end

5.4 DATABASE DESIGN

The database was designed from the entities identified in the requirements and normalised to the third normal form (3NF): every non-key attribute depends on the key, the whole key and nothing but the key. Information that occurs at most once per incident but only for some incidents (e-mail alert, call audit, video statistics and AI assessment) is stored in separate tables with a one-to-zero-or-one relationship rather than as many nullable columns, while genuinely variable-length structures (bounding boxes, recommended services, immediate actions and audit details) are stored as JSON columns. Figure 5.2 shows the entity–relationship diagram.

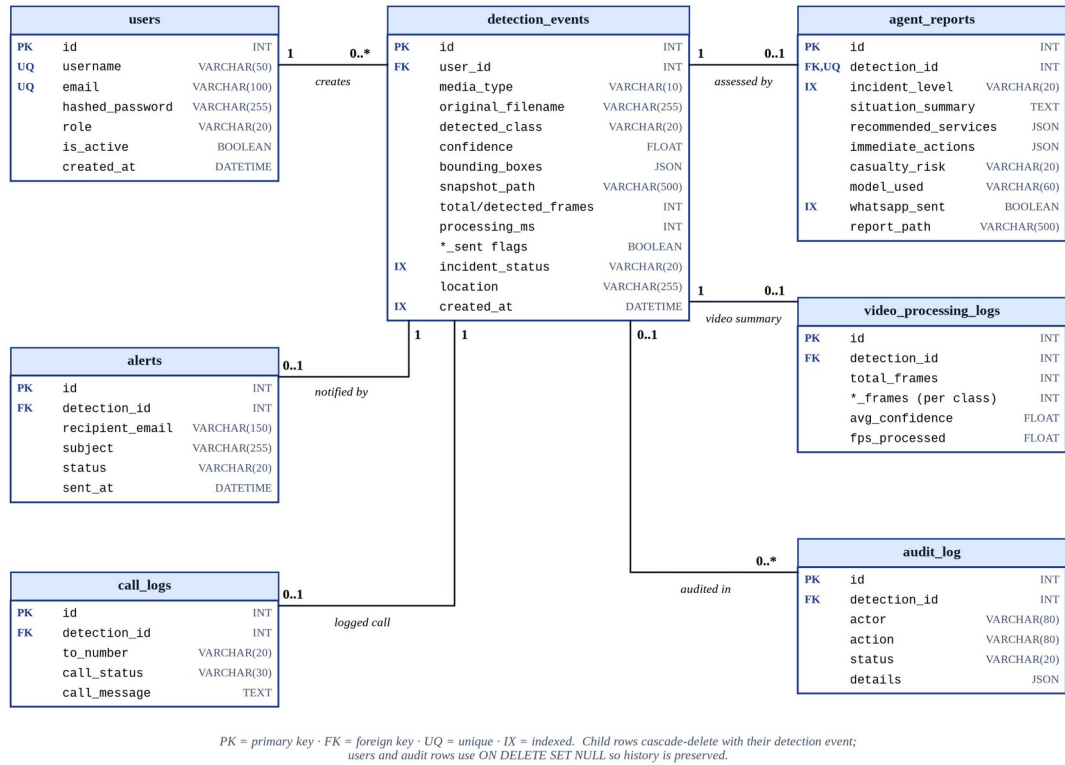


Figure 5.2: Entity–relationship diagram of the database

Referential integrity is enforced with foreign keys. Deleting a detection event cascades to its alert, call log, video log and assessment, whereas users and audit entries use ON DELETE SET NULL so that historical records are preserved if an account or incident is removed. Indexes support the most frequent queries: incidents by status and by creation time, assessments by level and WhatsApp status, and audit entries by incident and time. Table 5.11 summarises the tables and Table 5.12 gives the data dictionary of the central table.

Table 5.11: Database tables

Table	Purpose	Relationship
users	Accounts with hashed password, role and active flag	1 : N events
detection_events	One row per analysed image or video: class, confidence, boxes, evidence, status, location	Central entity
alerts	E-mail notification attempts and outcomes	1 : 0..1
call_logs	Local audit of simulated emergency calls	1 : 0..1
video_processing_logs	Per-class frame counts, average confidence and processing rate	1 : 0..1
agent_reports	Structured AI assessment, model used, WhatsApp outcome, PDF path	1 : 0..1 (unique)
audit_log	Append-only record of agent and operator actions	1 : N

Table 5.12: Data dictionary of the detection_events table

Column	Type	Null	Description
id	INT, PK, auto-increment	No	Incident identifier (displayed as INC-0001)
user_id	INT, FK → users.id	Yes	Owner; set to NULL if the user is deleted
media_type	VARCHAR(10)	No	‘image’ or ‘video’
original_filename	VARCHAR(255)	Yes	Name of the uploaded file
processed_filename	VARCHAR(255)	Yes	Name of the annotated output file
detected_class	VARCHAR(20)	No	fire, moderate, severe or no_detection
confidence	FLOAT	No	Top confidence (image) or average confidence (video), 0–1
bounding_boxes	JSON	Yes	List of {class, confidence, x1, y1, x2, y2}
snapshot_path	VARCHAR(500)	Yes	Annotated image or video key frame
total_frames / detected_frames	INT	Yes	Video only: total frames and frames with detections
dominant_class	VARCHAR(20)	Yes	Video only: most frequent accident class
processing_ms	INT	Yes	Inference time in milliseconds
alert_sent / call_triggered / whatsapp_sent	BOOLEAN	No	Notification flags
incident_status	VARCHAR(20), indexed	No	open, acknowledged, resolved or false_alarm
location	VARCHAR(255)	No	Operator-entered location; default ‘Unknown’
created_at	DATETIME, indexed	No	Creation time (server default)

5.5 DYNAMIC BEHAVIOUR

5.5.1 Authentication sequence

Figure 5.3 shows the login interaction. The browser sends the credentials as a form; the API loads the user, verifies the bcrypt hash, checks that the account is active and returns a signed token together with the user profile. The client keeps the token only in memory and navigates to the dashboard.

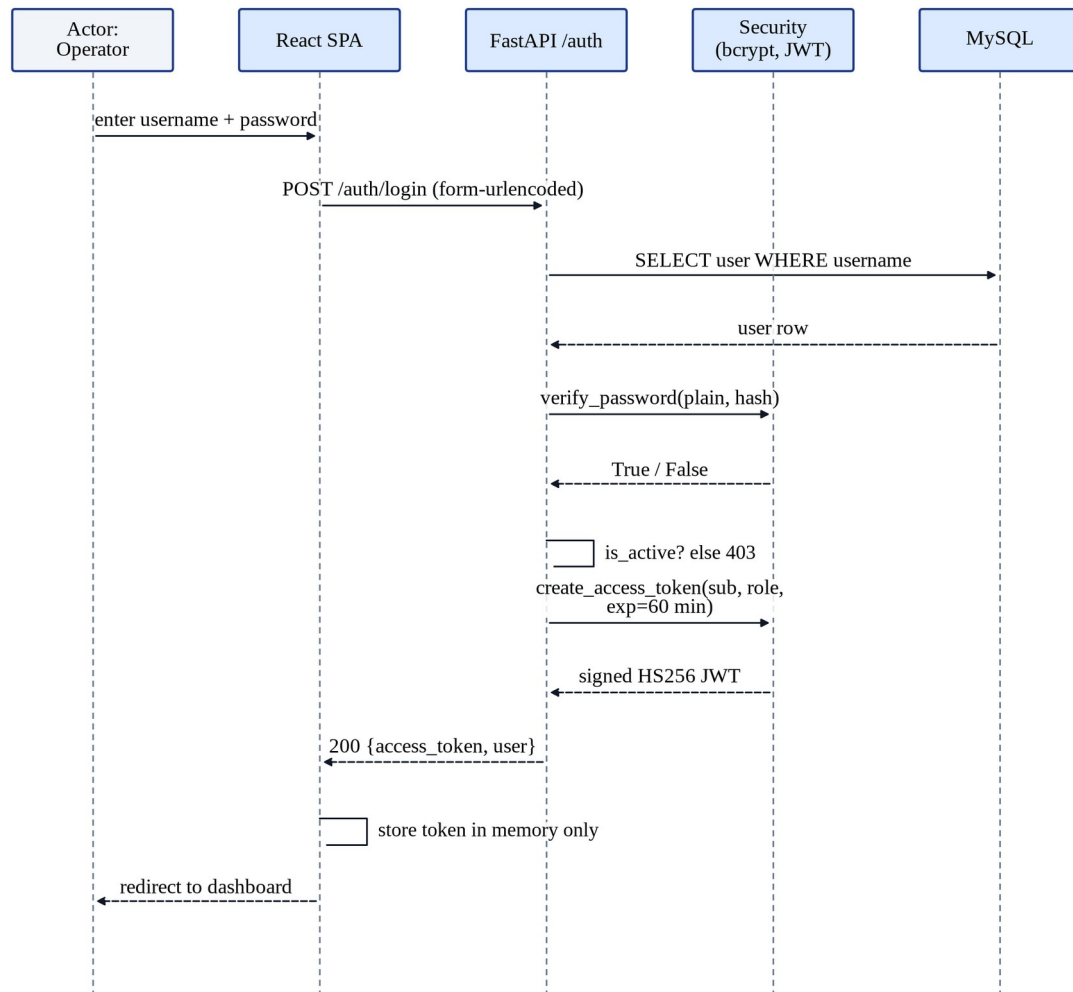


Figure 5.3: Sequence diagram: user authentication

5.5.2 Image detection sequence

Figure 5.4 shows the image-analysis interaction. Validation and inference complete within the request, so the operator sees the verdict in about a second, while alerting and the AI assessment run as background tasks. The client then polls the assessment endpoint until the final report is available.

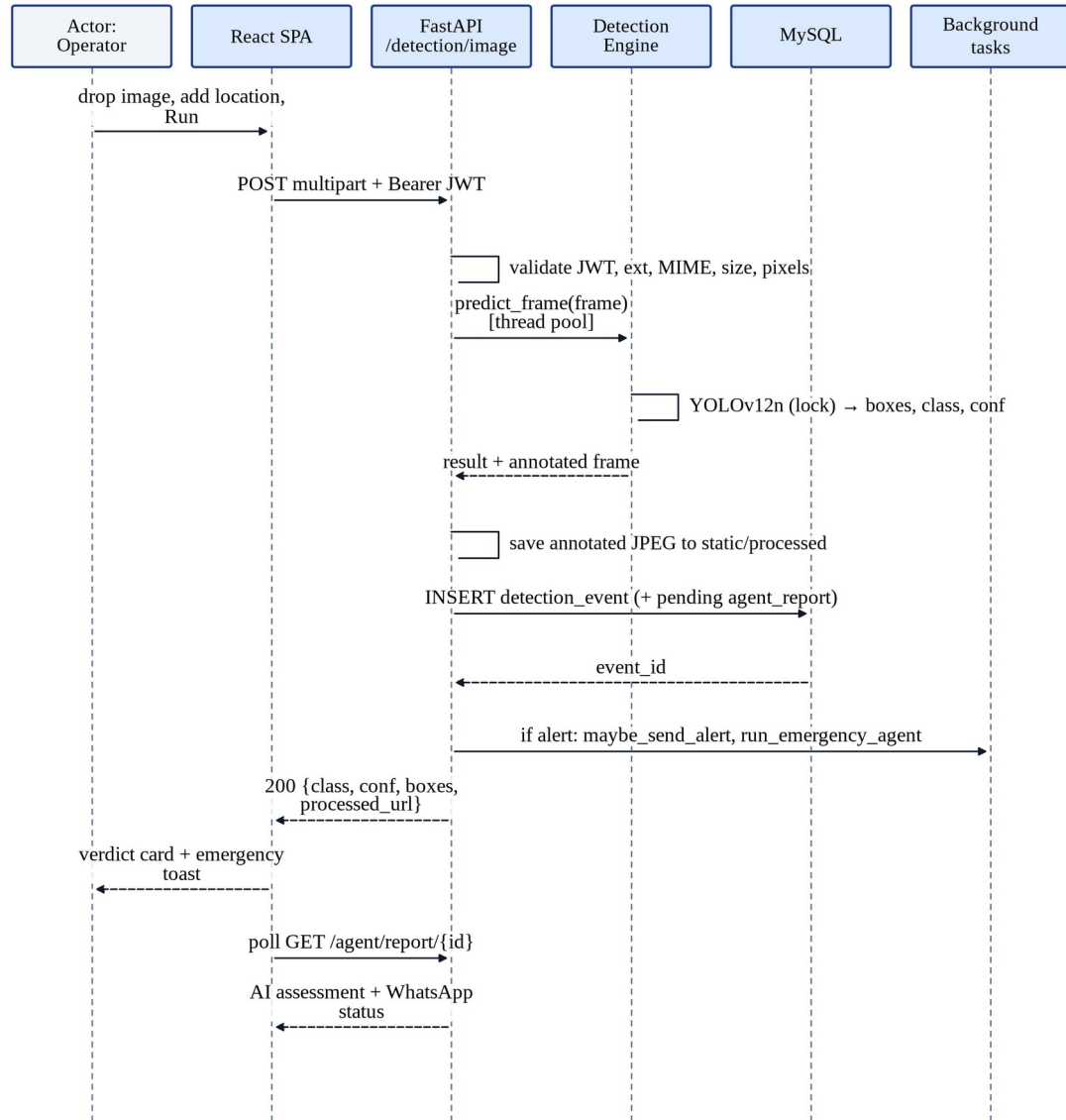


Figure 5.4: Sequence diagram: image detection and alerting

5.5.3 Video detection sequence

Figure 5.5 shows the video interaction. The upload request returns as soon as a job has been created. The browser then opens the live MJPEG stream and polls the job status, while the video processor analyses sampled frames, applies the confirmation rule and finally triggers alerts only for confirmed incidents.

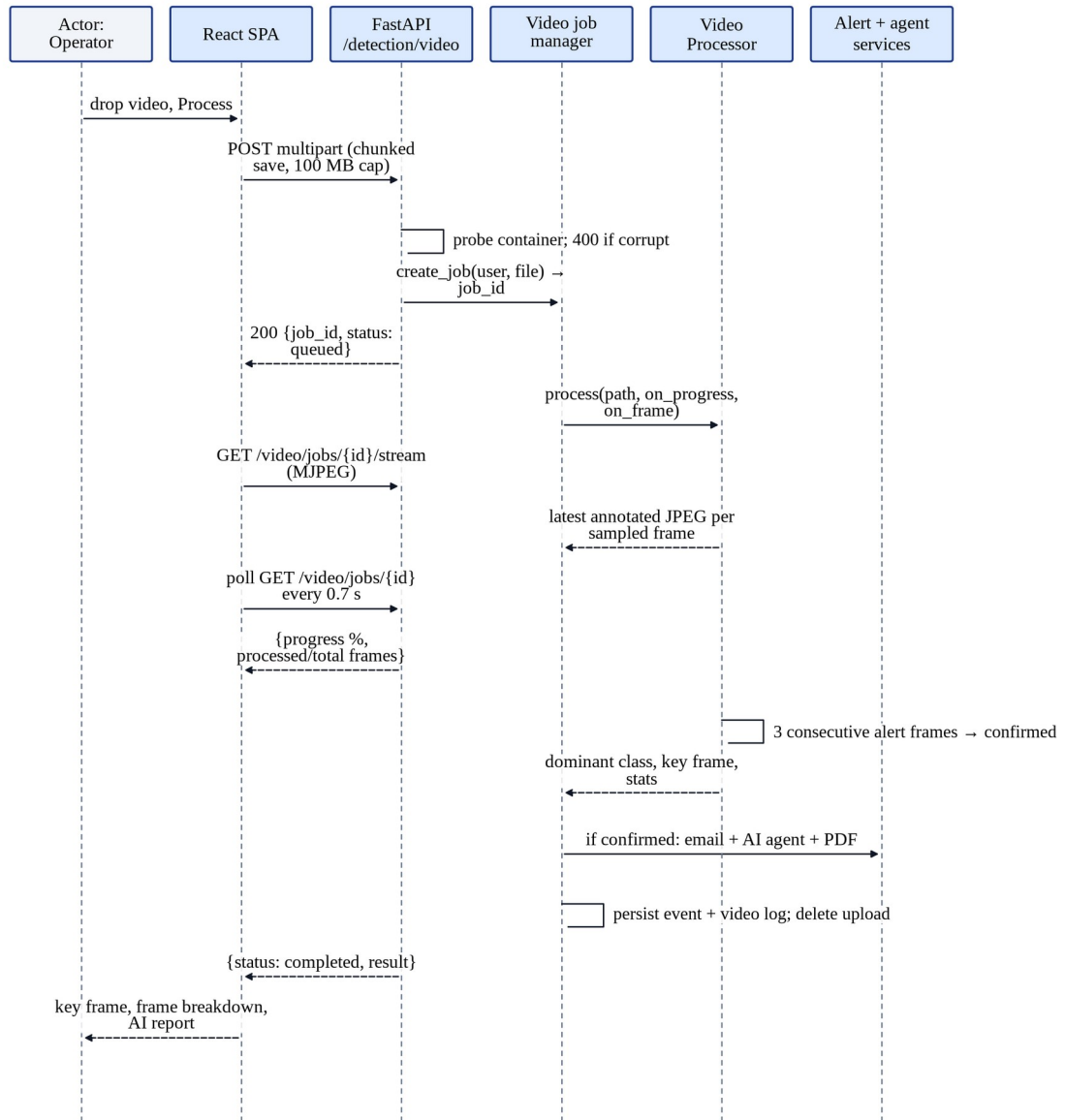


Figure 5.5: Sequence diagram: video detection with live streaming and confirmation

5.5.4 Emergency agent activity

Figure 5.6 is an activity diagram of the emergency agent. The ordering of activities, with a rule-based WhatsApp alert before any LLM call, and the error-fallback branch implement the fail-safe requirement NFR-03.

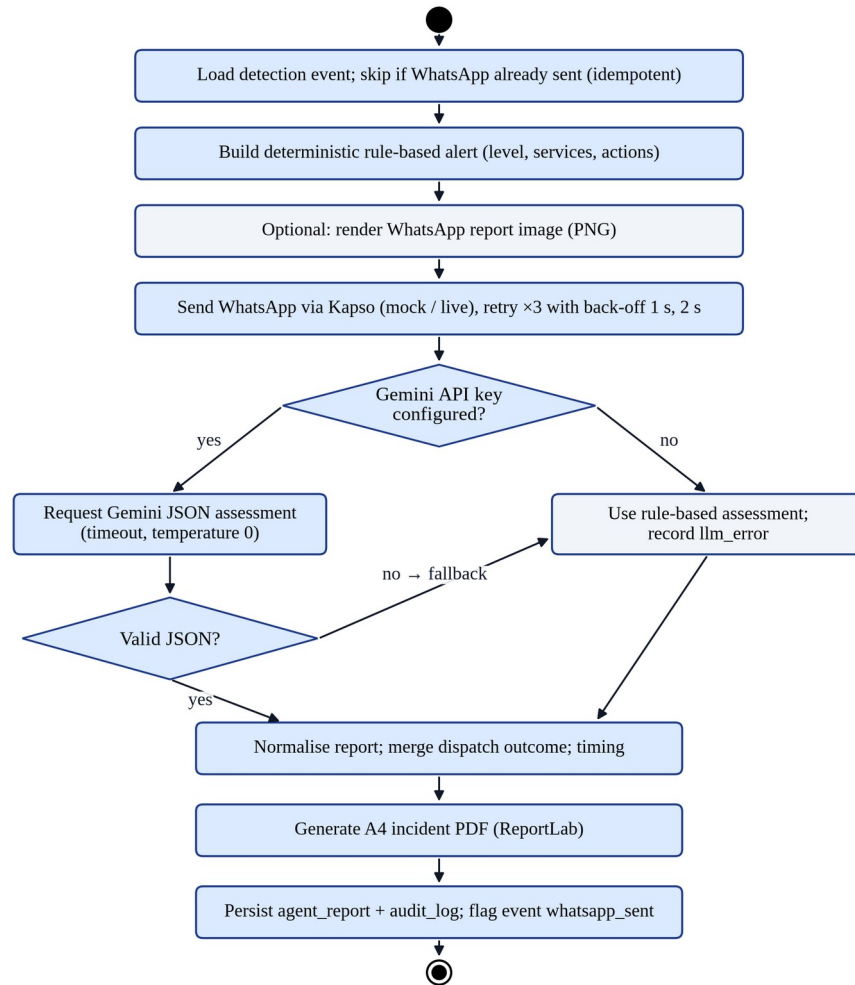


Figure 5.6: Activity diagram of the emergency agent

5.5.5 State machines

Figure 5.7 shows two state machines. An incident is created in the *open* state and can be acknowledged, resolved or marked as a false alarm; every transition is recorded in the audit log. A video job moves from *queued* to *processing* and ends as *completed* or *failed*; finished jobs are removed from memory after one hour.

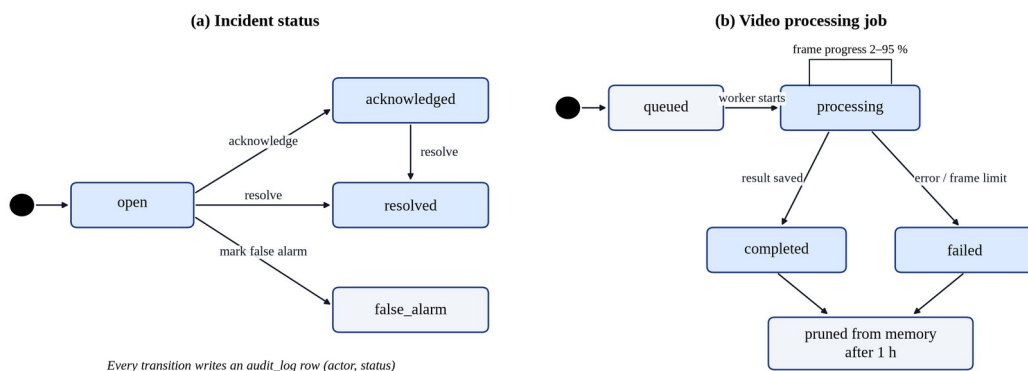


Figure 5.7: State machines: (a) incident status; (b) video-processing job

5.6 API DESIGN

The REST API follows resource-oriented conventions: nouns in paths, HTTP verbs for actions, JSON bodies, standard status codes (200, 201, 204, 400, 401, 403, 404, 413, 422, 503) and pagination through skip and limit parameters. The complete endpoint reference, with methods, paths, access rules and purposes, is given in Appendix A.

5.7 USER INTERFACE DESIGN

The interface was designed as a dark ‘command-centre’ theme that is comfortable for control-room operators who monitor screens for long periods. The design follows four principles: **severity first** (incident colour always communicates priority); **evidence first** (the annotated image is the largest element on analysis screens); **progressive disclosure** (verdict, then metrics, then details and AI assessment); and **consistency** (one set of shared components and colour tokens across all pages). Severity colours are fixed across the interface: red for severe collisions, orange for fire, amber for moderate collisions and slate for no incident, while green marks successful deliveries and violet marks AI assessments. The interactive accent is deliberately sky blue so that it never competes with the red–orange–amber severity scale.

The screens resulting from this design, captured from the running system, are presented in Section 6.2.7.

IMPLEMENTATION AND TESTING

This chapter describes how the design was implemented and how the implementation was verified. It presents the development environment, the implementation of each major module and the user interface, and the security measures in the code. It then describes the testing strategy, the automated test suite, the system test cases that cover every use case, the non-functional tests, and the plan for deploying the system.

6.1 DEVELOPMENT ENVIRONMENT

The hardware and software used during development and evaluation are listed in Table 6.1. Model training was carried out in the cloud on a free Kaggle GPU; all other development, integration and evaluation, including the CPU performance measurements in Chapter 7, were carried out on an ordinary student laptop.

Table 6.1: Development and evaluation environment

Item	Specification
Training platform	Kaggle notebook, NVIDIA Tesla T4 GPU (15 GB), CUDA 12.8, PyTorch 2.10, Ultralytics 8.4.126, Python 3.12
Development machine	12th Gen Intel Core i5-1235U (10 cores / 12 threads), 16 GB RAM, Linux, no discrete GPU used for inference
Back-end runtime	Python 3.12 virtual environment (Python 3.11 in the Docker image), FastAPI 0.111, Uvicorn 0.29
Front-end runtime	Node.js 22, Vite 7 development server; Nginx for production
Database	MySQL 8.0 (development and Docker); SQLite for automated tests
Tools	Git, Visual Studio Code, pytest, ESLint, Docker Compose, Playwright (Chromium) browser automation, headless Firefox for screenshots

6.2 IMPLEMENTATION OF MODULES

6.2.1 Model training

The model was trained in a single, reproducible Kaggle notebook. The notebook first installs Ultralytics, fixes the random seed and records the PyTorch and GPU versions. It then audits the dataset, reporting 14,400, 1,800 and 1,800 image–label pairs for the three splits and no missing or orphaned files, and runs a small PyTorch DataLoader smoke test to confirm that images and labels can be batched. It writes the data configuration file, creates the model from the pretrained yolo12n.pt weights and trains it with the configuration of Table 3.6. After training, the best checkpoint is evaluated on the test split, sample predictions are saved, and a bundle containing the weights,

metrics, training plots and a README is exported for the back end. Training 40 epochs of 450 iterations each took 2.45 hours on the T4 GPU.

6.2.2 Back-end application

The back end is a FastAPI application organised into the layers of Chapter 4: API routers, core services, database modules and Pydantic schemas. At start-up a lifespan handler creates any missing tables, ensures that the storage directories exist, loads the detection model and logs whether the agent and WhatsApp integration are configured. All configuration, including database URL, secret key, thresholds, limits, SMTP, Gemini and Kapso settings, is loaded by a typed settings class from environment variables or an untracked .env file, so the same code runs unchanged in development, testing and Docker. Middleware adds the trusted-host check, CORS policy and security headers, and blocks direct access to incident PDFs.

6.2.3 Inference engine

The detection engine wraps the Ultralytics model. Because the model is shared by concurrent image requests and background video jobs, inference is protected by a lock, and the image endpoint runs inference through a thread pool so that the asynchronous server can continue to serve other requests, including the live video stream, while a frame is being analysed. Detections are converted into plain dictionaries and drawn onto a copy of the frame with class-specific colours and confidence labels, so that the evidence image is self-explanatory.

6.2.4 Video pipeline

Video uploads are written to disk in 1 MB chunks, with the transfer aborted and the partial file deleted as soon as the size limit is exceeded. The container is probed with OpenCV before a job is created, so a corrupt file is rejected with an immediate, meaningful error rather than a failed background job. The job manager stores job state in a lock-protected dictionary and exposes progress and the latest JPEG-encoded annotated frame. The stream endpoint yields this frame as a multipart MJPEG stream roughly every 80 ms, which the browser displays as a live video without any client-side decoding. When the job finishes, the original upload is deleted and the finished job is pruned from memory after an hour.

6.2.5 Alerts and emergency agent

E-mail alerts are built as multipart HTML messages with the annotated image embedded by content-ID so that it displays inside the message. The emergency agent is implemented as a background function that opens its own database session, which avoids sharing a request's session across threads. The WhatsApp adapter supports a *mock* mode, used during development and testing, that returns a simulated message ID

without contacting any external service, and a *Kapso* mode that posts a WhatsApp Cloud API payload to the Kapso gateway and records the provider's message ID. The Gemini integration uses LangChain with temperature 0, a configurable timeout and no automatic retries, and the response parser tolerates Markdown code fences and extracts the JSON object before validation.

6.2.6 Incident reports

PDF reports are generated with ReportLab using A4 pages, a centred title block, a two-column facts table, sections for the executive assessment, hazards, recommended response and notification record, the annotated evidence image and an operational notice, with a footer on every page. An example generated by the running system is shown in Chapter 7 (Figure 7.9).

6.2.7 User interface

The web client was implemented with React 18 and Tailwind CSS following the design of Section 5.7. The following figures were captured from the running system connected to the real model; the incidents shown were produced by analysing images from the held-out test set.

The public landing page (Figure 6.1) introduces the system, shows a real detection produced by the model and reports the model's validation metrics. The login page (Figure 6.2) combines sign-in and registration in a single form with inline validation.

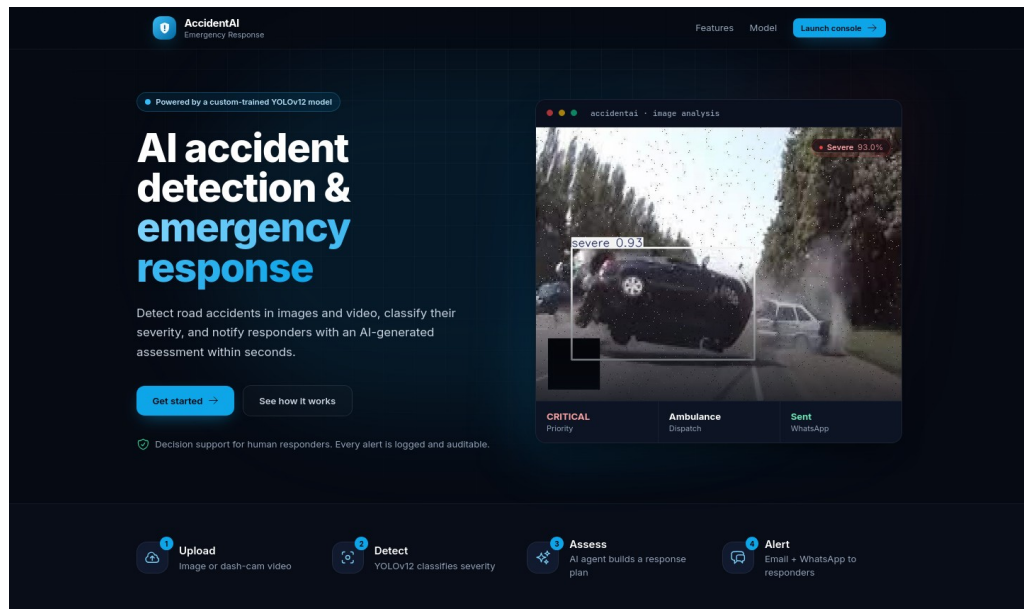


Figure 6.1: Landing page of the web application

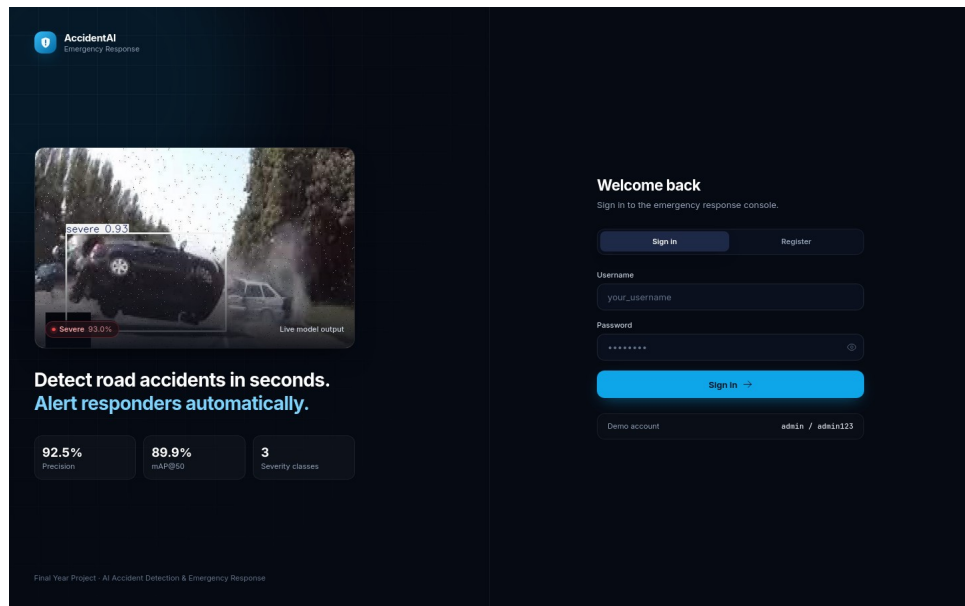


Figure 6.2: Login and registration page

The dashboard (Figure 6.3) greets the operator and presents six key performance indicators, a seven-day detection-activity chart per class, the class distribution and a table of recent incidents with their priority. A status indicator in the top bar shows whether the model is online.

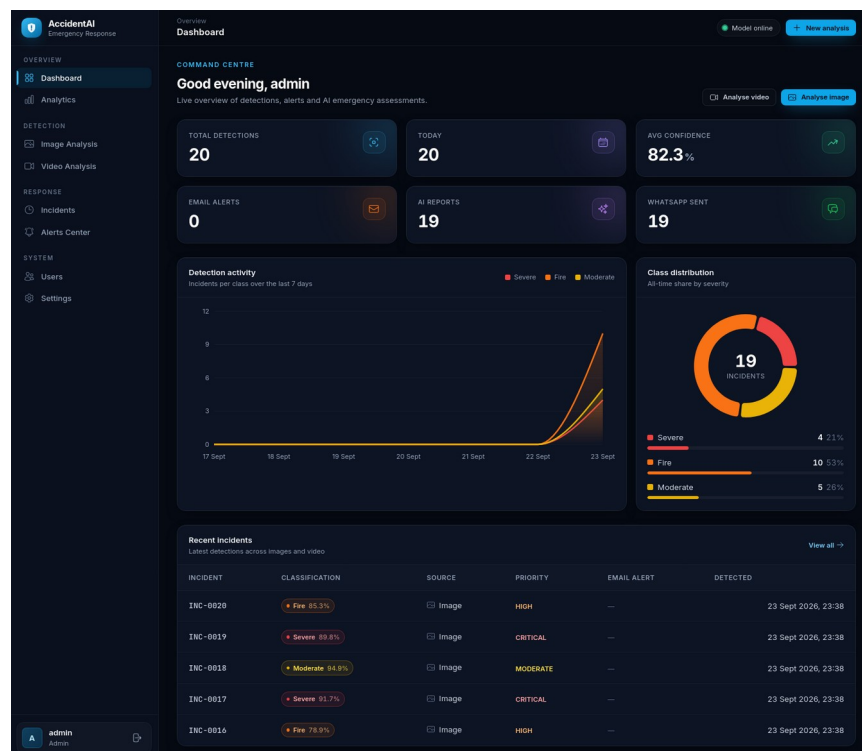


Figure 6.3: Operator dashboard with KPIs, activity trend, class distribution and recent incidents

The image-analysis page accepts an image by drag-and-drop together with an optional location. After detection (Figure 6.4) the page shows the annotated evidence with a

toggle to the original image, a colour-coded verdict card, the number of objects, confidence and inference time, the list of detected objects and, after a few seconds, the AI emergency assessment with a button to download the PDF report.

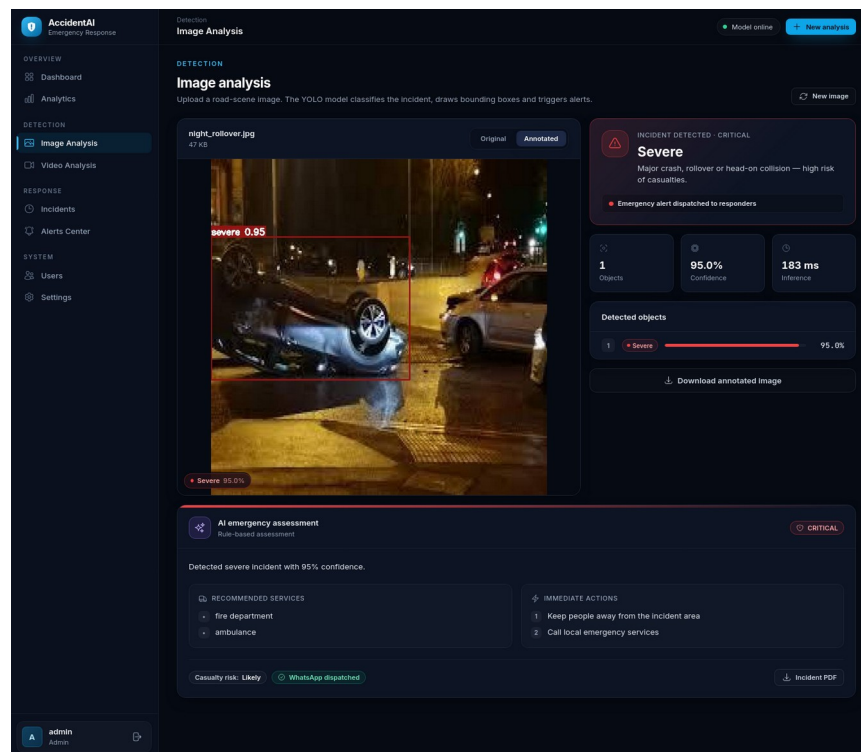


Figure 6.4: Result of analysing a night-time rollover: severe collision detected with 95% confidence, alert dispatched and AI assessment shown

The video-analysis page (Figure 6.5) shows a three-step progress tracker, a live annotated stream during processing and, when processing is complete, the confirmed incident, total frames, average confidence, a per-class breakdown of the sampled frames and the AI assessment.

The incidents page (Figure 6.6) presents every detection as a card with its evidence image, severity, identifier, time, location, status and notification channels, with filters by class and status and one-click acknowledgement. An alerts center shows the delivery log of each notification channel (e-mail, WhatsApp and call audit), and the analytics page (Figure 6.7) offers selectable time ranges for trends, confidence distribution, class distribution and alert delivery.

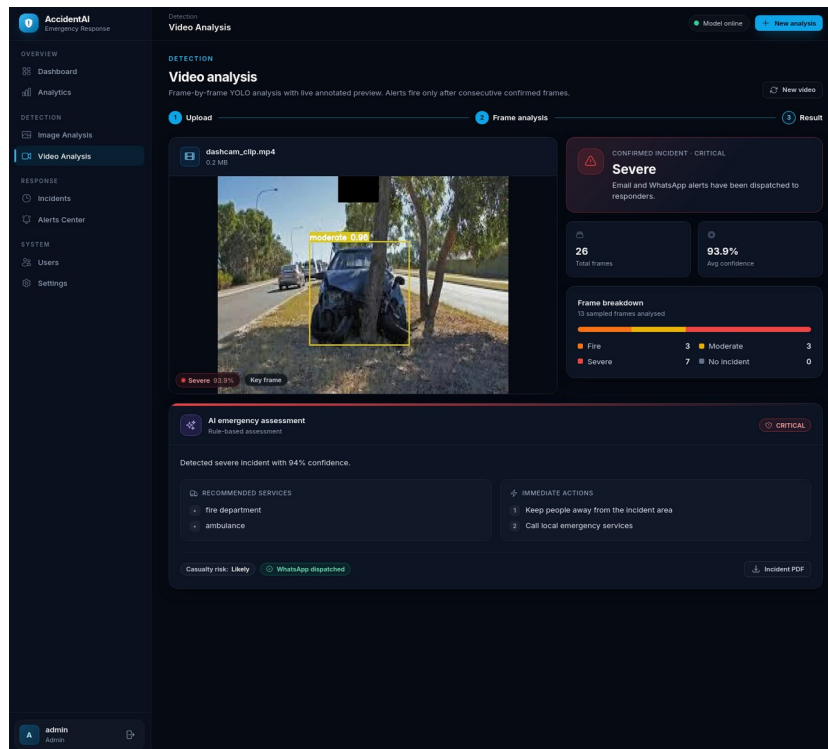


Figure 6.5: Video-analysis result for a 26-frame dash-cam clip: incident confirmed as severe after temporal confirmation

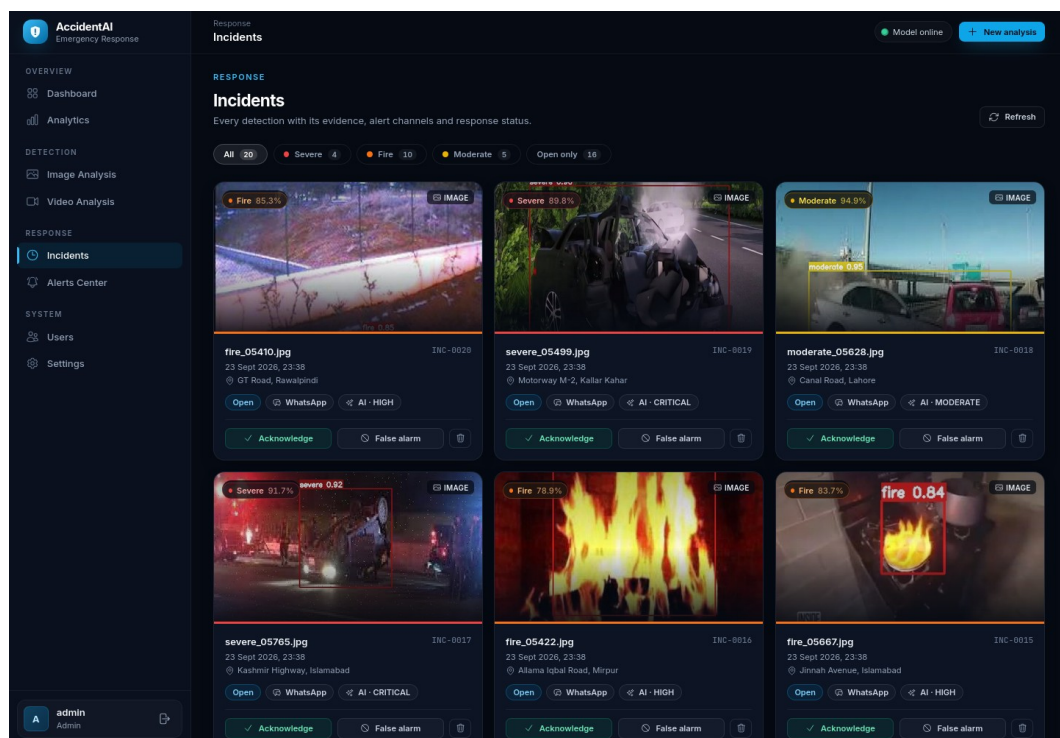


Figure 6.6: Incidents page with evidence cards, filters and triage actions

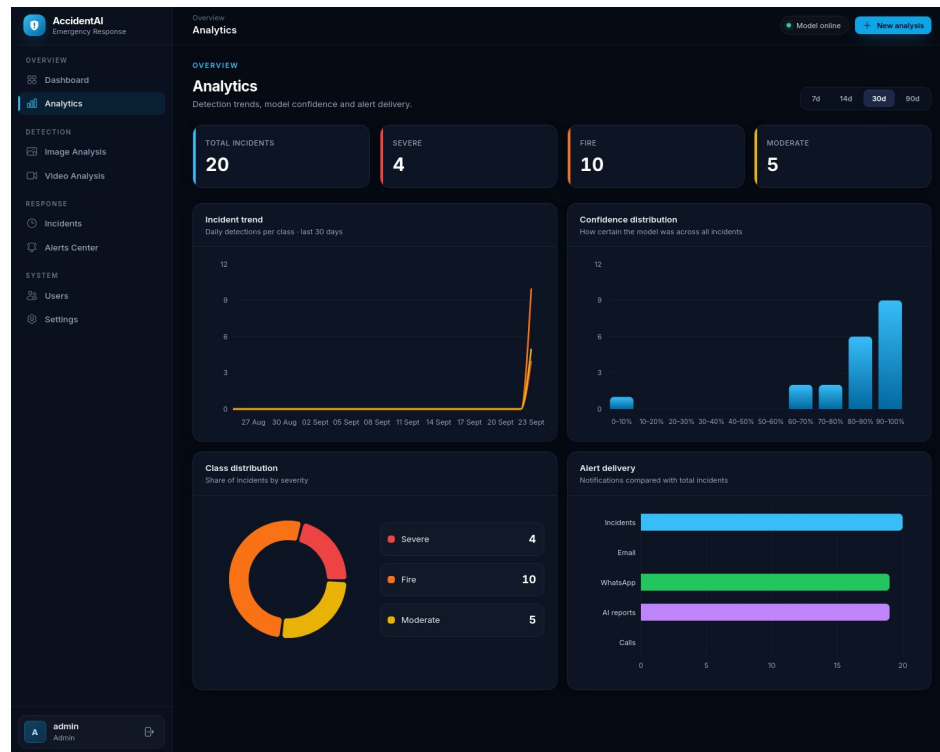


Figure 6.7: Analytics page with trend, confidence, class and alert-delivery charts

Administrators manage accounts on the users page (Figure 6.8); the settings page (Figure 6.9) shows the user's profile and the live configuration of the model, the AI agent and the notification channels. The interface is fully responsive; Figure 6.10 shows the dashboard on a 390-pixel-wide mobile screen, where the sidebar collapses into a menu.

Figure 6.8 displays the User management page of the AccidentAI Emergency Response system. The page features a sidebar with navigation options: Overview (Dashboard, Analytics), Detection (Image Analysis, Video Analysis), Response (Incidents, Alerts Center), and System (Users, Settings). The main content area is titled 'User management' and includes a 'TOTAL USERS' summary (4), an 'ADMINS' summary (1), and an 'ACTIVE' summary (3). It also features a table of all users with columns: USER, ROLE, STATUS, JOINED, and ACTIONS.

USER	ROLE	STATUS	JOINED	ACTIONS
admin YOU admin@must.edu.pk	Admin	Active	23 Sept 2026, 23:37	Edit
operator1 operator1@must.edu.pk	Operator	Active	23 Sept 2026, 23:37	Edit
viewer1 viewer1@must.edu.pk	Viewer	Active	23 Sept 2026, 23:37	Edit
trainee trainee@must.edu.pk	Viewer	Inactive	23 Sept 2026, 23:37	Edit

Figure 6.8: User-management page (administrators only)

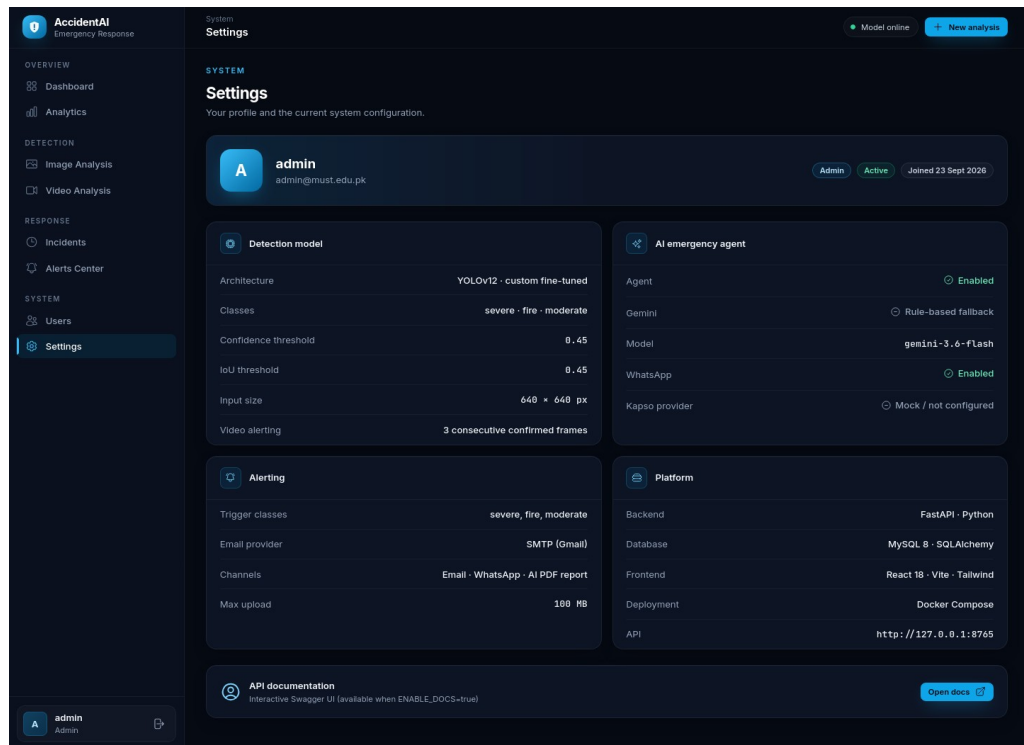


Figure 6.9: Settings page showing profile and live system configuration



Figure 6.10: Responsive dashboard on a mobile-sized screen (390 px)

6.3 SECURITY IMPLEMENTATION

The security architecture of Section 4.6 was implemented and hardened during the final increment. Particular attention was paid to issues discovered during a code review: generated incident PDFs, whose names contain sequential incident numbers, were

originally reachable through the public static-file route; they are now blocked by middleware, with path normalisation to prevent bypasses such as double slashes or ‘..’ segments, and are served only through the ownership-checked PDF endpoint. WhatsApp report images received unguessable random names. Administrators can no longer deactivate or demote their own account, which prevents accidental lock-out. Image uploads are read with a size cap before decoding, and inference was moved off the event loop so that one large image cannot stall the server. Each of these fixes is protected by an automated regression test.

6.4 TESTING STRATEGY

Testing followed the levels shown in Table 6.2. Automated tests run against an isolated SQLite database that is recreated for every test, with the model path deliberately pointed at a missing file so that API tests do not depend on the model; model behaviour is instead evaluated separately with the real weights (Chapter 7). Black-box techniques such as equivalence partitioning and boundary-value analysis were used to choose inputs, for example valid versus invalid file types, passwords of 72 versus 73 bytes, and three versus fewer consecutive positive frames.

Table 6.2: Testing levels and techniques

Level	Scope	Technique / tool
Unit	Security utilities, class contract, WhatsApp formatter, report generator, agent fallback	pytest with mocks
Integration	API endpoints with database, authentication and background tasks	pytest + FastAPI TestClient + SQLite
System	Complete use cases through the API and the user interface with the real model	Scripted API scenarios; Playwright browser automation
Security / regression	Access control, isolation, headers, upload validation, static-file exposure	pytest regression suite
Performance	CPU inference latency, image-level accuracy	Benchmark script over 300 test images
Usability / compatibility	Layout from 390 px to 1440 px; Firefox and Chromium-based browsers	Manual inspection
Static analysis	Front-end code quality	ESLint with zero-warning policy; production build

6.5 AUTOMATED TEST RESULTS

The back-end test suite contains 36 automated tests grouped into seven modules. All 36 tests pass, and the complete suite runs in about 38 seconds, which makes it practical to run before every commit. Figure 6.11 shows the number of tests per module and Table 6.3 lists what each module verifies. The front end passes ESLint with zero warnings and builds successfully for production.

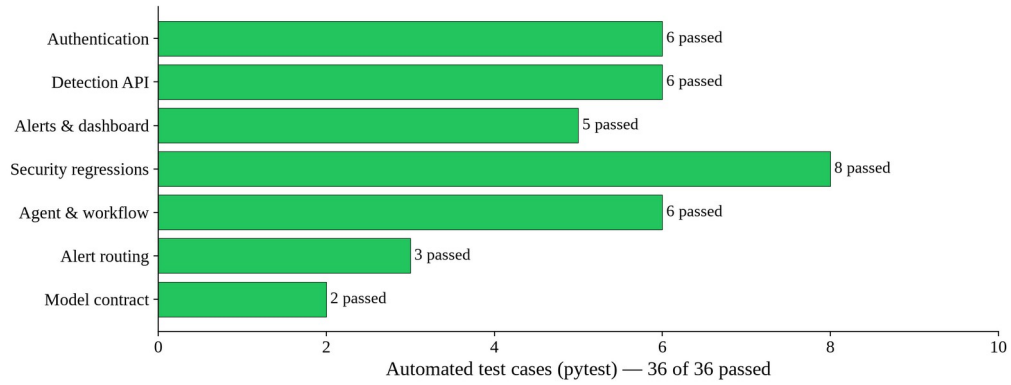


Figure 6.11: Automated back-end tests per module (36 of 36 passed)

Table 6.3: Automated test modules and coverage

Module	Tests	What is verified
Authentication	6	Health endpoint; registration; login success and wrong password; profile with and without a token
Detection API	6	Authenticated image detection; unauthenticated rejection; invalid file type; corrupt image; history with and without authentication
Alerts and dashboard	5	Alert and call logs for authenticated users; unauthenticated rejection; test e-mail forbidden for non-administrators; dashboard statistics
Security regressions	8	Security headers; API docs disabled by default; registration cannot create an admin; password byte limit; per-user history isolation; corrupt video rejected; incident PDFs not publicly served; admin cannot deactivate or demote self
Agent and workflow	6	Mock WhatsApp returns a message ID; Kapso payload format; professional WhatsApp message; A4 PDF generation; agent fallback without Gemini; agent writes report and mock WhatsApp
Alert routing	3	E-mail alert runs for the moderate class; no external call provider is used; mock call logging is the default
Model contract	2	Class order is fire, moderate, severe; prediction is safe when weights are missing

6.6 SYSTEM TEST CASES

System test cases were written for every use case of Figure 3.2, including both main and alternative flows. They are listed in Table 6.4. Test cases marked ‘A’ are also covered by the automated suite; those marked ‘M’ were executed against the running system with the trained model, either as scripted API scenarios (26 checks) or through the real user interface using Playwright browser automation (13 checks, including login, analysis, view toggling, filtering, acknowledgement, analytics ranges, role-based navigation and the mobile layout). All 40 test cases passed.

Table 6.4: System test cases and results

TC	UC	Test scenario and input	Expected result	Type	Result
TC-01	UC-01	Register with new username, valid e-mail, 8-char password	201 Created; role is viewer	A	Pass
TC-02	UC-01	Register with an existing username	400 'Username already taken'	M	Pass
TC-03	UC-01	Register requesting role 'admin'	Account created as viewer only	A	Pass
TC-04	UC-01	Register with a 73-byte password	422 validation error	A	Pass
TC-05	UC-02	Login with correct credentials	200 with JWT and user profile	A	Pass
TC-06	UC-02	Login with a wrong password	401 'Invalid username or password'	A	Pass
TC-07	UC-02	Login to a deactivated account	403 'Account deactivated'	M	Pass
TC-08	UC-02	Call a protected endpoint without / with expired token	401; client redirects to login	A	Pass
TC-09	UC-03	Analyse a severe-crash test image with location	Severe detected (0.95), annotated image, location stored	M	Pass
TC-10	UC-03	Analyse a fire test image	Fire detected with box; alert triggered	M	Pass
TC-11	UC-03	Analyse an image containing no accident	'No incident'; no alert sent	M	Pass
TC-12	UC-03	Upload a .txt file, or an image whose MIME type does not match its extension	400 unsupported type / MIME mismatch	A	Pass
TC-13	UC-03	Upload a corrupt image	400 'Cannot decode image'	A	Pass
TC-14	UC-03	Toggle original / annotated view; download annotated image	Views switch; file downloads	M	Pass
TC-15	UC-04	Analyse a 26-frame clip containing a crash	Live stream shown; incident confirmed; 13 frames sampled	M	Pass
TC-16	UC-04	Upload a corrupt video	400 'Cannot open video'; no job created	A	Pass
TC-17	UC-04	Poll another user's job ID	404 'job not found'	M	Pass
TC-18	UC-04	Video processed successfully	Upload deleted; event and video log stored	M	Pass
TC-19	UC-05	Viewer lists incidents	Only own incidents returned	A	Pass
TC-20	UC-05	Filter incidents by class and 'open only'	Cards filtered; counts correct	M	Pass
TC-21	UC-06	Acknowledge an open incident	Status 'acknowledged'; audit row written	M	Pass
TC-22	UC-06	Mark an incident as false alarm	Status 'false_alarm'	M	Pass
TC-23	UC-06	Send an invalid status value	422 'Invalid incident status'	M	Pass
TC-24	UC-07	Open dashboard and analytics	KPIs, charts and tables match stored data	A/M	Pass
TC-25	UC-07	Change analytics range 7/14/30/90 days	Charts reload for selected range	M	Pass
TC-26	UC-08	Download PDF for own incident	A4 PDF with evidence downloaded	A/M	Pass
TC-27	UC-08	Open /static/reports/incident-1.pdf directly (incl. '/' and '..' variants)	404 for all variants	A	Pass
TC-28	UC-10	Fire / severe incident with WhatsApp in mock mode	Assessment HIGH / CRITICAL; WhatsApp dispatched	A/M	Pass

TC	UC	Test scenario and input	Expected result	Type	Result
TC-29	UC-10	Gemini key not configured	Rule-based assessment stored; llm_error recorded	A	Pass
TC-30	UC-10	Kapso mode: payload sent to gateway (mocked HTTP)	Correct WhatsApp Cloud API payload; message ID stored	A	Pass
TC-31	UC-10	E-mail alert for a moderate incident (mocked SMTP)	E-mail sent and logged	A	Pass
TC-32	UC-10	SMTP not configured	Alert skipped without error	M	Pass
TC-33	UC-11	Admin changes a user's role and active flag	User updated	M	Pass
TC-34	UC-11	Admin tries to deactivate or demote self	400 refused	A	Pass
TC-35	UC-11	Viewer opens /users	Redirected / 403 Admin access required	M	Pass
TC-36	UC-12	Admin deletes an incident	204; child records removed	M	Pass
TC-37	UC-13	Non-admin calls test e-mail	403 forbidden	A	Pass
TC-38	UC-13	Admin sends test WhatsApp (mock)	Success toast; simulated message ID	M	Pass
TC-39	—	GET /health with weights installed; predict without weights	model_loaded true; safe 'no_detection' without weights	A/M	Pass
TC-40	—	Security headers on every response	nosniff, frame and referrer policies present	A	Pass

6.7 NON-FUNCTIONAL TESTING

Performance. Inference latency was measured on the development laptop over 300 randomly selected test images: the mean was 109 ms and the 95th percentile 126 ms per image, satisfying NFR-02 (≤ 200 ms). Details are given in Section 7.4. During video processing, other API requests and the live stream remained responsive because inference runs outside the event loop.

Security. Beyond the automated regression tests, the code was reviewed manually for broken access control, sensitive-data exposure, injection and unrestricted uploads. The review led to the fixes described in Section 6.3; remaining limitations are documented honestly in Section 8.5.

Usability and compatibility. Every page was inspected at desktop (1440 px) and mobile (390 px) widths (Figure 6.10). Severity colours, icons and text labels are used together so that information is never conveyed by colour alone, and keyboard focus is clearly visible.

Reliability. Failure scenarios were exercised deliberately: a missing model file, a missing Gemini key, a disabled SMTP server and corrupt media. In each case the system continued to operate, recorded the reason and presented a meaningful message, satisfying NFR-03.

6.8 DEPLOYMENT PLAN

The system is deployed with Docker Compose using the architecture of Figure 4.4. The deployment proceeds in the following steps:

1. **Prepare the host.** Install Docker Engine with Compose on a Linux server or workstation with at least 4 CPU cores and 8 GB RAM; open ports 80 (web) and 8000 (API) on the firewall.
2. **Install the model.** Copy the trained best.pt into backend/ml_model/.
3. **Configure secrets.** Create the root .env with a long random SECRET_KEY and database passwords, and backend/.env with optional SMTP, Gemini and Kapso settings. Keep both files out of version control.
4. **Build and start.** Run docker compose up --build. MySQL initialises the schema on first start; the back end waits for the database health check.
5. **Verify.** Open /health and confirm status 'ok' and model_loaded true; log in, analyse a known test image and confirm that the incident, alert log and PDF are created.
6. **Operate.** Change the default administrator password, create operator accounts, and send test e-mail and WhatsApp messages from the Alerts Center.
7. **Back up and update.** Back up the mysql_data and media_data volumes regularly; to update the model, replace best.pt and restart the back end; to roll back, restore the previous image tag and model file.

Before a live pilot, the pre-deployment checklist also requires HTTPS end to end, rotation of all development credentials, a retention policy for generated media, and review of recipient consent for WhatsApp alerts.

RESULTS AND DISCUSSION

This chapter presents and discusses the results obtained with the implemented system. It first analyses the training process, then reports the accuracy of the model on the validation and held-out test sets overall and per class, supported by precision–recall and F1 curves and a confusion matrix. It then shows qualitative predictions, reports the runtime performance measured on a GPU and on an ordinary laptop CPU, and evaluates the complete system end to end. The chapter closes with a discussion of the findings, their comparison with related work, and threats to validity.

7.1 TRAINING CONVERGENCE

Figure 7.1 shows the three components of the training and validation loss over the 40 epochs. All losses decrease steadily. The validation losses fall below the training losses for most of the run, which is expected because training images are heavily augmented (mosaic, colour jitter, random erasing) while validation images are not, and there is no sign of the validation loss rising again, which would indicate over-fitting. When mosaic augmentation is disabled for the last ten epochs (dotted line), the training losses drop sharply as the model adapts to natural, unmodified images.

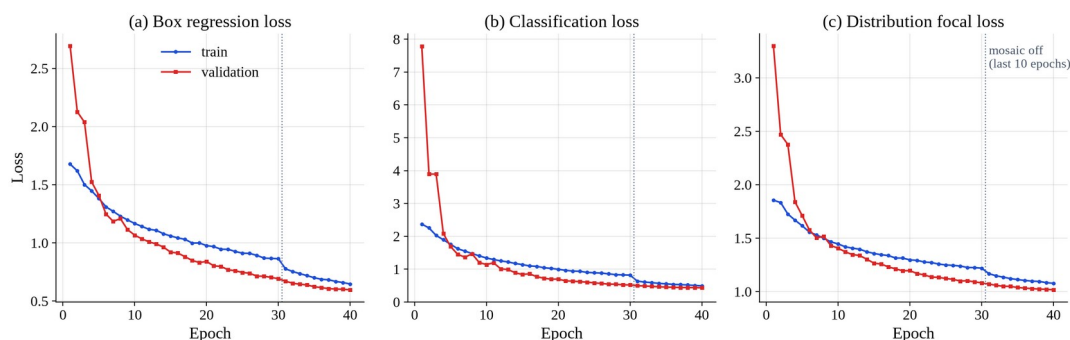


Figure 7.1: Training and validation losses per epoch: (a) box regression, (b) classification, (c) distribution focal loss

Figure 7.2 shows the validation metrics per epoch. Starting from a mAP@0.5 of only 0.06 after the first epoch, when the new three-class head has not yet been trained, the model exceeds 0.72 by epoch 10, 0.86 by epoch 20 and 0.90 by epoch 30, and reaches its best values at the final epoch: precision 0.951, recall 0.875, mAP@0.5 0.913 and mAP@0.5:0.95 0.800. The curves are still rising slightly at the end, suggesting that a longer schedule could yield a small further improvement.

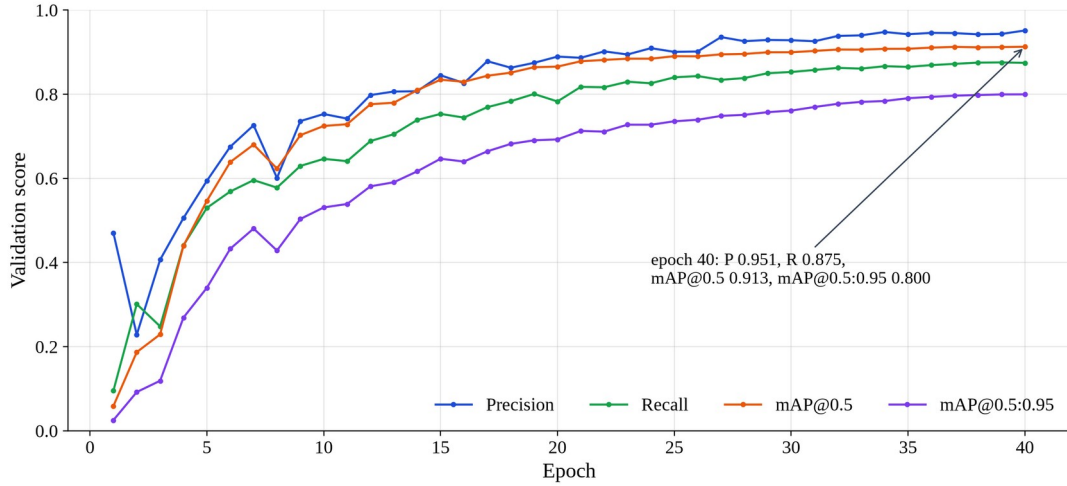


Figure 7.2: Validation precision, recall, mAP@0.5 and mAP@0.5:0.95 per epoch

7.2 QUANTITATIVE DETECTION RESULTS

The best checkpoint was evaluated on the validation set and, once, on the held-out test set that was never used for training or model selection. Table 7.1 reports the overall results and Table 7.2 the per-class results. On the test set the model achieves a precision of 92.5%, a recall of 86.8%, a mAP@0.5 of 89.9% and a mAP@0.5:0.95 of 79.0%, exceeding the target of 85% mAP@0.5 set in objective 2 and NFR-01. The small gap between validation and test results (1.4 percentage points in mAP@0.5) indicates that the model generalises well to unseen images from the same distribution.

Table 7.1: Overall detection performance of YOLOv12n

Split	Images	Instances	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Validation	1,800	2,475	0.950	0.875	0.913	0.800
Test (held out)	1,800	2,443	0.925	0.868	0.899	0.790

Table 7.2: Per-class results on the validation and test sets

Class (set)	Instances	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Fire (val)	1,247	0.886	0.692	0.783	0.540
Fire (test)	1,223	0.830	0.643	0.729	0.495
Moderate (val)	607	0.995	0.965	0.974	0.946
Moderate (test)	612	0.976	0.969	0.973	0.943
Severe (val)	621	0.971	0.968	0.982	0.914
Severe (test)	608	0.969	0.990	0.994	0.932

Figure 7.3 visualises the per-class results. The two collision classes are detected almost perfectly: severe collisions reach a test mAP@0.5 of 99.4% with a recall of 99.0%, and moderate collisions 97.3% with a recall of 96.9%. Their high mAP@0.5:0.95 values

(0.93–0.94) show that the boxes are also placed very precisely. The fire class is clearly the most difficult, with a test mAP@0.5 of 72.9% and, in particular, a lower recall of 64.3%.

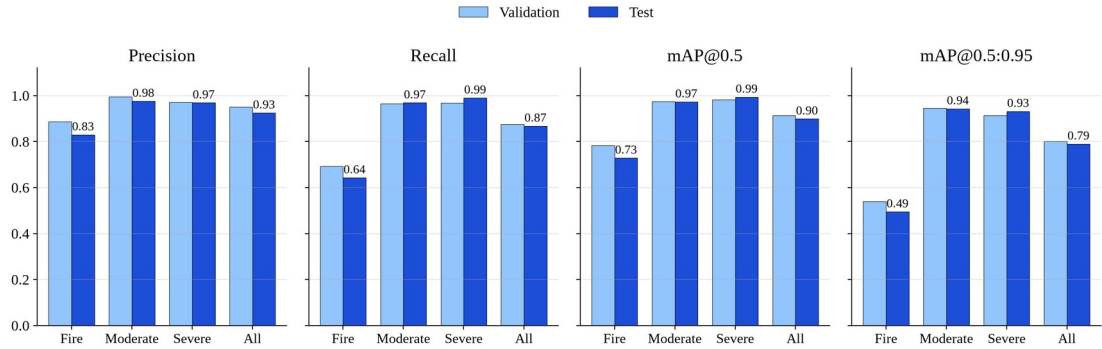


Figure 7.3: Per-class precision, recall, mAP@0.5 and mAP@0.5:0.95 on the validation and test sets

Figure 7.4(left) shows the precision–recall curve of each class and Figure 7.4(right) the F1 score as a function of the confidence threshold. The collision classes maintain precision close to 1.0 up to a recall above 0.95, while the fire curve drops earlier. The F1 score over all classes peaks at 0.91 at a confidence threshold of 0.365 and remains at about 0.90 at the threshold of 0.45 used by the deployed system. The slightly higher operating threshold was chosen deliberately: it gives up very little F1 while reducing false alarms, which are costly in an emergency setting.

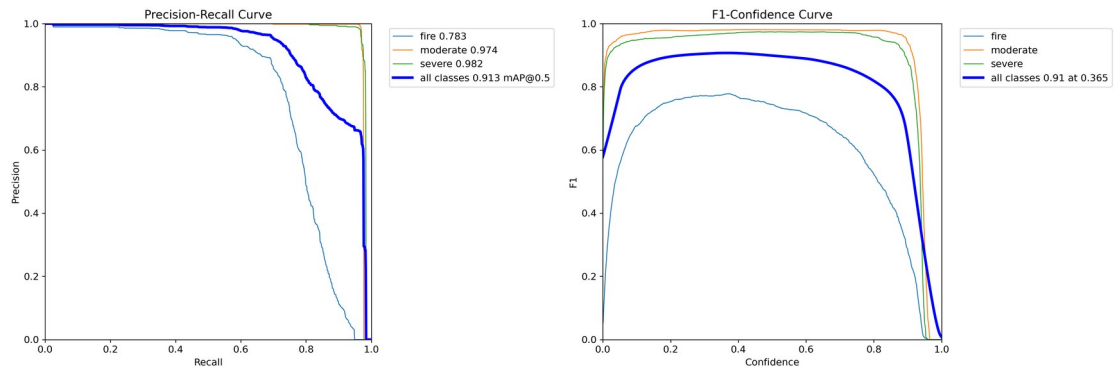


Figure 7.4: Precision–recall curves (left) and F1 score versus confidence threshold (right) per class on the validation set

7.3 ERROR ANALYSIS

The normalised confusion matrix in Figure 7.5 explains the per-class results. Its most important property is that there is **no confusion between the three incident classes**: no fire object is predicted as a collision, and moderate and severe collisions are never mistaken for one another. For a severity-aware system this is the most valuable outcome, because a wrongly assigned severity would send the wrong response. All errors instead involve the *background*: 26% of fire objects are missed (predicted as background) compared with only 3% of moderate and severe collisions, and among the

false detections produced on background regions, most are labelled as fire (0.86 of the background column), with a smaller share labelled severe (0.12).

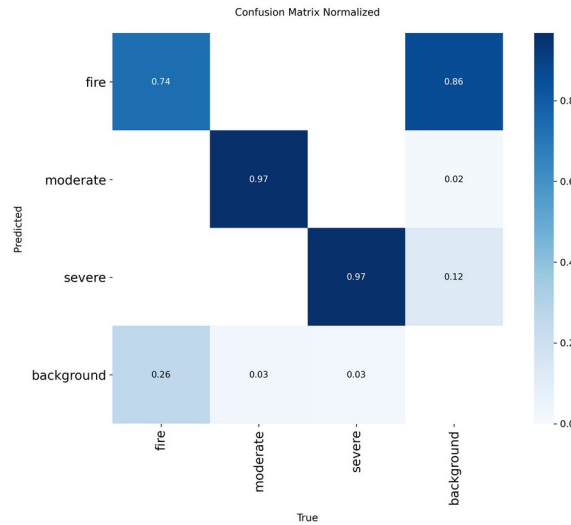


Figure 7.5: Normalised confusion matrix on the validation set (columns: true class; rows: predicted class)

The fire errors have identifiable causes. First, fire images contain on average about two annotated flame regions per image, many of them small, partially occluded by smoke or overlapping, so recall is counted over many hard, small instances. Second, flames have no fixed shape or boundary, so annotators and the model may reasonably disagree about whether a large blaze is one box or several; such disagreements are counted as both missed objects and false positives, which lowers precision and recall even when the image is correctly recognised as a fire. Third, bright lights, sunsets and reflections resemble flames. The image-level evaluation in Section 7.5 supports this interpretation: when only the top prediction of each image is considered, 81.7% of fire images are correctly classified, which is considerably higher than the object-level recall of 64.3%.

7.4 QUALITATIVE RESULTS

Figure 7.6 compares ground-truth labels with predictions on a validation batch. The model localises overturned and heavily damaged vehicles and flames under a wide range of conditions, including night scenes, dash-cam views, overhead CCTV and cluttered backgrounds. Figure 7.7 shows further predictions on test images produced with the deployed confidence threshold of 0.45.



Figure 7.6: Model predictions with confidence scores on a validation batch

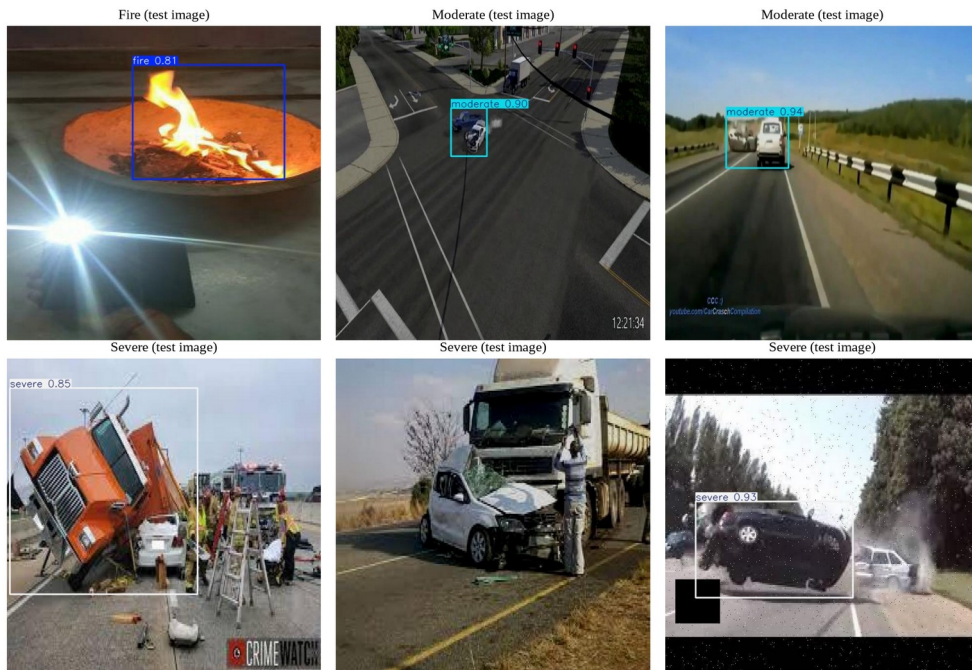


Figure 7.7: Sample predictions on held-out test images: fire, moderate and severe incidents

7.5 RUNTIME PERFORMANCE

Table 7.3 summarises the runtime characteristics of the model. On the Tesla T4 GPU, inference takes about 4 to 6 ms per image. More relevant for low-cost deployment is performance on an ordinary CPU. To measure it, 300 images were drawn at random (seed 42) from the test set and processed one at a time by the system's own detection engine on the development laptop, an Intel Core i5-1235U without GPU acceleration, after a warm-up of five images.

Table 7.3: Model size and inference speed

Measure	GPU (Tesla T4)	CPU (Core i5-1235U)
Parameters / GFLOPs / weight file	2.56 M / 7.3 / 5.5 MB	same
Pre-process + inference + post-process per image	0.9 + 5.8 + 0.8 ms (test run)	—
Mean end-to-end engine latency (n = 300)	—	109.1 ms
Median / 95th percentile	—	111.0 ms / 125.6 ms
Minimum / maximum	—	69.5 ms / 174.6 ms
Throughput	≈ 130 images/s	≈ 9 images/s

The mean CPU latency of 109 ms (median 111 ms, 95th percentile 126 ms) comfortably satisfies NFR-02 (≤ 200 ms), and the latency distribution in Figure 7.8(a) is narrow, which means that response times are predictable. At about nine frames per second, and with a stride of two, the system can analyse a 30 frames-per-second video at roughly 60% of real-time speed on a laptop CPU; on a GPU the model is far faster than real time.

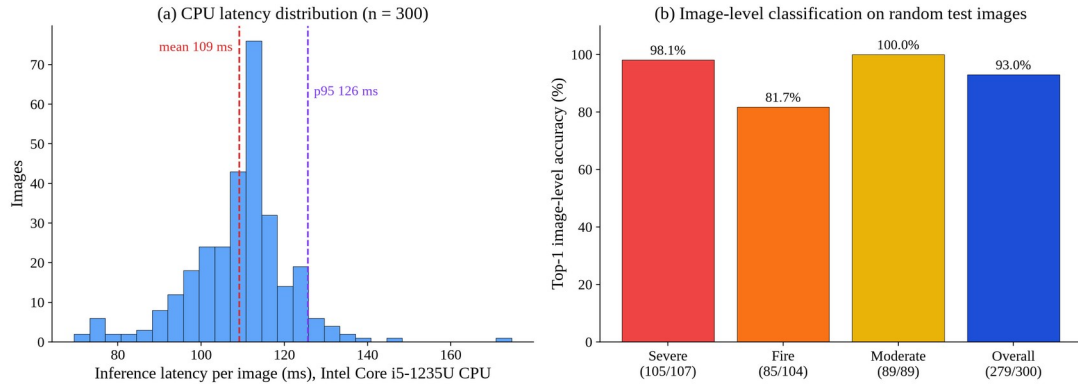


Figure 7.8: (a) CPU inference latency over 300 test images; (b) image-level classification accuracy on the same images

Figure 7.8(b) shows the image-level top-1 accuracy on the same 300 images: 279 images (93.0%) were assigned the correct class by their highest-confidence detection, including 105 of 107 severe images (98.1%), all 89 moderate images (100%) and 85 of 104 fire images (81.7%). This measure is closer to how the system is actually used, where the question is whether an uploaded image is correctly recognised as a severe crash, a moderate crash or a fire.

7.6 END-TO-END SYSTEM EVALUATION

The complete pipeline was evaluated by using the running system exactly as an operator would. For a night-time image of an overturned car from the test set (Figure 6.4), the system detected a severe collision with 95.0% confidence in 183 ms of inference time, returned the annotated evidence, recorded the location, sent the WhatsApp alert (mock gateway), and displayed the CRITICAL assessment recommending the fire department

and an ambulance within a few seconds. The corresponding A4 incident report generated by the system is shown in Figure 7.9.

AI Accident Detection System	
FORMAL INCIDENT ASSESSMENT REPORT	
Generated 2026-09-23 23:39 UTC · Incident reference INC-000021	
Incident reference	INC-000021
Detection date and time	2026-09-23 23:39:19
Reported location	GT Road, near Gujar Khan
Incident classification	SEVERE
Assessment level	CRITICAL
Model confidence	95.0%
Incident status	open

Executive assessment Detected severe incident with 95% confidence. Casualty risk: likely Hazards observed • severe Recommended emergency response Services: • fire department • ambulance Immediate actions: • Keep people away from the incident area • Call local emergency services Notification record Simulation only — no WhatsApp recipient was contacted.

Figure 7.9: First page of an A4 incident report generated automatically by the system

For video, a 26-frame dash-cam clip containing moderate, severe and fire segments was processed (Figure 6.5). With a stride of two, 13 frames were analysed (3 fire, 3 moderate and 7 severe); the three-frame confirmation rule confirmed a severe incident, the dominant class was severe with an average confidence of 93.9%, and alerts and the assessment followed automatically. The original upload was deleted after processing, as designed. In a separate test with a clip containing only a severe crash, all five sampled frames were classified as severe and the incident was confirmed. Across all executed system test cases (Section 6.6), every notification attempt, status change and assessment was recorded in the database and visible in the dashboard, the alerts center and the incident list.

Table 7.4 maps the results to the objectives stated in Section 1.4.

Table 7.4: Achievement of project objectives

Objective	Evidence	Status
1. Balanced dataset; fine-tuned YOLOv12	18,000 images, 4,800 per class for training; YOLOv12n trained 40 epochs	Met
2. Test mAP@0.5 $\geq$ 85%; CPU-friendly speed	89.9% mAP@0.5; 109 ms mean CPU latency	Met

Objective	Evidence	Status
3. Secure REST back end with isolation	26 endpoints; JWT, roles, ownership filters; security regression tests	Met
4. Live video with temporal confirmation	MJPEG stream; 3-frame confirmation verified (TC-15)	Met
5. Structured assessment with fallback	Gemini or rule-based assessment always stored (TC-28, TC-29)	Met
6. E-mail, WhatsApp, PDF	E-mail (tested with mocked SMTP), WhatsApp mock and Kapso payload, A4 PDF	Met*
7. Responsive dashboard	10 pages, 390–1440 px layouts	Met
8. Testing and deployment	36/36 automated, 40/40 system test cases; Docker Compose	Met

* E-mail and live WhatsApp delivery depend on external credentials. They were verified with mocked providers and the WhatsApp mock gateway; delivery to real recipients must be confirmed during a pilot with configured accounts.

7.7 DISCUSSION

The results show that a very small detector can deliver high accuracy for this task. With 2.56 million parameters, YOLOv12n achieves 89.9% mAP@0.5 on a held-out test set and near-perfect detection of collisions, while running at about nine images per second on a laptop CPU. This supports the design decision to prefer the nano variant: a larger model would have improved accuracy somewhat but would have made CPU-based video analysis impractical, and it would not solve the fire-class problem, which is caused mainly by the ambiguity of flame annotations rather than by model capacity.

Direct numerical comparison with the related work of Chapter 2 is not meaningful, because those studies use different datasets, camera types, class definitions and metrics, and most of them solve a binary accident/no-accident problem. Qualitatively, however, the proposed system differs in three important respects: it distinguishes three severity-relevant classes and never confused them in the evaluation; it addresses false alarms in continuous video with an explicit confirmation rule; and it connects detection to notification, documentation and human review, which the reviewed vision-based studies do not address.

The end-to-end evaluation also revealed a design subtlety. In the mixed video clip, the key frame shown to the operator was the frame with the highest single detection confidence, which was a moderate-collision frame (0.96), while the confirmed and dominant class was severe. Both facts are correct, but an operator could find the combination confusing. A simple improvement, selecting the key frame from the dominant class, is listed as future work.

Finally, the fail-safe design of the emergency agent proved valuable in practice: during all tests without a Gemini key, the system produced consistent, rule-based assessments and recorded why the LLM was not used, so the absence of an optional service never degraded the core function of detecting and reporting incidents.

7.8 THREATS TO VALIDITY

- **Dataset distribution.** Training, validation and test images come from the same merged collection, so the test results measure generalisation within that distribution. Performance on footage from a specific camera network in AJK may differ and should be measured on a locally collected sample before deployment.
- **Possible near-duplicates.** Public accident images are often re-published and augmented; if near-duplicate images exist across splits, the test results may be optimistic. The small validation–test gap is reassuring but does not rule this out.
- **Still images for video.** The model was trained on individual images; temporal context in video is used only through the confirmation rule, not learned by the model.
- **Synthetic video tests.** End-to-end video tests used clips assembled from test images; long real-world videos with camera motion, compression artefacts and weather effects should be added to the evaluation.
- **Single hardware configuration.** CPU timings were measured on one laptop; other CPUs will be faster or slower in proportion to their single-thread and vector performance.

CONCLUSION AND FUTURE WORK

This final chapter summarises the work carried out, restates the most important results quantitatively and qualitatively, and reflects on the contributions, strengths and limitations of the system. It also provides cost information, reflects on the design process, and sets out a programme of future work that would take the prototype towards a production deployment.

8.1 SUMMARY OF THE WORK

The project set out to shorten and structure the first step of the emergency response to road accidents: recognising that an accident has happened, how severe it is, and telling the right people with the right evidence. A class-balanced dataset of 18,000 images was prepared for three classes (fire, moderate and severe), and a YOLOv12n detector was fine-tuned on it. Around this model a complete, secure web platform was engineered with a layered client-server architecture: a FastAPI back end with JWT authentication, role-based access control and per-user data isolation; a normalised MySQL schema of seven tables; a background video pipeline with a live annotated stream and three-frame temporal confirmation; a fail-safe emergency agent that combines deterministic rules with an optional Gemini language model; e-mail and WhatsApp notifications; formal A4 PDF incident reports; and a responsive React dashboard for analysis, triage, analytics and administration. The system was verified with automated and manual tests and packaged for reproducible deployment with Docker Compose.

8.2 SUMMARY OF RESULTS

On the 1,800 held-out test images the model reached 89.9% mAP@0.5 and 79.0% mAP@0.5:0.95, with 92.5% precision and 86.8% recall; per class, mAP@0.5 was 99.4% for severe collisions, 97.3% for moderate collisions and 72.9% for fire. The 5.5 MB model runs in about 5.8 ms per image on a Tesla T4 GPU and 109 ms on an Intel Core i5 laptop CPU, and the system passed all 36 automated tests and all 40 system test cases.

Qualitatively, the system demonstrates that detection can be turned into an operational response: every confirmed incident produces an annotated evidence image, a severity-aware assessment with recommended services and actions, notification records for each channel, a printable report and an auditable status history, all visible to authorised operators within seconds.

8.3 CONTRIBUTIONS

The project makes the following contributions: (i) an application of the attention-centric YOLOv12 architecture to three-class, severity-aware accident and fire detection, with an honest per-class evaluation; (ii) a simple, explainable temporal confirmation rule for suppressing single-frame false alarms in video; (iii) a fail-safe pattern for integrating a large language model into a safety-critical notification path, in which the deterministic alert is always sent first; and (iv) a complete, tested and documented reference implementation of an accident-detection platform, including security controls, that can serve as a foundation for pilots and further research.

8.4 STRENGTHS OF THE SYSTEM

- **Accurate and severity-aware.** Near-perfect detection of moderate and severe collisions and no confusion between incident classes, so the recommended response is appropriate to the incident.
- **Lightweight and low-cost.** Runs in real time on a GPU and at about nine images per second on a laptop CPU, with no hardware purchase required.
- **Explainable.** Every alert carries bounding boxes, confidence values and an annotated image; the video confirmation rule is easy to understand and tune.
- **Robust.** Optional services (LLM, e-mail, WhatsApp) can fail without affecting detection and record keeping; failures are recorded rather than hidden.
- **Secure and private by design.** Authentication, roles, ownership filtering, private uploads, validated inputs and hardened headers, protected by regression tests.
- **Maintainable and reproducible.** Layered code, migrations, configuration through environment variables, automated tests, linting and a Docker-based deployment.

8.5 LIMITATIONS

The limitations below are stated objectively so that the system is not over-trusted and so that future work can be prioritised.

- **Fire recall.** The fire class has a test recall of 64.3% at object level (81.7% at image level); some fires, particularly small or smoke-obscured ones, may be missed.
- **Uploaded media only.** The system analyses uploaded images and video files; it does not yet ingest live RTSP camera streams, and the incident location is entered manually.

- **Single-process video jobs.** Video jobs are held in the memory of one back-end process; they are lost if the process restarts, and horizontal scaling requires a durable job queue.
- **Stream access control.** The live MJPEG stream is protected only by the unguessable job identifier rather than by the user's token, and annotated images in public storage are protected only by random file names.
- **No alert cooldown.** Repeated uploads of the same scene generate repeated alerts; a de-duplication or cooldown policy is not yet implemented.
- **Output video format.** Processed videos are written with the MPEG-4 Part 2 codec, which most browsers cannot play inline; the user interface therefore shows the live stream and key frame instead.
- **Session persistence.** For security the access token is kept only in browser memory, so reloading the page or opening a new tab requires the user to sign in again; a secure refresh-token mechanism would improve convenience.
- **Key-frame selection.** The key frame is the highest-confidence frame of any class, which may differ from the dominant class (Section 7.7).
- **Evaluation scope.** The model has not yet been evaluated on footage from local AJK camera networks, and live e-mail and WhatsApp delivery were verified with mocked providers.

8.6 COST INFORMATION

A deliberate goal of the project was to keep costs close to zero so that the solution is realistic for public-sector organisations with limited budgets. Table 8.1 summarises the costs incurred during development and the expected running costs.

Because the model runs on a CPU, a pilot deployment for a single control room does not require GPU hardware. The main recurring cost for a live deployment would be per-message charges for WhatsApp notifications, which scale with the number of incidents rather than with the number of cameras.

Table 8.1: Development and operating costs

Item	Used in this project	Cost
Model training compute	Kaggle notebook, Tesla T4 GPU, 2.45 hours	Free (within Kaggle's weekly GPU quota)
Development hardware	Existing student laptop (Core i5-1235U, 16 GB RAM)	No additional cost
Software and libraries	Python, PyTorch, Ultralytics, FastAPI, React, MySQL, Docker, ReportLab	Free (open source)
LLM assessment	Google Gemini API	Free tier for prototype volumes; optional

Item	Used in this project	Cost
WhatsApp notifications	Kapso WhatsApp gateway (mock mode during development)	Provider-dependent per-message pricing for live use
E-mail notifications	SMTP (e.g., Gmail app password)	Free for low volumes
Dataset	Merged image collections, cleaned and labelled by the team	No licence fee
Deployment host (pilot)	Any Linux machine with Docker; no GPU required	Existing machine; no GPU needed
Total development cash cost		Rs. 0

8.7 REFLECTIONS ON THE DESIGN PROCESS

The iterative, incremental process proved well suited to a project that mixed machine-learning experimentation with software engineering. Building a thin end-to-end pipeline early, from upload to stored result, made cross-cutting dependencies visible early; for example, the order of the class labels must be identical in the training data, the model file and the back end, which led to an explicit class contract protected by an automated test. Reviews with the supervisor repeatedly shifted attention from raw accuracy to operational concerns: false alarms in video led to the confirmation rule, and the question ‘what happens if the AI service is down?’ led to the fail-safe agent design.

A dedicated hardening increment at the end was valuable. A structured code review uncovered problems that functional tests had not revealed, among them publicly reachable incident PDFs with predictable names, a server that could stall while analysing a large image, and a job registry that grew without bound. Each fix was accompanied by a regression test. In hindsight, security and operational requirements should have been written as explicit, testable requirements from the first increment rather than addressed at the end. Similarly, collecting a small evaluation set from the intended deployment environment early would have given earlier warning about domain shift.

8.8 FUTURE WORK

The following work is recommended, in approximate order of priority:

1. **Improve fire detection.** Re-audit flame annotations for consistency (one box per fire region), add hard negative examples such as headlights, sunsets and reflections, add a smoke class, and evaluate a larger variant (YOLOv12s) or test-time augmentation for this class.
2. **Live camera ingestion.** Add RTSP and HLS stream ingestion with per-camera configuration and geolocation, so that incidents are detected without manual upload and the location is filled in automatically.

3. **Temporal models.** Complement the confirmation rule with tracking or a lightweight temporal model to capture collision dynamics and to anticipate accidents.
4. **Durable, scalable processing.** Move video jobs to a persistent queue (e.g., Redis-based workers) and protect the live stream with the same token and ownership checks as other endpoints.
5. **Alert de-duplication.** Introduce a per-location cooldown and incident merging so that the same scene does not trigger repeated alerts.
6. **Local evaluation and pilot.** Collect and label footage from AJK roads, measure accuracy and false-alarm rates in the field, and run a supervised pilot with a rescue service, including live WhatsApp and e-mail delivery and recipient consent.
7. **Media governance.** Define retention and deletion policies, require authentication for all evidence files, and add face and number-plate blurring for privacy.
8. **Browser-playable output and key frames.** Encode processed videos in H.264 and choose the key frame from the dominant class.
9. **Edge deployment.** Export the model to ONNX or TensorRT for deployment on low-cost edge devices placed at accident-prone locations.
10. **Mobile application.** Provide a companion mobile app for responders with push notifications, maps and one-tap acknowledgement.

8.9 CONCLUDING REMARKS

This project has shown that a modern, lightweight detector, combined with careful system engineering, can turn camera images into fast, explainable and auditable accident alerts on inexpensive hardware. The system detects severe and moderate collisions with near-perfect accuracy, never confused the severity classes in evaluation, and delivers a complete response workflow from detection to notification and documentation. Its limitations, above all the lower recall for fire, are well understood and point to concrete next steps. With live camera ingestion and a field pilot, the system could become a practical tool for reducing the time between a road accident and the arrival of help.

REFERENCES

- [1] World Health Organization, *Global Status Report on Road Safety 2023*. Geneva, Switzerland: World Health Organization, 2023.
- [2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.
- [3] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, “A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS,” *Mach. Learn. Knowl. Extr.*, vol. 5, no. 4, pp. 1680–1716, 2023, doi: 10.3390/make5040083.
- [4] Y. Tian, Q. Ye, and D. Doermann, “YOLOv12: Attention-centric real-time object detectors,” arXiv:2502.12524, Feb. 2025.
- [5] E. P. Ijjina, D. Chand, S. Gupta, and K. Goutham, “Computer vision-based accident detection in traffic surveillance,” in *Proc. 10th Int. Conf. Comput., Commun. Netw. Technol. (ICCCNT)*, 2019, pp. 1–6, doi: 10.1109/ICCCNT45670.2019.8944469.
- [6] H. Ghahremannezhad, H. Shi, and C. Liu, “Real-time accident detection in traffic surveillance using deep learning,” in *Proc. IEEE Int. Conf. Imaging Syst. Techn. (IST)*, 2022, pp. 1–6, doi: 10.1109/IST55454.2022.9827736.
- [7] J. Fang, J. Qiao, J. Xue, and Z. Li, “Vision-based traffic accident detection and anticipation: A survey,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 34, no. 4, pp. 1983–1999, Apr. 2024, doi: 10.1109/TCSVT.2023.3307655.
- [8] J. White, C. Thompson, H. Turner, B. Dougherty, and D. C. Schmidt, “WreckWatch: Automatic traffic accident detection and notification with smartphones,” *Mobile Netw. Appl.*, vol. 16, no. 3, pp. 285–303, 2011, doi: 10.1007/s11036-011-0304-8.
- [9] J. Zaldivar, C. T. Calafate, J. C. Cano, and P. Manzoni, “Providing accident detection in vehicular networks through OBD-II devices and Android-based smartphones,” in *Proc. IEEE 36th Conf. Local Comput. Netw. (LCN)*, 2011, pp. 813–819, doi: 10.1109/LCN.2011.6115556.
- [10] F.-H. Chan, Y.-T. Chen, Y. Xiang, and M. Sun, “Anticipating accidents in dashcam videos,” in *Computer Vision – ACCV 2016*, Lecture Notes in Computer Science. Cham, Switzerland: Springer, 2017, pp. 136–153, doi: 10.1007/978-3-319-54190-7_9.
- [11] W. Bao, Q. Yu, and Y. Kong, “Uncertainty-based traffic accident anticipation with spatio-temporal relational learning,” in *Proc. 28th ACM Int. Conf. Multimedia*, 2020, pp. 2682–2690, doi: 10.1145/3394171.3413827.
- [12] R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2014, pp. 580–587, doi: 10.1109/CVPR.2014.81.
- [13] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017, doi: 10.1109/TPAMI.2016.2577031.
- [14] W. Liu *et al.*, “SSD: Single shot multibox detector,” in *Computer Vision – ECCV 2016*, Lecture Notes in Computer Science. Cham, Switzerland: Springer, 2016, pp. 21–37, doi: 10.1007/978-3-319-46448-0_2.

- [15] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2999–3007, doi: 10.1109/ICCV.2017.324.
- [16] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal speed and accuracy of object detection,” arXiv:2004.10934, Apr. 2020.
- [17] A. Wang *et al.*, “YOLOv10: Real-time end-to-end object detection,” arXiv:2405.14458, May 2024.
- [18] G. Jocher, J. Qiu, and A. Chaurasia, “Ultralytics YOLO,” software, version 8, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [19] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017, pp. 5998–6008. arXiv:1706.03762.
- [20] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with transformers,” in *Computer Vision – ECCV 2020*, Lecture Notes in Computer Science. Cham, Switzerland: Springer, 2020, pp. 213–229, doi: 10.1007/978-3-030-58452-8_13.
- [21] Y. Zhao *et al.*, “DETRs beat YOLOs on real-time object detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 16965–16974, doi: 10.1109/CVPR52733.2024.01605.
- [22] K. Muhammad, J. Ahmad, I. Mehmood, S. Rho, and S. W. Baik, “Convolutional neural networks based fire detection in surveillance videos,” *IEEE Access*, vol. 6, pp. 18174–18183, 2018, doi: 10.1109/ACCESS.2018.2812835.
- [23] T.-Y. Lin *et al.*, “Microsoft COCO: Common objects in context,” in *Computer Vision – ECCV 2014*, Lecture Notes in Computer Science. Cham, Switzerland: Springer, 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.
- [24] Gemini Team, Google, “Gemini: A family of highly capable multimodal models,” arXiv:2312.11805, Dec. 2023.
- [25] *IEEE Standard for Information Technology—Systems Design—Software Design Descriptions*, IEEE Std 1016-2009, 2009, doi: 10.1109/IEEESTD.2009.5167255.

APPENDIX A: API REFERENCE

All endpoints except registration, login and health require an Authorization: Bearer <token> header. Non-administrators only ever receive their own data. Responses are JSON unless stated otherwise.

Table A.1: Complete REST API reference

Method	Path	Access	Description and main parameters
POST	/auth/register	Public	Body {username, email, password}; creates a viewer; 201
POST	/auth/login	Public	Form {username, password}; returns {access_token, token_type, user}
GET	/auth/me	User	Current user profile
POST	/detection/image	User	Multipart file; query location; returns class, confidence, boxes, processed_url, alert_triggered
POST	/detection/video	User	Multipart file; query location; returns {job_id, status: queued}
GET	/detection/video/jobs/{job_id}	Owner	Status, stage, progress, processed and total frames, result, error
GET	/detection/video/jobs/{job_id}/stream	Job ID	multipart/x-mixed-replace MJPEG stream of annotated frames
GET	/detection/history	Owner	Query skip, limit (1–100); list of incidents with assessment summary
GET	/detection/history/{id}	Owner	Single incident
PATCH	/detection/history/{id}/status	Owner	Query status $\in$ {open, acknowledged, false_alarm, resolved}
DELETE	/detection/history/{id}	Admin	Delete incident and child records; 204
GET	/agent/status	User	Agent, Gemini and WhatsApp configuration flags
GET	/agent/report/{id}	Owner	Assessment, or a PENDING placeholder while in progress
GET	/agent/report/{id}/pdf	Owner	application/pdf incident report
GET	/agent/reports	Owner	Query skip, limit, whatsapp_only
POST	/agent/test-whatsapp	Admin	Sends a test WhatsApp message; 503 on provider failure
GET	/alerts	Owner	E-mail alert log (skip, limit)
GET	/alerts/calls	Owner	Call-audit log (skip, limit)
POST	/alerts/test-email	Admin	Sends a test e-mail; returns status sent / failed / skipped
POST	/alerts/test-call	Admin	Records a local simulated call entry
GET	/dashboard/stats	Owner	KPIs, class counts and five recent events
GET	/dashboard/timeline	Owner	Query days (1–90); per-day per-class counts
GET	/dashboard/confidence-distribution	Owner	Ten confidence buckets with counts

Method	Path	Access	Description and main parameters
GET	/users	Admin	List users (skip, limit)
PUT	/users/{id}	Admin	Body {role?, is_active?}; cannot demote or deactivate self
DELETE	/users/{id}	Admin	Deactivates the account; cannot target self
GET	/health	Public	{status, model, model_loaded, version, agent}

APPENDIX B: CONFIGURATION PARAMETERS

All parameters are read from environment variables or from an untracked .env file. Secrets are shown here only by name.

Table B.1: Principal configuration parameters and defaults

Parameter	Default	Meaning
DATABASE_URL	(required)	SQLAlchemy URL of the MySQL database
SECRET_KEY	(required in production)	Key used to sign JWTs
ACCESS_TOKEN_EXPIRE_MINUTES	60	Token lifetime
CORS_ORIGINS / TRUSTED_HOSTS	localhost values	Allowed browser origins and host headers
ENABLE_DOCS	false	Expose interactive API documentation
MODEL_PATH	ml_model/best.pt	Location of the trained weights
DETECTION_CONFIDENCE / IOU_THRESHOLD / IMG_SIZE	0.45 / 0.45 / 640	Inference thresholds and input size
VIDEO_FRAME_STRIDE / VIDEO_CONFIRMATION_FRAMES	2 / 3	Frame sampling and temporal confirmation
MAX_VIDEO_FRAMES / MAX_UPLOAD_SIZE_MB / MAX_IMAGE_PIXELS	9,000 / 100 / 25,000,000	Resource limits
SMTP_HOST / SMTP_PORT / SMTP_USER / SMTP_PASSWORD / ALERT_EMAIL_TO	smtp.gmail.com / 587 / — / — / —	E-mail alerting (disabled when empty)
FRONTEND_URL	http://localhost:5173	Dashboard link inserted in alert e-mails
AGENT_ENABLED / GEMINI_API_KEY / GEMINI_MODEL / AGENT_TIMEOUT_SECONDS	true / — / Gemini Flash / 90	Emergency agent and LLM
WHATSAPP_ENABLED / WHATSAPP_MODE	true / mock	WhatsApp dispatch; mode mock or kapso
KAPSO_API_KEY / KAPSO_PHONE_NUMBER_ID / KAPSO_WHATSAPP_TO	—	Kapso gateway credentials and recipient
WHATSAPP_INCLUDE_MEDIA / PUBLIC_BASE_URL	false / —	Attach report image (requires a public HTTPS URL)
UPLOAD_DIR / PROCESSED_DIR / SNAPSHOTS_DIR / REPORTS_DIR	data/uploads, static/...	Storage locations

APPENDIX C: INSTALLATION AND USER GUIDE

Running with Docker (recommended).

1. Install Docker Engine with Docker Compose.
2. Copy the trained model to backend/ml_model/best.pt.
3. Create a root .env file containing SECRET_KEY, MYSQL_ROOT_PASSWORD, MYSQL_PASSWORD, MYSQL_DATABASE and MYSQL_USER, and create backend/.env (optionally with SMTP, Gemini and Kapso settings).
4. Run docker compose up --build and wait until the three containers are healthy.
5. Open <http://localhost> in a browser; the API is available at <http://localhost:8000> and its health at /health.

Using the system.

1. Sign in with the administrator account and immediately change its default password; create accounts for operators or ask them to register and then assign roles on the Users page.
2. To analyse an image, open Image Analysis, drop an image, optionally enter the location and press Run detection. Review the verdict, the annotated evidence and the AI assessment, and download the PDF if needed.
3. To analyse a video, open Video Analysis, drop a video, optionally enter the location and press Process video. Watch the live stream and progress; the result and assessment appear when processing completes.
4. Review incidents on the Incidents page; use the filters and acknowledge incidents or mark false alarms after verification.
5. Monitor delivery on the Alerts Center page and trends on the Dashboard and Analytics pages. Administrators can send test e-mail and WhatsApp messages from the Alerts Center.

Development setup. Create a Python virtual environment in backend/, install requirements.txt, apply the database migrations with Alembic and start Uvicorn; in frontend/, install the Node.js dependencies and start the Vite development server with VITE_API_URL pointing to the API. Run pytest in backend/ and npm run lint and npm run build in frontend/ before each change is integrated.

APPENDIX D: PROJECT TIMELINE AND WORK DIVISION

Table D.1: Project timeline by increment

No.	Increment and main deliverables	Period	Status
1	Proposal, literature review, dataset curation and YOLOv12n training	April 2026	Completed
2	FastAPI inference service, MySQL schema and migrations	May 2026	Completed
3	React dashboard; image and video workflows; live streaming	August 2026 (weeks 1–2)	Completed
4	E-mail and WhatsApp alerts, emergency agent, PDF reports	August 2026 (weeks 3–4)	Completed
5	Security hardening, testing, UI redesign, report writing	September 2026	Completed

Table D.2: Work division among group members

Member	Main responsibilities
Adeel Danish (FA22-BSE-001)	Requirements gathering and use-case descriptions; UML and data-flow diagrams; manual system testing of the user interface; report formatting and proof-reading
Syed Ali Ibtissam Bukhari (FA22-BSE-023)	Literature review; dataset collection, cleaning and label verification; model evaluation and result analysis; test-case documentation and presentation material
Syed Qosain Hyder Bukhari (FA22-BSE-025)	Main developer: YOLOv12n training and integration; FastAPI back end, MySQL database and video pipeline; emergency agent, e-mail and WhatsApp alerts and PDF reports; React dashboard; security hardening, automated tests and Docker deployment